Metric Markov cotype two of ℓ_1 with $C^2 = 3024$: a machine-checked proof

Abstract

We present a Lean 4 / Mathlib formalization of the statement that the real space ℓ_1 has metric Markov cotype two with constant $C^2 = 3024$, that is, $N_2(\ell_1) \le 12\sqrt{21}$. For every reversible stochastic matrix A on $n \ge 1$ states with probability vector π, every $t \ge 1$ and all $x_1, \dots, x_n \in \ell_1$, there are $y_1, \dots, y_n \in \ell_1$ with $\sum_i \pi_i \|x_i - y_i\|_1^2 + t \sum_{i,j} \pi_i a_{ij} \|y_i - y_j\|_1^2 \le 3024 \sum_{i,j} \pi_i (\frac{1}{t} \sum_{s=1}^{t} A^s)_{ij} \|x_i - x_j\|_1^2$. The formal proof follows the source argument: cut decomposition, the cubic map $3r^2 - 2r^3$ with a quartic potential, a resolvent-harmonic extension, and a comparison of geometric and Cesàro averages. The martingale and the random geometric time are replaced by deterministic sums against the resolvent matrix. The whole proof is checked by Lean; the constant is $108 \cdot 28 = 3024$.

Source	openai/math@fd4aeeb, family 332, OpenAIMath_332_MetricMarkovCotypeL1
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no `sorry`; statement matches target; module build
Axioms	`propext`, `Classical.choice`, `Quot.sound`
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Compute	83 AIProver sessions in 63 jobs, 69 session-hours; 17 GPU-hours on 2 x NVIDIA GH200 (H100, 96 G
Hosted models	34 calls to claude-opus-5-5, claude-sonnet-5-5, $11.67 up to the final review
Run	`srv_20261013_195822_OpenAIMath_332_MetricMarkovCotypeL1`

Contents

1 Statement **1**

2 Formalization **2**

3 Proof **4**

3.1 Overview 4

3.2 Lemmas 6

3.3 Proof of the theorem 53

4 Independent review **54**

A Complete Lean file **55**

1 Statement

Let ℓ_1 be the real Banach space of real sequences $x = (x(k))_k$ with $\|x\|_1 = \sum_k |x(k)| < \infty$. Write $[n]$ for an n-element index set.

A matrix $A = (a_{ij})_{i,j\in[n]}$ is *stochastic* if its entries are nonnegative and every row sums to 1. A *probability vector* $\pi = (\pi_i)_{i\in[n]}$ has nonnegative entries summing to 1; zero entries are allowed. A is *reversible* with respect to π if $\pi_i a_{ij} = \pi_j a_{ji}$ for all i, j. A^s denotes the s-th matrix power.

Theorem 1.1 (metric Markov cotype two of ℓ_1, $C^2 = 3024$)**.** *Let $n, t \geq 1$ be integers, π a probability vector on $[n]$, A a stochastic matrix on $[n]$ reversible with respect to π, and $x_1, \dots, x_n \in \ell_1$. Then there exist $y_1, \dots, y_n \in \ell_1$ with*

$$\sum_i \pi_i \|x_i - y_i\|_1^2 + t \sum_{i,j} \pi_i a_{ij} \|y_i - y_j\|_1^2 \leq 3024 \sum_{i,j} \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A^s\Big)_{ij} \|x_i - x_j\|_1^2.$$

The source proof uses reversibility only through stationarity, $\pi A = \pi$.

2 Formalization

The formalization uses Mathlib. It relies on Mathlib's space ℓ^p (the type `lp`), its finite and infinite sums, and its matrix powers. No other earlier results are used.

Index sets. The index set $[n]$ is Lean's $\{0, 1, \dots, n-1\}$. $\mathcal{P}([n])$ denotes the set of *all* subsets of $[n]$, including $\emptyset$ and $[n]$. For $B \in \mathcal{P}([n])$ and $i \in [n]$ put $\mathbf{1}_B(i) = 1$ if $i \in B$ and 0 otherwise. We write δ_{ij} for the entries of the identity matrix.

Infinite sums. We say that a series $\sum_s f(s)$ of reals *converges to a* if it converges unconditionally (Mathlib's `HasSum`). For real series this is equivalent to absolute convergence with sum a. When a hypothesis contains $\sum_{s=0}^{\infty} f(s)$, this is Mathlib's `tsum`. It equals the sum when f is summable and is 0 otherwise. Every such series below is shown to be summable (Lemma 3.16).

The space ℓ_1. The space is Mathlib's ℓ^1 of real sequences indexed by $\mathbb{N} = \{0, 1, 2, \dots\}$. Its elements are sequences x with $\sum_k |x(k)| < \infty$, with coordinatewise operations and norm $\|x\| = \sum_{k\geq 0} |x(k)|$. Shifting the index from $\{1, 2, \dots\}$ to $\{0, 1, \dots\}$ is an isometric relabeling.

Lean: `L1Space`

```
/-- The real Banach space `ℓ₁` of absolutely summable real sequences (indexed by `ℕ`). -/
abbrev L1Space : Type := lp (fun _ : ℕ => ℝ) 1
```

Definition 2.1. A vector $\pi : [n] \to \mathbb{R}$ is a probability vector if $\pi_i \geq 0$ for all i and $\sum_i \pi_i = 1$.

Lean: `IsProbVec`

```
/-- `π` is a probability vector on `[n]`: nonnegative entries summing to one. -/
def IsProbVec {n : ℕ} (π : Fin n → ℝ) : Prop :=
  (∀ i, 0 ≤ π i) ∧ ∑ i, π i = 1
```

Definition 2.2. A matrix $A \in \mathbb{R}^{[n]\times[n]}$ is stochastic if $A_{ij} \geq 0$ for all i, j and $\sum_j A_{ij} = 1$ for all i.

Lean: IsStochasticMatrix

```
/-- `A` is a stochastic matrix: nonnegative entries and all row sums equal to one. -/
def IsStochasticMatrix {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  (∀ i j, 0 ≤ A i j) ∧ ∀ i, ∑ j, A i j = 1
```

Definition 2.3. A is reversible with respect to π if $\pi_i A_{ij} = \pi_j A_{ji}$ for all $i, j \in [n]$.

Lean: IsReversibleWrt

```
/-- `A` is reversible with respect to `π`: `π i * a_ij = π j * a_ji` for all `i, j`. -/
def IsReversibleWrt {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  ∀ i j, π i * A i j = π j * A j i
```

Three facts about the setting are proved with the definitions.

Lemma 2.4 (setting_consistent)**.** *For $n = 1$, the vector $\pi = (1)$ is a probability vector, and the 1×1 identity matrix is stochastic and reversible with respect to π.*

Lean: setting_consistent

```
theorem setting_consistent :
    IsProbVec (fun _ : Fin 1 => (1:ℝ)) ∧ IsStochasticMatrix (1 : Matrix (Fin 1) (Fin 1) ℝ) ∧
      IsReversibleWrt (fun _ : Fin 1 => (1:ℝ)) (1 : Matrix (Fin 1) (Fin 1) ℝ) := by
  refine ⟨⟨fun _ => zero_le_one, by simp⟩, ⟨fun i j => ?_, fun i => ?_⟩, fun i j => ?_⟩
  · rw [Matrix.one_apply]
    split_ifs <;> norm_num
  · rw [Fin.sum_univ_one, Subsingleton.elim i 0, Matrix.one_apply_eq]
  · have h : i = j := Subsingleton.elim i j
    subst h
    rfl
```

Proof. All conditions reduce to $1 \geq 0$, $1 = 1$ and $1 \cdot 1 = 1 \cdot 1$. □

Lemma 2.5 (setting_stationary)**.** *Let $\pi : [n] \to \mathbb{R}$ be arbitrary and let A be stochastic and reversible with respect to π. Then $\sum_i \pi_i A_{ij} = \pi_j$ for every j.*

Lean: setting_stationary

```
theorem setting_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (j : Fin n) :
    ∑ i, π i * A i j = π j := by
  calc ∑ i, π i * A i j = ∑ i, π j * A j i := Finset.sum_congr rfl (fun i _ => hrev i j)
    _ = π j * ∑ i, A j i := by rw [Finset.mul_sum]
    _ = π j := by rw [hA.2 j, mul_one]
```

Proof. By reversibility, $\sum_i \pi_i A_{ij} = \sum_i \pi_j A_{ji} = \pi_j \sum_i A_{ji} = \pi_j$, using the row sum of row j. □

Lemma 2.6 (setting_stationary_pow)**.** *Under the same hypotheses, $\sum_i \pi_i (A^s)_{ij} = \pi_j$ for every $s \in \mathbb{N}$ and every j.*

Lean: `setting_stationary_pow`

```
theorem setting_stationary_pow {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (s : ℕ) (j : Fin n) :
    ∑ i, π i * (A ^ s) i j = π j := by
  induction s generalizing j with
  | zero => simp [Matrix.one_apply]
  | succ k ih =>
    rw [pow_succ]
    simp only [Matrix.mul_apply, Finset.mul_sum]
    rw [Finset.sum_comm]
    calc ∑ l, ∑ i, π i * ((A ^ k) i l * A l j)
        = ∑ l, (∑ i, π i * (A ^ k) i l) * A l j := by
          refine Finset.sum_congr rfl (fun l _ => ?_)
          rw [Finset.sum_mul]
          refine Finset.sum_congr rfl (fun i _ => ?_)
          ring
      _ = ∑ l, π l * A l j := by simp only [ih]
      _ = ∑ l, π j * A j l := Finset.sum_congr rfl (fun l _ => hrev l j)
      _ = π j * ∑ l, A j l := by rw [Finset.mul_sum]
      _ = π j := by rw [hA.2 j, mul_one]
```

Proof. Induction on s. The case $s = 0$ is $\sum_i \pi_i \delta_{ij} = \pi_j$.

For the step, write $A^{s+1} = A^s A$. Exchanging sums gives

$$\sum_i \pi_i (A^{s+1})_{ij} = \sum_l \Big(\sum_i \pi_i (A^s)_{il} \Big) A_{lj} = \sum_l \pi_l A_{lj}.$$

The last expression equals π_j by the argument of Lemma 2.5. □

The target statement. Lean: target statement `openaimath_332_metricmarkovcotypel1`, fixed before the proof; `solution` is checked to have exactly this type

```
theorem openaimath_332_metricmarkovcotypel1
    (n t : ℕ) (hn : 0 < n) (ht : 0 < t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (hrev : IsReversibleWrt π A)
    (x : Fin n → L1Space) :
    ∃ y : Fin n → L1Space,
      (∑ i, π i * ‖x i - y i‖ ^ 2)
        + (t : ℝ) * (∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2)
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
```

The target asserts the following; this is also the statement proved by `solution` in the last subsection. Let n, t be natural numbers with $0 < n$ and $0 < t$. Let $\pi : [n] \to \mathbb{R}$ be a probability vector, A a stochastic matrix reversible with respect to π, and $x : [n] \to \ell_1$. Then there exists $y : [n] \to \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_i \sum_j \pi_i A_{ij} \|y_i - y_j\|^2 \le 3024 \sum_i \sum_j \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} (A^s)_{ij} \Big) \|x_i - x_j\|^2.$$

This is exactly the source theorem:

- The hypotheses $0 < n$ and $0 < t$ encode $n, t \in \{1, 2, \dots\}$.
- $\|\cdot\|$ is the ℓ_1 norm.
- The entry of the Cesàro average is written as $\frac{1}{t}\sum_{s=1}^{t}(A^s)_{ij}$.

Reversibility is kept as a hypothesis, as in the source. The proof uses it only through Lemma 2.6. The more general statement for stationary matrices (the source's closing remark) is not formalized.

3 Proof

3.1 Overview

Throughout, $q = t/(t+1)$ and $p = 1 - q = 1/(t+1)$, as in the source. The *resolvent matrix* is

$$G_{ij} = \sum_{s \geq 0} (1-q) q^s (A^s)_{ij}.$$

In probabilistic terms, $G_{ij} = \mathbb{P}_i(X_S = j)$ for the geometric time S of the source.

Cut decomposition (source Lemma 1).

- Lemmas 3.31 to 3.35 treat one real coordinate. They decompose $a : [n] \to \mathbb{R}$ into nested upper cuts.
- Lemmas 3.36, 3.37 and 3.41 to 3.44 assemble the coordinates into vectors $\beta_B \in \ell_1$. They produce the binary codes z_i, the weights $w_B = \|\beta_B\|$ and the 1-Lipschitz affine map T.

Deviations from the source:

- The cut vectors are built by induction on the number of distinct values, not by the explicit formula $(\min_B - \max_{B^c})_+$.
- All subsets $B \in \mathcal{P}([n])$ are kept, with weights $w_B \geq 0$ that may vanish, so there is no separate "all points coincide" case.
- The base point of T is x_{i_0} for the first index i_0, rather than the coordinatewise minimum. This is why $n > 0$ is needed.
- The identity $\|x_i - x_j\|_1 = \|z_i - z_j\|_H^2$ is not stated separately. It is used in Lemma 3.40 through $(z_{iB} - z_{kB})^2 = |z_{kB} - z_{iB}|$ for bits.
- No Hilbert space is introduced. All quantities are explicit weighted sums.

The flat map (source Lemma 2). Lemmas 3.24 to 3.26, with $\phi(r) = 3r^2 - 2r^3$. The scalar estimate is proved algebraically from

$$\phi(r') - \phi(r) = \delta\big(6r(1-r) + \delta(3 - 4r - 2r')\big), \qquad \delta = r' - r,$$

instead of by integration.

Martingale step (source Corollary 3 and Section 4.1). There is no probability space, martingale or filtration. Instead:

- Lemmas 3.27 and 3.28 state the one-step inequality for a convex combination. The balance relation $u = pv + \sum_j c_j U_j$ plays the role of the martingale property, and the linear terms cancel exactly.
- Lemma 3.29 sums these one-step inequalities with weights $\pi_k G_{ki}$, where k is the starting state (the centre $e = z_k$) and i the current live state. It telescopes the quartic potential with the identity $G = pI + qGA$ (Lemma 3.23) and stationarity $\pi G = \pi$ (Lemma 3.21). This is the source's computation of $\mathbb{E}\sum_m$ along the killed chain, multiplied by p.
- The source's exact identity $B + tE_A$ is replaced by the decomposition of Lemma 3.38 and the division by p in Lemma 3.39.

The resolvent facts are Lemmas 3.16 and 3.18 to 3.23. The harmonic extension $h = Gz$ is Lemma 3.30. The assembly is Lemmas 3.40 and 3.45.

Comparison of costs (source Lemma 4). The random time S is replaced by the geometric weights $(1-q)q^s$, and J, R by $\lfloor s/t \rfloor$ and $s \bmod t$:

- subadditivity of $\sqrt{D}$: Lemmas 3.1 to 3.5 and 3.7;
- the bounds $D(t) \le 4W$ and the block estimate: Lemmas 3.10 and 3.11;
- the geometric moments: Lemmas 3.6, 3.8, 3.9, 3.12 and 3.13;
- the conclusion: Lemmas 3.14, 3.15 and 3.17.

The bound $\mathbb{P}(R = r) \le 2/t$ becomes the closed form for $\sum_s q^s f(s \bmod t)$.

Hypotheses actually used. Only $\pi_i \ge 0$ is used; the normalization $\sum_i \pi_i = 1$ never enters the proof. Reversibility enters only through Lemma 2.6, in Lemmas 3.5 and 3.21.

3.2 Lemmas

Lemma 3.1 (stoch_pow). *If A is stochastic, then A^s is stochastic for every $s \in \mathbb{N}$.*

Lean: stoch_pow — proved by AIProver, job 1, session 0

```
theorem stoch_pow {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (s : ℕ) :
    IsStochasticMatrix (A ^ s) := by
  induction s with
  | zero =>
    refine ⟨?_, ?_⟩
    · intro i j
      by_cases h : i = j
      · subst h; simp
      · simp [h]
    · intro i
      simp [Matrix.one_apply]
  | succ s ih =>
    rw [pow_succ]
    rcases hA with ⟨hA_nonneg, hA_row⟩
```

```
    rcases ih with ⟨hB_nonneg, hB_row⟩
    refine ⟨?_, ?_⟩
    · intro i j
      simp only [Matrix.mul_apply]
      refine Finset.sum_nonneg (fun k _ => ?_)
      have hk1 : 0 ≤ (A ^ s) i k := hB_nonneg i k
      have hk2 : 0 ≤ A k j := hA_nonneg k j
      nlinarith
    · intro i
      simp only [Matrix.mul_apply]
      calc
       ∑ j, ∑ k : Fin n, ((A ^ s) i k) * A k j = ∑ k : Fin n, ((A ^ s) i k) * (∑ j : Fin n, A k j) := by
         rw [Finset.sum_comm]
         refine Finset.sum_congr rfl (fun k _ => ?_)
         rw [Finset.mul_sum]
       _ = ∑ k : Fin n, ((A ^ s) i k) * 1 := by
         refine Finset.sum_congr rfl (fun k _ => ?_)
         rw [hA_row k]
       _ = ∑ k : Fin n, (A ^ s) i k := by simp
       _ = 1 := hB_row i
```

Proof. Induction on s. The identity matrix is stochastic.

For the step, write $A^{s+1} = A^s A$.

- Its entries $\sum_k (A^s)_{ik} A_{kj}$ are sums of products of nonnegative numbers.
- Its row sums are $\sum_k (A^s)_{ik} \sum_j A_{kj} = \sum_k (A^s)_{ik} = 1$.

□

Lemma 3.2 (weighted_minkowski). *Let ι be a finite set and $c, f, g, F : \iota \to \mathbb{R}$ nonnegative with $F_i \le f_i + g_i$ for all i. Then*

$$\sqrt{\textstyle\sum_i c_i F_i^2} \le \sqrt{\textstyle\sum_i c_i f_i^2} + \sqrt{\textstyle\sum_i c_i g_i^2}.$$

Lean: weighted_minkowski — proved by AIProver, job 1, session 0

```
theorem weighted_minkowski {ι : Type*} [Fintype ι] (c f g F : ι → ℝ)
    (hc : ∀ i, 0 ≤ c i) (hf : ∀ i, 0 ≤ f i) (hg : ∀ i, 0 ≤ g i) (hF0 : ∀ i, 0 ≤ F i)
    (hF : ∀ i, F i ≤ f i + g i) :
   Real.sqrt (∑ i, c i * F i ^ 2) ≤ Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2) := by
  have h_sum_cF_sq : 0 ≤ ∑ i, c i * F i ^ 2 :=
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_sum_cf_sq : 0 ≤ ∑ i, c i * f i ^ 2 :=
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_sum_cg_sq : 0 ≤ ∑ i, c i * g i ^ 2 :=
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_nonneg_prod : 0 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) :=
    mul_nonneg h_sum_cf_sq h_sum_cg_sq


  have h_cauchy : (∑ i, c i * f i * g i) ^ 2 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) := by
    set u := fun i => Real.sqrt (c i) * f i with hu
    set v := fun i => Real.sqrt (c i) * g i with hv
    have h_cs := Finset.sum_mul_sq_le_sq_mul_sq (Finset.univ : Finset ι) u v
```

```
    have hu_sq : ∀ i, u i ^ 2 = c i * f i ^ 2 := by
      intro i
      dsimp [u]
      calc
        (Real.sqrt (c i) * f i) ^ 2 = (Real.sqrt (c i)) ^ 2 * (f i) ^ 2 := by ring
        _ = c i * (f i) ^ 2 := by rw [Real.sq_sqrt (hc i)]
    have hv_sq : ∀ i, v i ^ 2 = c i * g i ^ 2 := by
      intro i
      dsimp [v]
      calc
        (Real.sqrt (c i) * g i) ^ 2 = (Real.sqrt (c i)) ^ 2 * (g i) ^ 2 := by ring
        _ = c i * (g i) ^ 2 := by rw [Real.sq_sqrt (hc i)]
    have huv : ∀ i, u i * v i = c i * f i * g i := by
      intro i
      dsimp [u, v]
      calc
        (Real.sqrt (c i) * f i) * (Real.sqrt (c i) * g i) = (Real.sqrt (c i) * Real.sqrt (c i)) * (f i * g i)
↪   := by ring
        _ = c i * (f i * g i) := by rw [Real.mul_self_sqrt (hc i)]
        _ = c i * f i * g i := by ring
    have hsum_u_sq : (∑ i : ι, u i ^ 2) = ∑ i, c i * f i ^ 2 :=
      Finset.sum_congr rfl (fun i _ => by rw [hu_sq i])
    have hsum_v_sq : (∑ i : ι, v i ^ 2) = ∑ i, c i * g i ^ 2 :=
      Finset.sum_congr rfl (fun i _ => by rw [hv_sq i])
    have hsum_uv : (∑ i : ι, u i * v i) = ∑ i, c i * f i * g i :=
      Finset.sum_congr rfl (fun i _ => by rw [huv i])
    simpa [hsum_uv, hsum_u_sq, hsum_v_sq] using h_cs
  have h_bound : ∑ i, c i * f i * g i ≤ Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2)) := by
    have h_sq : (∑ i, c i * f i * g i) ^ 2 ≤ (Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2))) ^
↪   2 := by
      calc
        (∑ i, c i * f i * g i) ^ 2 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) := h_cauchy
        _ = (Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2))) ^ 2 := by
          rw [Real.sq_sqrt h_nonneg_prod]
    have h_nonneg_sqrt : 0 ≤ Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2)) :=
      Real.sqrt_nonneg _
    have h_nonneg_sum : 0 ≤ ∑ i, c i * f i * g i :=
      Finset.sum_nonneg fun i _ => mul_nonneg (mul_nonneg (hc i) (hf i)) (hg i)
    nlinarith
  have h_goal_sq : (∑ i, c i * F i ^ 2) ≤ (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^
↪   2)) ^ 2 := by
    calc
      (∑ i, c i * F i ^ 2) ≤ (∑ i : ι, c i * ((f i + g i) ^ 2)) := by
        refine Finset.sum_le_sum fun i _ => ?_
        have hF_i : F i ^ 2 ≤ (f i + g i) ^ 2 := by
          have hF_nonneg : 0 ≤ F i := hF0 i
          have h_fg_nonneg : 0 ≤ f i + g i := add_nonneg (hf i) (hg i)
          have h_le : F i ≤ f i + g i := hF i
          nlinarith
        exact mul_le_mul_of_nonneg_left hF_i (hc i)
      _ = ∑ i : ι, c i * (f i ^ 2 + 2 * f i * g i + g i ^ 2) := by
        refine Finset.sum_congr rfl (fun i _ => ?_)
        ring
      _ = (∑ i, c i * f i ^ 2) + (∑ i, c i * g i ^ 2) + 2 * (∑ i, c i * f i * g i) := by
        simp only [Finset.sum_add_distrib, mul_add, Finset.mul_sum]
        ring
      _ ≤ (∑ i, c i * f i ^ 2) + (∑ i, c i * g i ^ 2) + 2 * Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i
↪   * g i ^ 2)) := by
        nlinarith
```

```
      _ = (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2)) ^ 2 := by
        rw [add_sq]
        rw [Real.sq_sqrt h_sum_cf_sq, Real.sq_sqrt h_sum_cg_sq]
        rw [Real.sqrt_mul h_sum_cf_sq (∑ i, c i * g i ^ 2)]
        ring
    have h_sq_sqrt_cF : (Real.sqrt (∑ i, c i * F i ^ 2)) ^ 2 = ∑ i, c i * F i ^ 2 :=
      Real.sq_sqrt h_sum_cF_sq
    have h_nonneg_sqrt_sum : 0 ≤ Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2) := by
      positivity
    have h_sq_goal : (Real.sqrt (∑ i, c i * F i ^ 2)) ^ 2 ≤ (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt
↪   (∑ i, c i * g i ^ 2)) ^ 2 := by
      rw [h_sq_sqrt_cF]
      exact h_goal_sq
    nlinarith
```

Proof. Apply Cauchy–Schwarz to $u_i = \sqrt{c_i} f_i$ and $v_i = \sqrt{c_i} g_i$:

$$\sum_i c_i f_i g_i \le \sqrt{\sum c_i f_i^2 \sum c_i g_i^2}.$$

Since $0 \le F_i \le f_i + g_i$,

$$\sum_i c_i F_i^2 \le \sum_i c_i f_i^2 + \sum_i c_i g_i^2 + 2\sum_i c_i f_i g_i \le \left(\sqrt{\sum c_i f_i^2} + \sqrt{\sum c_i g_i^2}\right)^2.$$

Both sides are nonnegative, so taking square roots gives the claim. □

Lemma 3.3 (markov_subadd_expand)**.** *For arbitrary $\pi : [n] \to \mathbb{R}$, $A \in \mathbb{R}^{[n]\times[n]}$, $f : [n] \times [n] \to \mathbb{R}$ and $r, s \in \mathbb{N}$,*

$$\sum_{i,j} \pi_i (A^{r+s})_{ij} f_{ij} = \sum_{(i,k,j)\in[n]^3} \pi_i (A^r)_{ik} (A^s)_{kj} f_{ij}.$$

Lean: markov_subadd_expand — proved by solver 0 (aiprover), round 0

```
theorem markov_subadd_expand {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (f : Fin n → Fin n → ℝ) (r s : ℕ) :
    ∑ i, ∑ j, π i * (A ^ (r + s)) i j * f i j
      = ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.2 := by
  rw [pow_add]









  have hLHS : (∑ i, ∑ j, π i * ((A ^ r * A ^ s) i j) * f i j) =
              (∑ i, ∑ j, ∑ k, π i * ((A ^ r) i k * (A ^ s) k j) * f i j) := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    simp [Matrix.mul_apply, Finset.mul_sum, Finset.sum_mul, mul_assoc]
  rw [hLHS]


  rw [← Finset.univ_product_univ (α := Fin n), Finset.sum_product]
```

```
  refine Finset.sum_congr rfl (fun i' hi' => ?_)
  rw [← Finset.univ_product_univ (α := Fin n), Finset.sum_product]









  simp









  rw [Finset.sum_comm]



  refine Finset.sum_congr rfl (fun x _ => ?_)
  refine Finset.sum_congr rfl (fun y _ => ?_)
  ring
```

Proof. Use $A^{r+s} = A^r A^s$, expand the matrix product, and reorder the finite triple sum. □

Lemma 3.4 (markov_subadd_left)**.** *Let A be stochastic, and let π, f, r, s be arbitrary. Then*

$$\sum_{(i,k,j)} \pi_i (A^r)_{ik} (A^s)_{kj} f_{ik} = \sum_{i,k} \pi_i (A^r)_{ik} f_{ik}.$$

Lean: markov_subadd_left — proved by AIProver, job 1, session 0

```
theorem markov_subadd_left {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (f : Fin n → Fin n → ℝ) (r s : ℕ) :
    ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.1
      = ∑ i, ∑ k, π i * (A ^ r) i k * f i k := by

  have h_pow_rowsum (k : Fin n) : ∑ j : Fin n, (A ^ s) k j = 1 := by
    induction s with
    | zero => simp [Matrix.one_apply]
    | succ s ih =>
      rw [pow_succ]
      calc
        ∑ j : Fin n, ((A ^ s) * A) k j = ∑ j : Fin n, (∑ l : Fin n, (A ^ s) k l * A l j) := rfl
```

```
      _ = (∑ l : Fin n, (A ^ s) k l * (∑ j : Fin n, A l j)) := by
        rw [Finset.sum_comm]; simp [Finset.mul_sum]
      _ = ∑ l : Fin n, (A ^ s) k l * 1 := by simp [hA.2]
      _ = ∑ l : Fin n, (A ^ s) k l := by simp
      _ = 1 := ih
  have h_triple_sum {α β γ δ : Type _} [AddCommMonoid δ] [Fintype α] [Fintype β] [Fintype γ] [DecidableEq
↪  α] [DecidableEq β] [DecidableEq γ] (g : α → β → γ → δ) :
    (∑ x : α × β × γ, g x.1 x.2.1 x.2.2) = ∑ a : α, ∑ b : β, ∑ c : γ, g a b c := by
    calc
      (∑ x : α × β × γ, g x.1 x.2.1 x.2.2) = (∑ x : α × (β × γ), g x.1 x.2.1 x.2.2) := rfl
      _ = ∑ a : α, ∑ bc : β × γ, g a bc.1 bc.2 := by
       simpa using Finset.sum_product (s := (Finset.univ : Finset α)) (t := (Finset.univ : Finset (β × γ)))
          (f := fun (x : α × (β × γ)) => g x.1 x.2.1 x.2.2)
      _ = ∑ a : α, ∑ b : β, ∑ c : γ, g a b c := by
        refine Finset.sum_congr rfl (fun a _ => ?_)
        simpa using Finset.sum_product (s := (Finset.univ : Finset β)) (t := (Finset.univ : Finset γ))
          (f := fun (bc : β × γ) => g a bc.1 bc.2)
  calc
    ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.1
    = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f i k := by
      simpa using h_triple_sum (fun i k j => (π i * (A ^ r) i k * (A ^ s) k j) * f i k)
    _ = ∑ i : Fin n, ∑ k : Fin n, (π i * (A ^ r) i k * (∑ j : Fin n, (A ^ s) k j)) * f i k := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      calc
        ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f i k
            = (∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j)) * f i k := by
          rw [Finset.sum_mul]
        _ = (π i * (A ^ r) i k * (∑ j : Fin n, (A ^ s) k j)) * f i k := by
          simp [Finset.mul_sum, mul_assoc]
    _ = ∑ i : Fin n, ∑ k : Fin n, (π i * (A ^ r) i k * 1) * f i k := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      rw [h_pow_rowsum k]
    _ = ∑ i : Fin n, ∑ k : Fin n, π i * (A ^ r) i k * f i k := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      ring
```

Proof. Write the triple sum as an iterated sum over i, k, j and pull out the factors that do not depend on j. Then use $\sum_j (A^s)_{kj} = 1$. This row-sum identity is proved by induction on s exactly as in Lemma 3.1. □

Lemma 3.5 (markov_subadd_right)**.** *Let A be stochastic and reversible with respect to π, and let f, r, s be arbitrary. Then*

$$\sum_{(i,k,j)} \pi_i (A^r)_{ik} (A^s)_{kj} f_{kj} = \sum_{k,j} \pi_k (A^s)_{kj} f_{kj}.$$

Lean: markov_subadd_right — proved by AIProver, job 1, session 0

```
theorem markov_subadd_right {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (f : Fin n → Fin n → ℝ) (r s : ℕ) :
    ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2
      = ∑ k, ∑ j, π k * (A ^ s) k j * f k j := by
  classical
```

```
  have h_reorder : ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f
↪  p.2.1 p.2.2
                = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f k j := by
    calc
      ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2
          = ∑ p ∈ (Finset.univ : Finset (Fin n × Fin n × Fin n)),
            (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2 := by simp
      _ = ∑ p ∈ (Finset.univ : Finset (Fin n)) ×[U+02E2] ((Finset.univ : Finset (Fin n)) ×[U+02E2]
↪  (Finset.univ : Finset (Fin n))),
            (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2 := by simp
      _ = ∑ x ∈ (Finset.univ : Finset (Fin n)),
            ∑ yz ∈ ((Finset.univ : Finset (Fin n)) ×[U+02E2] (Finset.univ : Finset (Fin n))),
            (π x * (A ^ r) x yz.1 * (A ^ s) yz.1 yz.2) * f yz.1 yz.2 := by
        rw [Finset.sum_product]
      _ = ∑ x ∈ (Finset.univ : Finset (Fin n)),
            ∑ y ∈ (Finset.univ : Finset (Fin n)), ∑ z ∈ (Finset.univ : Finset (Fin n)),
            (π x * (A ^ r) x y * (A ^ s) y z) * f y z := by
        refine Finset.sum_congr rfl (fun x _ => ?_)
        rw [Finset.sum_product]
      _ = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n,
            (π i * (A ^ r) i k * (A ^ s) k j) * f k j := by simp
  rw [h_reorder]
  calc
    ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f k j
        = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k) * ((A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      refine Finset.sum_congr rfl (fun j _ => ?_)
      ring
    _ = ∑ k : Fin n, ∑ i : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k) * ((A ^ s) k j * f k j) := by
      rw [Finset.sum_comm]
    _ = ∑ k : Fin n, (∑ i : Fin n, π i * (A ^ r) i k) * (∑ j : Fin n, (A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun k _ => ?_)
      rw [Finset.sum_mul]
      refine Finset.sum_congr rfl (fun i _ => ?_)
      simp [Finset.mul_sum, mul_assoc]
    _ = ∑ k : Fin n, (π k) * (∑ j : Fin n, (A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun k _ => ?_)
      rw [setting_stationary_pow π A hA hrev r k]
    _ = ∑ k, ∑ j, π k * (A ^ s) k j * f k j := by
      simp [Finset.mul_sum, mul_assoc]
```

Proof. Rewrite the left side as

$$\sum_{k}\Big(\sum_{i}\pi_i (A^r)_{ik}\Big)\Big(\sum_{j}(A^s)_{kj} f_{kj}\Big).$$

By Lemma 2.6, the first factor equals π_k. □

Lemma 3.6 (comp_geom_sq_sqsum)**.** *If $0 \le q < 1$, the series $\sum_{s\ge 0} s^2 q^s$ converges to $q(1+q)/(1-q)^3$.*

Lean: comp_geom_sq_sqsum — proved by the captain after a hand-back

```
theorem comp_geom_sq_sqsum (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) :
    HasSum (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) (q * (1 + q) / (1 - q) ^ 3) := by
  have hq_norm : ‖q‖ < 1 := by
```

```
    rw [Real.norm_eq_abs, abs_of_nonneg hq0]; exact hq1
  have hsum2 : Summable (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) :=
    summable_pow_mul_geometric_of_norm_lt_one 2 hq_norm
  have hshift : HasSum (fun s : ℕ => ((s + 1 : ℕ) : ℝ) ^ 2 * q ^ (s + 1))
      (q * ((∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) + 2 * (q / (1 - q) ^ 2) + (1 - q)⁻¹)) := by
    have h := ((hsum2.hasSum.add ((hasSum_coe_mul_geometric_of_norm_lt_one hq_norm).mul_left 2)).add
      (hasSum_geometric_of_lt_one hq0 hq1)).mul_left q
    convert h using 1
    all_goals (try (funext s; push_cast; ring))
  have hT : (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s)
      = q * ((∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) + 2 * (q / (1 - q) ^ 2) + (1 - q)⁻¹) := by
    rw [← hshift.tsum_eq]
    rw [hsum2.tsum_eq_zero_add]
    simp
  have hpos : 0 < 1 - q := by linarith
  have hr : (1 - q)⁻¹ * (1 - q) = 1 := inv_mul_cancel₀ hpos.ne'
  have hval : (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) = q * (1 + q) / (1 - q) ^ 3 := by
    rw [div_eq_mul_inv, ← inv_pow]
    rw [div_eq_mul_inv, ← inv_pow] at hT
    generalize (1 - q)⁻¹ = r at hT hr ⊢
    generalize (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) = T at hT ⊢
    linear_combination r * hT - (T + q * r ^ 2) * hr
  rw [← hval]
  exact hsum2.hasSum
```

Proof. The series converges by Mathlib's summability of $s^k q^s$; call its sum T. The $s = 0$ term vanishes, so

$$T = \sum_{s \geq 0} (s+1)^2 q^{s+1}.$$

Expanding $(s+1)^2 = s^2 + 2s + 1$ and using $\sum s q^s = q/(1-q)^2$ and $\sum q^s = (1-q)^{-1}$ gives

$$T = q\bigl(T + 2q/(1-q)^2 + (1-q)^{-1}\bigr).$$

Solving this linear equation for T gives $T(1-q) = q(1+q)/(1-q)^2$, which is the claim. □

Lemma 3.7 (markov_subadd)**.** *Assume the following:*

- $\pi \geq 0$*;*
- A *is stochastic and reversible with respect to* π*;*
- $d : [n] \times [n] \to \mathbb{R}$ *satisfies* $d \geq 0$ *and* $d_{ik} \leq d_{ij} + d_{jk}$ *for all* i, j, k*;*
- $D : \mathbb{N} \to \mathbb{R}$ *satisfies* $D(s) = \sum_{i,j} \pi_i (A^s)_{ij} d_{ij}^2$ *for all* s*.*

Then $\sqrt{D(r+s)} \leq \sqrt{D(r)} + \sqrt{D(s)}$ *for all* $r, s \in \mathbb{N}$*.*

Lean: markov_subadd — proved by the captain after a hand-back

```
theorem markov_subadd {n : ℕ} (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (d : Fin n → Fin n → ℝ) (hd0 : ∀ i j, 0 ≤ d i j) (htri : ∀ i j k, d i k ≤ d i j + d j k)
    (D : ℕ → ℝ) (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * d i j ^ 2) (r s : ℕ) :
    Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := by
  have hc : ∀ p : Fin n × Fin n × Fin n, 0 ≤ π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 :=
```

```
    fun p => mul_nonneg (mul_nonneg (hπ _) ((stoch_pow A hA r).1 _ _)) ((stoch_pow A hA s).1 _ _)
  have e1 : D (r + s) = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.1 p.2.2 ^ 2 := by
    rw [hD]
    exact markov_subadd_expand π A (fun i j => d i j ^ 2) r s
  have e2 : D r = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.1 p.2.1 ^ 2 := by
    rw [hD]
    exact (markov_subadd_left π A hA (fun i j => d i j ^ 2) r s).symm
  have e3 : D s = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.2.1 p.2.2 ^ 2 := by
    rw [hD]
    exact (markov_subadd_right π A hA hrev (fun i j => d i j ^ 2) r s).symm
  rw [e1, e2, e3]
  exact weighted_minkowski
    (fun p : Fin n × Fin n × Fin n => π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2)
    (fun p => d p.1 p.2.1) (fun p => d p.2.1 p.2.2) (fun p => d p.1 p.2.2)
    hc (fun p => hd0 _ _) (fun p => hd0 _ _) (fun p => hd0 _ _) (fun p => htri _ _ _)
```

Proof. On $[n]^3$ take the weights $c_{(i,k,j)} = \pi_i (A^r)_{ik} (A^s)_{kj}$. They are nonnegative by Lemma 3.1. With these weights:

- $D(r+s) = \sum c\, d_{ij}^2$ by Lemma 3.3;
- $D(r) = \sum c\, d_{ik}^2$ by Lemma 3.4;
- $D(s) = \sum c\, d_{kj}^2$ by Lemma 3.5.

Apply Lemma 3.2 with $F = d_{ij}$, $f = d_{ik}$, $g = d_{kj}$. The hypothesis $F \le f + g$ is the triangle inequality. □

Lemma 3.8 (comp_geom_mod_pow_le)**.** *For every integer $t > 0$, $\left(t/(t+1)\right)^t \le 1/2$.*

Lean: comp_geom_mod_pow_le — proved by AIProver, job 1, session 0

```
theorem comp_geom_mod_pow_le (t : ℕ) (ht : 0 < t) : ((t : ℝ) / ((t : ℝ) + 1)) ^ t ≤ 1 / 2 := by
  have hpos : 0 < (t : ℝ) := by exact_mod_cast ht
  have h_denom_pos : 0 < (t : ℝ) + 1 := by linarith
  rcases Nat.eq_or_lt_of_le (Nat.one_le_of_lt ht) with (rfl | ht2)
  ·
    norm_num
  ·
    have h_div_nonneg : 0 ≤ 1 / (t : ℝ) := div_nonneg (by norm_num) (by exact_mod_cast ht.le)


    have h_bern : (2 : ℝ) ≤ ((1 : ℝ) + 1 / (t : ℝ)) ^ t := by
      have h_ge_neg2 : (-2 : ℝ) ≤ 1 / (t : ℝ) := by linarith
      have h_raw := one_add_mul_le_pow (a := 1 / (t : ℝ)) h_ge_neg2 t

      have h_mul : (t : ℝ) * (1 / (t : ℝ)) = 1 := by
        field_simp [ne_of_gt hpos]
      have h_two : (1 : ℝ) + (t : ℝ) * (1 / (t : ℝ)) = (2 : ℝ) := by
        rw [h_mul]
        norm_num
      rw [h_two] at h_raw
      exact h_raw
```

```
    have h_base_pos : 0 < (1 : ℝ) + 1 / (t : ℝ) := by linarith


    have h_eq : (t : ℝ) / ((t : ℝ) + 1) = ((1 : ℝ) + 1 / (t : ℝ))⁻¹ := by
      calc
        (t : ℝ) / ((t : ℝ) + 1) = ((t : ℝ) / (t : ℝ)) / (((t : ℝ) + 1) / (t : ℝ)) := by
          field_simp [ne_of_gt hpos]
        _ = 1 / (((t : ℝ) + 1) / (t : ℝ)) := by field_simp [ne_of_gt hpos]
        _ = 1 / ((t : ℝ) / (t : ℝ) + 1 / (t : ℝ)) := by ring
        _ = 1 / (1 + 1 / (t : ℝ)) := by field_simp [ne_of_gt hpos]
        _ = ((1 : ℝ) + 1 / (t : ℝ))⁻¹ := by simp


    have h_pow_inv : (((1 : ℝ) + 1 / (t : ℝ))⁻¹) ^ t = (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ := by
      rw [inv_pow]

    have h_inv_pos : 0 < ((1 : ℝ) + 1 / (t : ℝ)) ^ t := pow_pos h_base_pos _
    have h_inv : (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ ≤ (2 : ℝ)⁻¹ := by
      rw [inv_le_inv₀ h_inv_pos (by norm_num : 0 < (2 : ℝ))]
      exact h_bern
    calc
      ((t : ℝ) / ((t : ℝ) + 1)) ^ t = (((1 : ℝ) + 1 / (t : ℝ))⁻¹) ^ t := by rw [h_eq]
      _ = (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ := by rw [h_pow_inv]
      _ ≤ (2 : ℝ)⁻¹ := h_inv
      _ = 1 / 2 := by norm_num
```

Proof. For $t = 1$ the left side equals $1/2$.

For $t \geq 2$, Bernoulli's inequality $(1 + x)^t \geq 1 + tx$ with $x = 1/t$ gives $(1 + 1/t)^t \geq 2$. Since $t/(t+1) = (1 + 1/t)^{-1}$, the claim follows by inverting. □

Lemma 3.9 (comp_geom_mod_hasSum). *Let $t > 0$, $0 \leq q < 1$ and $f : \mathbb{N} \to \mathbb{R}$. The series $\sum_{s \geq 0} q^s f(s \bmod t)$ converges to*

$$\frac{\sum_{r=0}^{t-1} q^r f(r)}{1 - q^t}.$$

Lean: comp_geom_mod_hasSum — proved by the captain after a hand-back

```
theorem comp_geom_mod_hasSum (t : ℕ) (ht : 0 < t) (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (f : ℕ → ℝ) :
    HasSum (fun s : ℕ => q ^ s * f (s % t)) ((∑ r ∈ Finset.range t, q ^ r * f r) / (1 - q ^ t)) := by
  have hqt : q ^ t < 1 := pow_lt_one₀ hq0 hq1 (by omega)
  have hsum_abs : Summable (fun s : ℕ => |q ^ s * f (s % t)|) := by
    have hM : ∀ r ∈ Finset.range t, |f r| ≤ ∑ r ∈ Finset.range t, |f r| := fun r hr =>
      Finset.single_le_sum (f := fun r => |f r|) (fun i _ => abs_nonneg _) hr
    have h_bound : ∀ s : ℕ, |q ^ s * f (s % t)| ≤ (∑ r ∈ Finset.range t, |f r|) * q ^ s := by
      intro s
      rw [abs_mul, abs_of_nonneg (pow_nonneg hq0 s), mul_comm (q ^ s)]
      exact mul_le_mul_of_nonneg_right (hM _ (Finset.mem_range.mpr (Nat.mod_lt s ht))) (pow_nonneg hq0 s)
    exact Summable.of_nonneg_of_le (fun s => abs_nonneg _) h_bound
      ((summable_geometric_of_lt_one hq0 hq1).mul_left _)
  have hsum : Summable (fun s : ℕ => q ^ s * f (s % t)) := Summable.of_abs hsum_abs
  set g := fun s : ℕ => q ^ s * f (s % t) with hg
  have hT := hsum.hasSum
  have h1 := (hasSum_nat_add_iff' t).mpr hT
  have h2 : HasSum (fun n => g (n + t)) (q ^ t * ∑' s, g s) := by
    have : (fun n => g (n + t)) = fun n => q ^ t * g n := by
      funext n
```

```
      simp only [hg]
      rw [Nat.add_mod_right, pow_add]
      ring
    rw [this]
    exact hT.mul_left (q ^ t)
  have h3 := h1.unique h2
  have hP : ∑ i ∈ Finset.range t, g i = ∑ r ∈ Finset.range t, q ^ r * f r := by
    refine Finset.sum_congr rfl (fun r hr => ?_)
    simp only [hg]
    rw [Nat.mod_eq_of_lt (Finset.mem_range.mp hr)]
  have hne : (1 - q ^ t) ≠ 0 := by linarith
  have hval : ∑' s, g s = (∑ r ∈ Finset.range t, q ^ r * f r) / (1 - q ^ t) := by
    rw [eq_div_iff hne]
    linear_combination h3 + hP
  rw [← hval]
  exact hT
```

Proof. For every s,

$$|q^s f(s \bmod t)| \le \Big(\sum_{r<t} |f(r)|\Big) q^s.$$

Hence the series $g(s) = q^s f(s \bmod t)$ is absolutely summable; let Σ be its sum.

Since $g(s+t) = q^t g(s)$, the shifted series $\sum_s g(s+t)$ has sum $q^t\Sigma$. Splitting off the first t terms gives

$$\Sigma = \sum_{r<t} q^r f(r) + q^t \Sigma.$$

Divide by $1 - q^t \neq 0$. □

Lemma 3.10 (comp_block)**.** *Let $t \in \mathbb{N}$ and $D : \mathbb{N} \to \mathbb{R}$ with $D \ge 0$ and $\sqrt{D(r+s)} \le \sqrt{D(r)} + \sqrt{D(s)}$ for all r, s. Then for all $j, r \in \mathbb{N}$,*

$$D(jt + r) \le 2j^2 D(t) + 2D(r).$$

Lean: comp_block — proved by AIProver, job 1, session 0

```
theorem comp_block (t : ℕ) (D : ℕ → ℝ) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∀ j r : ℕ, D (j * t + r) ≤ 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by
  intro j r
  have h_base : ∀ j : ℕ, Real.sqrt (D (j * t + r)) ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    intro j'
    induction j' with
    | zero => simp
    | succ k ih =>
      have h_eq : (k.succ : ℕ) * t + r = (k * t + r) + t := by
        rw [Nat.succ_mul]
        omega
      rw [h_eq]
      calc
        Real.sqrt (D ((k * t + r) + t))
            ≤ Real.sqrt (D (k * t + r)) + Real.sqrt (D t) := hsub (k * t + r) t
        _ ≤ ((k : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) + Real.sqrt (D t) := by
          gcongr
        _ = ((k : ℝ) + 1) * Real.sqrt (D t) + Real.sqrt (D r) := by ring
        _ = (k.succ : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by simp
```

```
  have h_sqrt := h_base j
  have hR_nonneg : 0 ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    have h1 : 0 ≤ Real.sqrt (D t) := Real.sqrt_nonneg _
    have h2 : 0 ≤ Real.sqrt (D r) := Real.sqrt_nonneg _
    nlinarith
  have h_sq : D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
    calc
      D (j * t + r) = (Real.sqrt (D (j * t + r))) ^ 2 := by
        rw [Real.sq_sqrt (hDnn (j * t + r))]
      _ ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
        gcongr
  have h_sq_sqrt_t : (Real.sqrt (D t)) ^ 2 = D t := by
    rw [Real.sq_sqrt (hDnn t)]
  have h_sq_sqrt_r : (Real.sqrt (D r)) ^ 2 = D r := by
    rw [Real.sq_sqrt (hDnn r)]
  have h_expand : ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 ≤
      2 * ((j : ℝ) ^ 2) * D t + 2 * D r := by
    calc
      ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2
          = ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 +
            2 * ((j : ℝ) * Real.sqrt (D t)) * Real.sqrt (D r) := by ring
      _ ≤ ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 +
            (((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2) := by
        gcongr
        nlinarith [sq_nonneg ((j : ℝ) * Real.sqrt (D t) - Real.sqrt (D r))]
      _ = 2 * (((j : ℝ) * Real.sqrt (D t)) ^ 2) + 2 * (Real.sqrt (D r)) ^ 2 := by ring
      _ = 2 * ((j : ℝ) ^ 2) * (Real.sqrt (D t)) ^ 2 + 2 * (Real.sqrt (D r)) ^ 2 := by ring
      _ = 2 * ((j : ℝ) ^ 2) * D t + 2 * D r := by rw [h_sq_sqrt_t, h_sq_sqrt_r]
  nlinarith
```

Proof. Induction on j, using $(j+1)t + r = (jt + r) + t$, gives

$$\sqrt{D(jt + r)} \le j\sqrt{D(t)} + \sqrt{D(r)}.$$

Square both sides, which are nonnegative, and use $(a+b)^2 \le 2a^2 + 2b^2$. □

Lemma 3.11 (comp_Dt). *Let $t > 0$, and let $D : \mathbb{N} \to \mathbb{R}$ satisfy $D(0) = 0$, $D \ge 0$ and the subadditivity of Lemma 3.10. Then*

$$D(t) \le 4 \cdot \frac{1}{t} \sum_{s=1}^{t} D(s).$$

Lean: comp_Dt — proved by the captain after a hand-back

```
theorem comp_Dt (t : ℕ) (ht : 0 < t) (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have htne : (t : ℝ) ≠ 0 := ne_of_gt htpos
  have key : ∀ a b : ℕ, D (a + b) ≤ 2 * D a + 2 * D b := by
    intro a b
    have h1 := hsub a b
    have ha := Real.sq_sqrt (hDnn a)
    have hb := Real.sq_sqrt (hDnn b)
    have hab := Real.sq_sqrt (hDnn (a + b))
    have s1 := Real.sqrt_nonneg (D (a + b))
```

```
    have s2 := Real.sqrt_nonneg (D a)
    have s3 := Real.sqrt_nonneg (D b)
    have hm := mul_le_mul h1 h1 s1 (add_nonneg s2 s3)
    nlinarith [sq_nonneg (Real.sqrt (D a) - Real.sqrt (D b))]
  have hIcc : Finset.Icc 1 t = (Finset.range t).image (fun r => r + 1) := by
    ext s
    simp only [Finset.mem_Icc, Finset.mem_image, Finset.mem_range]
    constructor
    · intro h
      exact ⟨s - 1, by omega, by omega⟩
    · rintro ⟨r, hr, rfl⟩
      omega
  have hconv : ∑ s ∈ Finset.Icc 1 t, D s = ∑ r ∈ Finset.range t, D (r + 1) := by
    rw [hIcc, Finset.sum_image]
    intro a _ b _ h
    simpa using h
  have hstep : ∀ r ∈ Finset.range t, D t ≤ 2 * D (r + 1) + 2 * D (t - 1 - r) := by
    intro r hr
    have hr' := Finset.mem_range.mp hr
    have he : (r + 1) + (t - 1 - r) = t := by omega
    have h := key (r + 1) (t - 1 - r)
    rwa [he] at h
  have hsum := Finset.sum_le_sum hstep
  rw [Finset.sum_const, Finset.card_range, nsmul_eq_mul, Finset.sum_add_distrib,
    ← Finset.mul_sum, ← Finset.mul_sum, Finset.sum_range_reflect] at hsum
  have h2 := Finset.sum_range_succ' D t
  have h3 := Finset.sum_range_succ D t
  have hDt := hDnn t
  rw [hconv]
  have hfin : (t : ℝ) * D t ≤ 4 * ∑ r ∈ Finset.range t, D (r + 1) := by
    linarith
  calc D t = (1 / (t : ℝ)) * ((t : ℝ) * D t) := by
        rw [← mul_assoc, one_div_mul_cancel htne, one_mul]
    _ ≤ (1 / (t : ℝ)) * (4 * ∑ r ∈ Finset.range t, D (r + 1)) := by
        apply mul_le_mul_of_nonneg_left hfin
        positivity
    _ = 4 * ((1 / (t : ℝ)) * ∑ r ∈ Finset.range t, D (r + 1)) := by ring
```

Proof. Squaring the subadditivity and using $2ab \le a^2 + b^2$ gives $D(a+b) \le 2D(a) + 2D(b)$. For $0 \le r < t$ this yields

$$D(t) \le 2D(r+1) + 2D(t-1-r).$$

Summing over $r < t$ and reflecting the second sum gives

$$tD(t) \le 2\sum_{r<t} D(r+1) + 2\sum_{r<t} D(r).$$

Now compare the two expressions for $\sum_{r\le t} D(r)$:

$$\sum_{r<t} D(r+1) + D(0) = \sum_{r<t} D(r) + D(t).$$

With $D(0) = 0$ and $D(t) \ge 0$ this gives $\sum_{r<t} D(r) \le \sum_{r<t} D(r+1)$. Hence $tD(t) \le 4\sum_{s=1}^{t} D(s)$. Divide by t. □

Lemma 3.12 (comp_geom_sq). *Let $t > 0$ and $q = t/(t+1)$. The series $\sum_s q^s \lfloor s/t \rfloor^2$ converges, and*

$$(1-q)\sum_s q^s \lfloor s/t \rfloor^2 \le 3.$$

Lean: comp_geom_sq — proved by the captain after a hand-back

```
theorem comp_geom_sq (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1)) :
    Summable (fun s : ℕ => q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ 3 := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1r : (1 : ℝ) ≤ t := by exact_mod_cast ht
  have htne : (t : ℝ) ≠ 0 := ne_of_gt htpos
  have ht1ne : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have h1q : 1 - q = 1 / ((t : ℝ) + 1) := by
    rw [hq]
    field_simp <;> ring
  have hpos : 0 < 1 - q := by rw [h1q]; positivity
  have hq1 : q < 1 := by linarith
  have hpt : ∀ s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ (1 / (t : ℝ) ^ 2) * ((s : ℝ) ^ 2 * q ^ s) := by
    intro s
    have h1 : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) / t := Nat.cast_div_le
    have h0 : (0 : ℝ) ≤ ((s / t : ℕ) : ℝ) := Nat.cast_nonneg _
    have h2 : (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / t) ^ 2 := by nlinarith
    have hqs : 0 ≤ q ^ s := pow_nonneg hq0 s
    calc q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ q ^ s * ((s : ℝ) / t) ^ 2 :=
          mul_le_mul_of_nonneg_left h2 hqs
      _ = (1 / (t : ℝ) ^ 2) * ((s : ℝ) ^ 2 * q ^ s) := by ring
  have hS := comp_geom_sq_sqsum q hq0 hq1
  have hsum : Summable (fun s : ℕ => q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) :=
    Summable.of_nonneg_of_le (fun s => mul_nonneg (pow_nonneg hq0 s) (sq_nonneg _)) hpt
      (hS.mul_left (1 / (t : ℝ) ^ 2)).summable
  refine ⟨hsum, ?_⟩
  have hle := hasSum_le hpt hsum.hasSum (hS.mul_left (1 / (t : ℝ) ^ 2))
  have hval : (1 - q) * (1 / (t : ℝ) ^ 2 * (q * (1 + q) / (1 - q) ^ 3)) = (2 * (t : ℝ) + 1) / t := by
    rw [h1q, hq]
    field_simp <;> ring
  calc (1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      ≤ (1 - q) * (1 / (t : ℝ) ^ 2 * (q * (1 + q) / (1 - q) ^ 3)) :=
        mul_le_mul_of_nonneg_left hle hpos.le
    _ = (2 * (t : ℝ) + 1) / t := hval
    _ ≤ 3 := by rw [div_le_iff₀ htpos]; linarith
```

Proof. Since $\lfloor s/t \rfloor \le s/t$,

$$0 \le q^s \lfloor s/t \rfloor^2 \le t^{-2} s^2 q^s.$$

By Lemma 3.6, this gives convergence and

$$(1-q)\sum_s q^s \lfloor s/t \rfloor^2 \le t^{-2} \frac{q(1+q)}{(1-q)^2}.$$

With $1-q = 1/(t+1)$ and $1+q = (2t+1)/(t+1)$, the right side equals $(2t+1)/t \le 3$. □

Lemma 3.13 (comp_geom_mod). *Let $t > 0$, $q = t/(t+1)$ and $f : \mathbb{N} \to \mathbb{R}$ with $f \ge 0$. The series $\sum_s q^s f(s \bmod t)$ converges, and*

$$(1-q)\sum_s q^s f(s \bmod t) \le \frac{2}{t} \sum_{r=0}^{t-1} f(r).$$

Lean: comp_geom_mod — proved by the captain after a hand-back

```
theorem comp_geom_mod (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (f : ℕ → ℝ) (hf : ∀ r, 0 ≤ f r) :
    Summable (fun s : ℕ => q ^ s * f (s % t)) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * f (s % t) ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1 : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hqt : q * ((t : ℝ) + 1) = t := by rw [hq]; field_simp
  have hq1 : q < 1 := by nlinarith [hqt, htpos]
  have hqt' : (1 - q) * (t : ℝ) = q := by linear_combination (-1 : ℝ) * hqt
  have hpt : q ^ t ≤ 1 / 2 := by rw [hq]; exact comp_geom_mod_pow_le t ht
  have hS := comp_geom_mod_hasSum t ht q hq0 hq1 f
  refine ⟨hS.summable, ?_⟩
  rw [hS.tsum_eq]
  have hP : ∑ r ∈ Finset.range t, q ^ r * f r ≤ ∑ r ∈ Finset.range t, f r :=
    Finset.sum_le_sum (fun r _ => mul_le_of_le_one_left (hf r) (pow_le_one₀ hq0 hq1.le))
  have hP0 : 0 ≤ ∑ r ∈ Finset.range t, q ^ r * f r :=
    Finset.sum_nonneg (fun r _ => mul_nonneg (pow_nonneg hq0 r) (hf r))
  set P := ∑ r ∈ Finset.range t, q ^ r * f r with hPdef
  set F := ∑ r ∈ Finset.range t, f r with hFdef
  have hF0 : 0 ≤ F := le_trans hP0 hP
  have hden : 1 / 2 ≤ 1 - q ^ t := by linarith
  have hdpos : 0 < 1 - q ^ t := by linarith
  have hX : P / (1 - q ^ t) ≤ 2 * P := by
    rw [div_le_iff₀ hdpos]
    nlinarith [mul_le_mul_of_nonneg_left hden hP0]
  have h1 : 0 ≤ 1 - q := by linarith
  have hE : (1 - q) * (2 * F) * (t : ℝ) = 2 * F * q := by
    linear_combination (2 * F) * hqt'
  calc (1 - q) * (P / (1 - q ^ t)) ≤ (1 - q) * (2 * F) :=
       mul_le_mul_of_nonneg_left (by linarith) h1
    _ ≤ 2 / (t : ℝ) * F := by
       rw [div_mul_eq_mul_div, le_div_iff₀ htpos]
       rw [hE]
       nlinarith [mul_nonneg hF0 h1]
```

Proof. Put $P = \sum_{r<t} q^r f(r)$ and $F = \sum_{r<t} f(r)$. Then $0 \le P \le F$, because $0 \le q^r \le 1$.

By Lemma 3.9, the series converges with sum $P/(1-q^t)$. By Lemma 3.8, $1 - q^t \ge 1/2$, so the sum is at most $2P \le 2F$.

Finally $(1-q)t = q \le 1$, hence $(1-q)\cdot 2F \le (2/t)F$. □

Lemma 3.14 (comp_real_range)**.** *If $D : \mathbb{N} \to \mathbb{R}$ satisfies $D(0) = 0$ and $D \ge 0$, then for every $t \in \mathbb{N}$,*

$$\sum_{r=0}^{t-1} D(r) \le \sum_{s=1}^{t} D(s).$$

Lean: comp_real_range — helper of comp_real, proved by the captain after a hand-back

```
theorem comp_real_range (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s) (t : ℕ) :
    ∑ r ∈ Finset.range t, D r ≤ ∑ s ∈ Finset.Icc 1 t, D s := by
  have key : ∀ m : ℕ, ∑ r ∈ Finset.range (m + 1), D r = D 0 + ∑ s ∈ Finset.Icc 1 m, D s := by
    intro m
    induction m with
```

```
    | zero => simp [Finset.Icc_eq_empty]
    | succ k ih =>
      rw [Finset.sum_range_succ, ih, Finset.sum_Icc_succ_top (show 1 ≤ k + 1 by omega)]
      ring
  have h := key t
  rw [Finset.sum_range_succ, hD0] at h
  linarith [hDnn t]
```

Proof. By induction, $\sum_{r=0}^{m} D(r) = D(0) + \sum_{s=1}^{m} D(s)$. With $m = t$ and $D(0) = 0$,

$$\sum_{r<t} D(r) + D(t) = \sum_{s=1}^{t} D(s).$$

Now use $D(t) \geq 0$. □

Lemma 3.15 (comp_real)**.** *Let $t > 0$ and $q = t/(t+1)$. Let $D : \mathbb{N} \to \mathbb{R}$ satisfy $D(0) = 0$, $D \geq 0$ and $\sqrt{D(r+s)} \leq \sqrt{D(r)} + \sqrt{D(s)}$ for all r, s. Then $\sum_s q^s D(s)$ converges and*

$$(1-q)\sum_{s} q^s D(s) \leq 28 \cdot \frac{1}{t} \sum_{s=1}^{t} D(s).$$

Lean: comp_real — proved by the captain after a hand-back

```
theorem comp_real (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    Summable (fun s : ℕ => q ^ s * D s) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * D s ≤ 28 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have hq1 : 0 ≤ 1 - q := by
    rw [hq, sub_nonneg, div_le_one (by positivity)]
    linarith
  obtain ⟨hs1, hb1⟩ := comp_geom_sq t ht q hq
  obtain ⟨hs2, hb2⟩ := comp_geom_mod t ht q hq D hDnn
  have hDt := comp_Dt t ht D hD0 hDnn hsub
  have hblk := comp_block t D hDnn hsub
  have hle : ∀ s : ℕ, q ^ s * D s ≤
      2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)) := by
    intro s
    have hqs : 0 ≤ q ^ s := pow_nonneg hq0 s
    calc q ^ s * D s = q ^ s * D (s / t * t + s % t) := by rw [Nat.div_add_mod']
      _ ≤ q ^ s * (2 * ((s / t : ℕ) : ℝ) ^ 2 * D t + 2 * D (s % t)) :=
          mul_le_mul_of_nonneg_left (hblk _ _) hqs
      _ = 2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)) := by ring
  have hH : HasSum (fun s : ℕ => 2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)))
      (2 * D t * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 + 2 * ∑' s : ℕ, q ^ s * D (s % t)) :=
    (hs1.hasSum.mul_left (2 * D t)).add (hs2.hasSum.mul_left 2)
  have hf : Summable (fun s : ℕ => q ^ s * D s) :=
    Summable.of_nonneg_of_le (fun s => mul_nonneg (pow_nonneg hq0 s) (hDnn s)) hle hH.summable
  refine ⟨hf, ?_⟩
  have htsum := hasSum_le hle hf.hasSum hH
  have hrange := comp_real_range D hD0 hDnn t
  have key : (1 - q) * ∑' s : ℕ, q ^ s * D s ≤
      2 * D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
```

```
      + 2 * ((1 - q) * ∑' s : ℕ, q ^ s * D (s % t)) := by
  calc (1 - q) * ∑' s : ℕ, q ^ s * D s
      ≤ (1 - q) * (2 * D t * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2
          + 2 * ∑' s : ℕ, q ^ s * D (s % t)) := mul_le_mul_of_nonneg_left htsum hq1
    _ = 2 * D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
      + 2 * ((1 - q) * ∑' s : ℕ, q ^ s * D (s % t)) := by ring
have h1 : D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ≤ D t * 3 :=
  mul_le_mul_of_nonneg_left hb1 (hDnn t)
have h2 : (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r
    ≤ 2 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have hinv : (0 : ℝ) ≤ 1 / (t : ℝ) := by positivity
  have := mul_le_mul_of_nonneg_left hrange hinv
  have he : (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r
      = 2 * ((1 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r) := by ring
  rw [he]
  linarith
nlinarith [key, h1, hb2, h2, hDt]
```

Proof. Write $s = \lfloor s/t \rfloor t + (s \bmod t)$. By Lemma 3.10,

$$q^s D(s) \le 2D(t)\, q^s \lfloor s/t \rfloor^2 + 2q^s D(s \bmod t).$$

The right side is summable by Lemmas 3.12 and 3.13. The left side is nonnegative, so it is summable, and the sums compare.

Put $W = \frac{1}{t}\sum_{s=1}^{t} D(s)$. Multiply by $1 - q \ge 0$ and use $D(t) \ge 0$:

$$\begin{aligned}(1-q)\sum_s q^s D(s) &\le 6D(t) + 2 \cdot \frac{2}{t}\sum_{r<t} D(r) \\ &\le 6 \cdot 4W + 4W = 28W.\end{aligned}$$

The first step uses Lemmas 3.12 and 3.13. The second uses Lemma 3.11 and Lemma 3.14. □

Lemma 3.16 (res_summable)**.** *Let A be stochastic, $0 \le q < 1$ and $i, j \in [n]$. The series $\sum_s (1-q)q^s (A^s)_{ij}$ converges.*

Lean: res_summable — proved by AIProver, job 1, session 0

```
theorem res_summable {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (i j : Fin n) :
    Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) := by
  rcases hA with ⟨hA_nonneg, hA_rowsum⟩
  have h_entry_le_one : ∀ i j, A i j ≤ 1 := by
    intro i j
    have hle : A i j ≤ ∑ k : Fin n, A i k :=
      Finset.single_le_sum (by
        intro k hk
        exact hA_nonneg i k) (Finset.mem_univ j)
    rw [hA_rowsum i] at hle
    exact hle
  have h_pow_rowsum : ∀ s i, ∑ j : Fin n, (A ^ s) i j = 1 := by
    intro s i
    induction s with
    | zero =>
      simp [Matrix.one_apply]
```

```
    | succ s ih =>
      rw [pow_succ]
      simp_rw [Matrix.mul_apply]
      rw [Finset.sum_comm]
      simp_rw [← Finset.mul_sum]
      simp_rw [hA_rowsum]
      simp [ih]
  have h_pow_nonneg : ∀ s i j, 0 ≤ (A ^ s) i j := by
    intro s
    induction s with
    | zero =>
      intro i j
      simp [Matrix.one_apply]
      split_ifs
      · exact zero_le_one
      · exact le_refl 0
    | succ s ih =>
      intro i j
      rw [pow_succ, Matrix.mul_apply]
      refine Finset.sum_nonneg (fun k _ => mul_nonneg (ih i k) (hA_nonneg k j))
  have h_pow_le_one : ∀ s i j, (A ^ s) i j ≤ 1 := by
    intro s i j
    have h := Finset.single_le_sum (fun k hk => h_pow_nonneg s i k) (Finset.mem_univ j)
    rw [h_pow_rowsum s i] at h
    exact h

  have hq_nonneg : 0 ≤ q := hq0
  have h_geo_summable : Summable (fun s : ℕ => q ^ s) :=
    summable_geometric_of_lt_one hq_nonneg hq1
  have h_geo_bound : Summable (fun s : ℕ => (1 - q) * q ^ s) := by
    apply Summable.mul_left (1 - q) h_geo_summable
  have h_nonneg_term : ∀ s : ℕ, 0 ≤ (1 - q) * q ^ s * (A ^ s) i j := by
    intro s
    have h1q_nonneg : 0 ≤ 1 - q := by linarith
    have hq_pow_nonneg : 0 ≤ q ^ s := pow_nonneg hq_nonneg _
    have hprod_nonneg : 0 ≤ (1 - q) * q ^ s := mul_nonneg h1q_nonneg hq_pow_nonneg
    exact mul_nonneg hprod_nonneg (h_pow_nonneg s i j)
  refine Summable.of_nonneg_of_le h_nonneg_term (fun s => ?_) h_geo_bound
  have h1q_nonneg : 0 ≤ 1 - q := by linarith
  have hq_pow_nonneg : 0 ≤ q ^ s := pow_nonneg hq_nonneg _
  have hprod_nonneg : 0 ≤ (1 - q) * q ^ s := mul_nonneg h1q_nonneg hq_pow_nonneg
  calc
    (1 - q) * q ^ s * (A ^ s) i j ≤ (1 - q) * q ^ s * 1 :=
      mul_le_mul_of_nonneg_left (h_pow_le_one s i j) hprod_nonneg
    _ = (1 - q) * q ^ s := by ring
```

Proof. By induction on s, the entries of A^s are nonnegative and its rows sum to 1, so $0 \le (A^s)_{ij} \le 1$. Hence the terms lie between 0 and $(1-q)q^s$, whose series converges. □

Lemma 3.17 (res_comparison)**.** *Assume the following:*

- $t > 0$*;*
- π *is a probability vector;*
- A *is stochastic and reversible with respect to* π*;*

- $x : [n] \to \ell_1$;
- $q = t/(t+1)$;
- $G \in \mathbb{R}^{[n]\times[n]}$ *satisfies* $G_{ij} = \sum_{s=0}^{\infty}(1-q)q^s(A^s)_{ij}$ *for all* i, j.

Then

$$\sum_{i,j}\pi_i G_{ij}\|x_i - x_j\|^2 \le 28\sum_{i,j}\pi_i\Big(\frac{1}{t}\sum_{s=1}^{t}(A^s)_{ij}\Big)\|x_i - x_j\|^2.$$

Lean: `res_comparison` — proved by the captain after a hand-back

```
theorem res_comparison {n : ℕ} (t : ℕ) (ht : 0 < t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (x : Fin n → L1Space) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hq1 : q < 1 := by
    rw [hq, div_lt_one (by positivity)]
    linarith
  have hd0 : ∀ i j, 0 ≤ ‖x i - x j‖ := fun i j => norm_nonneg _
  have htri : ∀ i j k, ‖x i - x k‖ ≤ ‖x i - x j‖ + ‖x j - x k‖ := by
    intro i j k
    calc ‖x i - x k‖ = ‖(x i - x j) + (x j - x k)‖ := by rw [sub_add_sub_cancel]
      _ ≤ ‖x i - x j‖ + ‖x j - x k‖ := norm_add_le _ _
  obtain ⟨D, hD⟩ : ∃ D : ℕ → ℝ, ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2 :=
    ⟨_, fun _ => rfl⟩
  have hD0 : D 0 = 0 := by
    rw [hD]
    refine Finset.sum_eq_zero (fun i _ => Finset.sum_eq_zero (fun j _ => ?_))
    rw [pow_zero, Matrix.one_apply]
    split_ifs with h
    · subst h; simp
    · simp
  have hDnn : ∀ s, 0 ≤ D s := by
    intro s
    rw [hD]
    refine Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => ?_))
    have h1 := hπ.1 i
    have h2 := (stoch_pow A hA s).1 i j
    positivity
  have hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
    fun r s => markov_subadd π hπ.1 A hA hrev (fun i j => ‖x i - x j‖) hd0 htri D hD r s
  obtain ⟨_, hbound⟩ := comp_real t ht q hq D hD0 hDnn hsub
  have hterm : ∀ i j, HasSum (fun s : ℕ => π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      (π i * G i j * ‖x i - x j‖ ^ 2) := by
    intro i j
    have h : HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) (G i j) := by
      rw [hG i j]
      exact (res_summable A hA q hq0 hq1 i j).hasSum
    exact (h.mul_left (π i)).mul_right _
  have hall : HasSum (fun s : ℕ => ∑ i, ∑ j, π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      (∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2) :=
    hasSum_sum (fun i _ => hasSum_sum (fun j _ => hterm i j))
```

```
  have hfun : (fun s : ℕ => ∑ i, ∑ j, π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      = fun s => (1 - q) * (q ^ s * D s) := by
    funext s
    rw [hD]
    simp only [Finset.mul_sum]
    exact Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => by ring))
  rw [hfun] at hall
  have hL : ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 = (1 - q) * ∑' s, q ^ s * D s := by
    rw [← hall.tsum_eq, tsum_mul_left]
  have hR : (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s
      = ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
    have hthis : ∀ i j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
        = ∑ s ∈ Finset.Icc 1 t, (1 / (t : ℝ)) * (π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
      intro i j
      simp only [Finset.mul_sum, Finset.sum_mul]
      refine Finset.sum_congr rfl (fun s _ => ?_)
      ring
    simp only [hthis]
    simp only [hD, Finset.mul_sum]
    refine Finset.sum_comm.trans ?_
    refine Finset.sum_congr rfl (fun i _ => ?_)
    exact Finset.sum_comm
  first
    | (rw [hL, ← hR]; exact hbound)
    | linarith [hbound, hL, hR]
```

Proof. Put $D(s) = \sum_{i,j} \pi_i (A^s)_{ij} \|x_i - x_j\|^2$. We check the hypotheses of Lemma 3.15:

- $D(0) = 0$, since $A^0 = I$ and $\|x_i - x_i\| = 0$;
- $D \geq 0$, by $\pi \geq 0$ and Lemma 3.1;
- subadditivity of $\sqrt{D}$ follows from Lemma 3.7 with $d_{ij} = \|x_i - x_j\|$, which satisfies the triangle inequality.

For each i, j, the series $\sum_s \pi_i (1-q) q^s (A^s)_{ij} \|x_i - x_j\|^2$ converges to $\pi_i G_{ij} \|x_i - x_j\|^2$ by Lemma 3.16. Summing these finitely many series gives

$$\sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2 = (1-q) \sum_s q^s D(s).$$

Exchanging finite sums gives

$$\frac{1}{t} \sum_{s=1}^{t} D(s) = \sum_{i,j} \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} (A^s)_{ij} \Big) \|x_i - x_j\|^2.$$

Now apply Lemma 3.15. □

Lemma 3.18 (res_nonneg)**.** *Let A be stochastic, $0 \leq q < 1$, and $G_{ij} = \sum_{s=0}^{\infty} (1-q) q^s (A^s)_{ij}$. Then $G_{ij} \geq 0$ for all i, j.*

Lean: res_nonneg — proved by AIProver, job 1, session 0

```
theorem res_nonneg {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, 0 ≤ G i j := by
  intro i j
  rw [hG i j]
  have h1mq : 0 ≤ 1 - q := by linarith
  refine tsum_nonneg (fun s => ?_)
  have hq_pow : 0 ≤ q ^ s := pow_nonneg hq0 s
  have hA_pow_nonneg : ∀ (k : ℕ), ∀ i j, 0 ≤ (A ^ k) i j := by
    intro k
    induction k with
    | zero =>
      intro i j
      rw [pow_zero, Matrix.one_apply]
      split_ifs <;> norm_num
    | succ k ih =>
      intro i j
      rw [pow_succ, Matrix.mul_apply]
      refine Finset.sum_nonneg (fun l _ => ?_)
      have hAk : 0 ≤ (A ^ k) i l := ih i l
      have hAkl : 0 ≤ A l j := hA.1 l j
      nlinarith
  have hAs : 0 ≤ (A ^ s) i j := hA_pow_nonneg s i j
  have hprod : 0 ≤ (1 - q) * q ^ s * (A ^ s) i j := by
    apply mul_nonneg
    · exact mul_nonneg h1mq hq_pow
    · exact hAs
  exact hprod
```

Proof. All terms are nonnegative, since $(A^s)_{ij} \geq 0$ by induction on s. A sum of nonnegative terms is nonnegative. □

Lemma 3.19 (res_rowsum)**.** *Under the hypotheses of Lemma 3.18, $\sum_j G_{ij} = 1$ for every i.*

Lean: res_rowsum — proved by the captain after a hand-back

```
theorem res_rowsum {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i, ∑ j, G i j = 1 := by
  intro i
  have hs : ∀ j, HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) (G i j) := by
    intro j
    rw [hG]
    exact (res_summable A hA q hq0 hq1 i j).hasSum
  have htot : HasSum (fun s : ℕ => ∑ j, (1 - q) * q ^ s * (A ^ s) i j) (∑ j, G i j) := by
    first
    | exact hasSum_sum (fun j _ => hs j)
    | exact HasSum.finsetSum (fun j _ => hs j)
    | exact hasSum_finsetSum (fun j _ => hs j)
  have hgeom : HasSum (fun s : ℕ => ∑ j, (1 - q) * q ^ s * (A ^ s) i j) 1 := by
    have h1 : ∀ s : ℕ, ∑ j, (1 - q) * q ^ s * (A ^ s) i j = (1 - q) * q ^ s := by
      intro s
```

```
      rw [← Finset.mul_sum, (stoch_pow A hA s).2 i, mul_one]
    simp only [h1]
    have hg := hasSum_geometric_of_lt_one hq0 hq1
    have h2 := hg.mul_left (1 - q)
    have hne : 1 - q ≠ 0 := by linarith
    rwa [mul_inv_cancel₀ hne] at h2
  exact htot.unique hgeom
```

Proof. By Lemma 3.16, the series $\sum_s \sum_j (1-q)q^s (A^s)_{ij}$ converges to $\sum_j G_{ij}$.

By Lemma 3.1, its terms equal $(1-q)q^s$. Their sum is $(1-q)(1-q)^{-1} = 1$. Sums are unique. □

Lemma 3.20 (res_stationary_hasSum_fin)**.** *Let $f : [n] \times \mathbb{N} \to \mathbb{R}$ and $a : [n] \to \mathbb{R}$. If, for each i, the series $\sum_s f(i,s)$ converges to a_i, then $\sum_s \sum_i f(i,s)$ converges to $\sum_i a_i$.*

Lean: res_stationary_hasSum_fin — helper of res_stationary, proved by the captain after a hand-back

```
theorem res_stationary_hasSum_fin {n : ℕ} (f : Fin n → ℕ → ℝ) (a : Fin n → ℝ)
    (h : ∀ i, HasSum (f i) (a i)) : HasSum (fun s => ∑ i, f i s) (∑ i, a i) := by
  classical
  have key : ∀ S : Finset (Fin n), HasSum (fun s => ∑ i ∈ S, f i s) (∑ i ∈ S, a i) := by
    intro S
    refine Finset.induction_on S ?_ ?_
    · simp only [Finset.sum_empty]
      exact hasSum_zero
    · intro i S hi ih
      simp only [Finset.sum_insert hi]
      exact (h i).add ih
  exact key Finset.univ
```

Proof. Induction over finite subsets of $[n]$, using additivity of convergent series. □

Lemma 3.21 (res_stationary)**.** *Let $\pi : [n] \to \mathbb{R}$ be arbitrary, A stochastic and reversible with respect to π, $0 \le q < 1$, and G as in Lemma 3.18. Then $\sum_i \pi_i G_{ij} = \pi_j$ for every j.*

Lean: res_stationary — proved by the captain after a hand-back

```
theorem res_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ j, ∑ i, π i * G i j = π j := by
  intro j
  have hq' : (1 - q) ≠ 0 := ne_of_gt (by linarith)
  have hs : ∀ i, HasSum (fun s : ℕ => π i * ((1 - q) * q ^ s * (A ^ s) i j)) (π i * G i j) := by
    intro i
    rw [hG i j]
    exact ((res_summable A hA q hq0 hq1 i j).hasSum).mul_left (π i)
  have htot : HasSum (fun s : ℕ => ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j))
      (∑ i, π i * G i j) := res_stationary_hasSum_fin _ _ hs
  have hgeo : HasSum (fun s : ℕ => (1 - q) * q ^ s * π j) (π j) := by
    have := (hasSum_geometric_of_lt_one hq0 hq1).mul_left ((1 - q) * π j)
    convert this using 1
    · funext s; ring
    · rw [mul_comm (1 - q) (π j), mul_assoc, mul_inv_cancel₀ hq', mul_one]
```

```
  have heq : (fun s : ℕ => ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j))
      = fun s : ℕ => (1 - q) * q ^ s * π j := by
    funext s
    have hp := setting_stationary_pow π A hA hrev s j
    calc ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j)
        = (1 - q) * q ^ s * ∑ i, π i * (A ^ s) i j := by
          rw [Finset.mul_sum]
          refine Finset.sum_congr rfl (fun i _ => ?_)
          ring
      _ = (1 - q) * q ^ s * π j := by rw [hp]
  rw [heq] at htot
  exact htot.unique hgeo
```

Proof. By Lemmas 3.16 and 3.20, the series $\sum_s \sum_i \pi_i (1-q) q^s (A^s)_{ij}$ converges to $\sum_i \pi_i G_{ij}$.

By Lemma 2.6, its terms equal $(1-q)q^s \pi_j$. Their sum is π_j. □

Lemma 3.22 (res_left)**.** *Let A be stochastic, $0 \le q < 1$, and G as in Lemma 3.18. Then for all i, j,*

$$G_{ij} = (1-q)\delta_{ij} + q \sum_l A_{il} G_{lj}.$$

Lean: res_left — proved by the captain after a hand-back

```
theorem res_left {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j := by
  intro i j
  have hs : ∀ l, Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) l j) :=
    fun l => res_summable A hA q hq0 hq1 l j
  have hs' : ∀ l ∈ (Finset.univ : Finset (Fin n)),
      Summable (fun s : ℕ => A i l * ((1 - q) * q ^ s * (A ^ s) l j)) :=
    fun l _ => (hs l).mul_left (A i l)
  have hswap : ∑' s : ℕ, ∑ l, A i l * ((1 - q) * q ^ s * (A ^ s) l j)
      = ∑ l, ∑' s : ℕ, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    first
    | exact Summable.tsum_finsetSum hs'
    | exact tsum_sum hs'
    | exact (hasSum_sum (fun l hl => (hs' l hl).hasSum)).tsum_eq
  have hsplit : ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j
      = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j
        + ∑' s : ℕ, (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j := by
    rw [(hs i).tsum_eq_zero_add]
    simp only [pow_zero, mul_one]
  have hstep : ∑' s : ℕ, (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j
      = q * ∑' s : ℕ, ∑ l, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    rw [← tsum_mul_left]
    refine tsum_congr (fun s => ?_)
    rw [pow_succ', pow_succ', Matrix.mul_apply, Finset.mul_sum, Finset.mul_sum]
    refine Finset.sum_congr rfl (fun l _ => ?_)
    ring
  have hR : ∀ l, A i l * G l j = ∑' s : ℕ, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    intro l
    rw [hG l j, tsum_mul_left]
  rw [hG i j, hsplit, hstep, hswap, Finset.sum_congr rfl (fun l _ => hR l)]
```

Proof. Split off the $s = 0$ term, which equals $(1-q)\delta_{ij}$. For the remaining terms use $(A^{s+1})_{ij} = \sum_l A_{il}(A^s)_{lj}$:

$$\sum_s (1-q)q^{s+1}(A^{s+1})_{ij} = q\sum_s \sum_l A_{il}(1-q)q^s(A^s)_{lj}.$$

By Lemma 3.16, the finite sum over l and the series can be exchanged. This gives $q\sum_l A_{il}G_{lj}$. □

Lemma 3.23 (res_right)**.** *Under the same hypotheses, for all i, j,*

$$G_{ij} = (1-q)\delta_{ij} + q\sum_l G_{il}A_{lj}.$$

Lean: res_right — proved by the captain after a hand-back

```
theorem res_right {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j := by
  intro i j
  have hs : ∀ l, Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i l) :=
    fun l => res_summable A hA q hq0 hq1 i l
  have h1 : HasSum (fun s : ℕ => ∑ l, (1 - q) * q ^ s * (A ^ s) i l * A l j)
      (∑ l, G i l * A l j) := by
    refine hasSum_sum (fun l _ => ?_)
    show HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i l * A l j) (G i l * A l j)
    rw [hG i l]
    exact (hs l).hasSum.mul_right (A l j)
  have h2 : HasSum (fun s : ℕ => (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j)
      (q * ∑ l, G i l * A l j) := by
    have := h1.mul_left q
    convert this using 1
    funext s
    rw [pow_succ q s, pow_succ A s, Matrix.mul_apply]
    simp only [Finset.mul_sum]
    refine Finset.sum_congr rfl (fun l _ => ?_)
    ring
  have h3 := (hasSum_nat_add_iff (f := fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) 1).mp h2
  rw [hG i j, h3.tsum_eq]
  simp only [Finset.sum_range_one, pow_zero, mul_one]
  ring
```

Proof. By Lemma 3.16, the series $\sum_s \sum_l (1-q)q^s(A^s)_{il}A_{lj}$ converges to $\sum_l G_{il}A_{lj}$.

Since $(A^{s+1})_{ij} = \sum_l (A^s)_{il}A_{lj}$, the shifted series $\sum_s (1-q)q^{s+1}(A^{s+1})_{ij}$ converges to $q\sum_l G_{il}A_{lj}$. Adding back the $s = 0$ term $(1-q)\delta_{ij}$ gives the claim. □

Lemma 3.24 (flat_scalar)**.** *Let $r, r' \in [0,1]$ and $d \in \{0,1\}$. With $\phi(r) = 3r^2 - 2r^3$,*

$$|\phi(r') - \phi(r)| \le 6|r-d||r'-r| + 3(r'-r)^2.$$

Lean: flat_scalar — proved by AIProver, job 1, session 0

```
theorem flat_scalar (r r' d : ℝ) (hr : 0 ≤ r ∧ r ≤ 1) (hr' : 0 ≤ r' ∧ r' ≤ 1) (hd : d = 0 ∨ d = 1) :
    |(3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3)|
      ≤ 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
  rcases hr with ⟨hr0, hr1⟩
```

```
  rcases hr' with ⟨hr0', hr1'⟩
  set δ := r' - r with hδ
  have hφdiff : (3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3) = δ * (6 * r * (1 - r) + δ * (3 - 4 * r -
↪  2 * r')) := by
    ring
  rw [hφdiff, abs_mul]

  have hA_nonneg : 0 ≤ 6 * r * (1 - r) := by
    nlinarith
  have hA_bound : 6 * r * (1 - r) ≤ 6 * |r - d| := by
    rcases hd with (hd0 | hd1)
    · rw [hd0, sub_zero]
      rw [abs_of_nonneg hr0]
      nlinarith
    · rw [hd1]
      have habs_eq : |r - 1| = 1 - r := by
        have h := abs_of_nonpos (show r - 1 ≤ 0 from by nlinarith)
        linarith
      rw [habs_eq]
      nlinarith
  have h_inner_bound : |3 - 4 * r - 2 * r'| ≤ 3 := by
    have hlow : -3 ≤ 3 - 4 * r - 2 * r' := by nlinarith
    have hhigh : 3 - 4 * r - 2 * r' ≤ 3 := by nlinarith
    exact abs_le.mpr ⟨hlow, hhigh⟩
  have h_abs_δ_le_one : |δ| ≤ 1 := by
    have hδ_low : -1 ≤ δ := by nlinarith
    have hδ_high : δ ≤ 1 := by nlinarith
    exact abs_le.mpr ⟨hδ_low, hδ_high⟩
  have hB_bound : |δ * (3 - 4 * r - 2 * r')| ≤ 3 * |δ| := by
    rw [abs_mul]
    have h_nonneg_absδ : 0 ≤ |δ| := abs_nonneg _
    nlinarith
  have h_sum_bound : |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤ 6 * |r - d| + 3 * |δ| := by
    calc
      |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤
          |6 * r * (1 - r)| + |δ * (3 - 4 * r - 2 * r')| := abs_add _ _
      _ = 6 * r * (1 - r) + |δ * (3 - 4 * r - 2 * r')| := by rw [abs_of_nonneg hA_nonneg]
      _ ≤ 6 * r * (1 - r) + 3 * |δ| := by nlinarith
      _ ≤ 6 * |r - d| + 3 * |δ| := by nlinarith
  calc
    |δ| * |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤ |δ| * (6 * |r - d| + 3 * |δ|) := by
      have hpos_absδ : 0 ≤ |δ| := abs_nonneg _
      have h_nonneg_nlinarith : 0 ≤ 6 * |r - d| + 3 * |δ| := by
        positivity
      nlinarith
    _ = |δ| * (6 * |r - d|) + |δ| * (3 * |δ|) := by ring
    _ = 6 * |r - d| * |δ| + 3 * (|δ| * |δ|) := by ring
    _ = 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
      have h_sq : |δ| * |δ| = δ ^ 2 := by
        calc
          |δ| * |δ| = |δ| * |δ| := rfl
          _ = δ * δ := abs_mul_abs_self δ
          _ = δ ^ 2 := by ring
      rw [h_sq, hδ]
```

Proof. With $\delta = r' - r$ one has the polynomial identity

$$\phi(r') - \phi(r) = \delta\big(6r(1-r) + \delta(3 - 4r - 2r')\big).$$

We bound the factors:

- $0 \le 6r(1-r) \le 6|r-d|$, since $r(1-r) \le r$ and $r(1-r) \le 1-r$;
- $|3 - 4r - 2r'| \le 3$, since $r, r' \in [0,1]$.

Hence the second factor is at most $6|r-d| + 3|\delta|$ in absolute value. Multiply by $|\delta|$. □

Lemma 3.25 (`flat_increment`)**.** *Assume the following:*

- ι *is a finite set and* $w : \iota \to \mathbb{R}$ *satisfies* $w \ge 0$*;*
- $\phi : \mathbb{R} \to \mathbb{R}$ *satisfies* $\phi(r) = 3r^2 - 2r^3$ *for all* r*;*
- $e, u, u' : \iota \to \mathbb{R}$ *with* $e_B \in \{0,1\}$ *and* $u_B, u'_B \in [0,1]$ *for all* B*.*

Then

$$\Big(\sum_B w_B |\phi(u'_B) - \phi(u_B)|\Big)^2 \le 72\Big(\sum_B w_B (u_B - e_B)^2\Big)\Big(\sum_B w_B (u'_B - u_B)^2\Big) + 18\Big(\sum_B w_B (u'_B - u_B)^2\Big)^2.$$

Lean: `flat_increment` — proved by the captain after a hand-back

```
theorem flat_increment {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u u' : ι → ℝ) (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hu' : ∀ B, 0 ≤ u' B ∧ u' B ≤ 1) :
    (∑ B, w B * |φ (u' B) - φ (u B)|) ^ 2
      ≤ 72 * (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2)
        + 18 * (∑ B, w B * (u' B - u B) ^ 2) ^ 2 := by
  have hS : ∀ B, w B * |φ (u' B) - φ (u B)|
      ≤ w B * (6 * |u B - e B| * |u' B - u B| + 3 * (u' B - u B) ^ 2) := by
    intro B
    have h1 := flat_scalar (u B) (u' B) (e B) (hu B) (hu' B) (he B)
    rw [hφ (u' B), hφ (u B)]
    exact mul_le_mul_of_nonneg_left h1 (hw B)
  have hsum : ∑ B, w B * |φ (u' B) - φ (u B)|
      ≤ 6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) + 3 * (∑ B, w B * (u' B - u B) ^ 2) := by
    calc ∑ B, w B * |φ (u' B) - φ (u B)|
        ≤ ∑ B, w B * (6 * |u B - e B| * |u' B - u B| + 3 * (u' B - u B) ^ 2) :=
          Finset.sum_le_sum (fun B _ => hS B)
      _ = 6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) + 3 * (∑ B, w B * (u' B - u B) ^ 2) := by
          rw [Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
          exact Finset.sum_congr rfl (fun B _ => by ring)
  have hCS : (∑ B, w B * (|u B - e B| * |u' B - u B|)) ^ 2
      ≤ (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2) := by
    have h : (∑ B, (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)) ^ 2
        ≤ (∑ B, (Real.sqrt (w B) * |u B - e B|) ^ 2) * (∑ B, (Real.sqrt (w B) * |u' B - u B|) ^ 2) :=
      Finset.sum_mul_sq_le_sq_mul_sq _ _ _
    have k1 : ∑ B, (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)
        = ∑ B, w B * (|u B - e B| * |u' B - u B|) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      have hs := Real.mul_self_sqrt (hw B)
      calc (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)
          = (Real.sqrt (w B) * Real.sqrt (w B)) * (|u B - e B| * |u' B - u B|) := by ring
```

```
      _ = w B * (|u B - e B| * |u' B - u B|) := by rw [hs]
  have k2 : ∑ B, (Real.sqrt (w B) * |u B - e B|) ^ 2 = ∑ B, w B * (u B - e B) ^ 2 := by
    refine Finset.sum_congr rfl (fun B _ => ?_)
    have hs := Real.mul_self_sqrt (hw B)
    calc (Real.sqrt (w B) * |u B - e B|) ^ 2
        = (Real.sqrt (w B) * Real.sqrt (w B)) * |u B - e B| ^ 2 := by ring
      _ = w B * (u B - e B) ^ 2 := by rw [hs, sq_abs]
  have k3 : ∑ B, (Real.sqrt (w B) * |u' B - u B|) ^ 2 = ∑ B, w B * (u' B - u B) ^ 2 := by
    refine Finset.sum_congr rfl (fun B _ => ?_)
    have hs := Real.mul_self_sqrt (hw B)
    calc (Real.sqrt (w B) * |u' B - u B|) ^ 2
        = (Real.sqrt (w B) * Real.sqrt (w B)) * |u' B - u B| ^ 2 := by ring
      _ = w B * (u' B - u B) ^ 2 := by rw [hs, sq_abs]
  rw [k1, k2, k3] at h
  exact h
have hX0 : 0 ≤ ∑ B, w B * (|u B - e B| * |u' B - u B|) :=
  Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (mul_nonneg (abs_nonneg _) (abs_nonneg _)))
have hY0 : 0 ≤ ∑ B, w B * (u' B - u B) ^ 2 :=
  Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (sq_nonneg _))
have hP0 : 0 ≤ ∑ B, w B * (u B - e B) ^ 2 :=
  Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (sq_nonneg _))
have hS0 : 0 ≤ ∑ B, w B * |φ (u' B) - φ (u B)| :=
  Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (abs_nonneg _))
have hmul := mul_le_mul hsum hsum hS0 (le_trans hS0 hsum)
nlinarith [hmul, hCS, hX0, hY0, hP0,
  sq_nonneg (6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) - 3 * (∑ B, w B * (u' B - u B) ^ 2))]
```

Proof. Write S for the sum on the left and set

$$P=\sum w_B(u_B-e_B)^2,\quad Y=\sum w_B(u'_B-u_B)^2,\quad X=\sum w_B|u_B-e_B||u'_B-u_B|.$$

Summing Lemma 3.24 with weights w_B gives $0\le S\le 6X+3Y$. Cauchy–Schwarz with $\sqrt{w_B}$ gives $X^2\le PY$. Hence

$$S^2\le(6X+3Y)^2\le 72X^2+18Y^2\le 72PY+18Y^2.$$

□

Lemma 3.26 (`flat_ineq`)**.** *Under the hypotheses of Lemma 3.25, put*

$$F(v)=\Big(\sum_B w_B(v_B-e_B)^2\Big)^2,\quad P=\sum_B w_B(u_B-e_B)^2,\quad L=\sum_B w_B(u_B-e_B)(u'_B-u_B).$$

Then

$$\Big(\sum_B w_B|\phi(u'_B)-\phi(u_B)|\Big)^2\le 108\big(F(u')-F(u)-4PL\big).$$

Lean: `flat_ineq` — proved by the captain after a hand-back

```
theorem flat_ineq {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u u' : ι → ℝ) (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hu' : ∀ B, 0 ≤ u' B ∧ u' B ≤ 1) :
    (∑ B, w B * |φ (u' B) - φ (u B)|) ^ 2
      ≤ 108 * ((∑ B, w B * (u' B - e B) ^ 2) ^ 2 - (∑ B, w B * (u B - e B) ^ 2) ^ 2
          - 4 * (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u B - e B) * (u' B - u B))) := by
  have hinc := flat_increment w hw φ hφ e u u' he hu hu'
```

```
  have hP' : ∑ B, w B * (u' B - e B) ^ 2
      = ∑ B, w B * (u B - e B) ^ 2 + 2 * ∑ B, w B * (u B - e B) * (u' B - u B)
        + ∑ B, w B * (u' B - u B) ^ 2 := by
    rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_add_distrib]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    ring
  have hCS : (∑ B, w B * (u B - e B) * (u' B - u B)) ^ 2
      ≤ (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2) := by
    have h := Finset.sum_mul_sq_le_sq_mul_sq Finset.univ
      (fun B => Real.sqrt (w B) * (u B - e B)) (fun B => Real.sqrt (w B) * (u' B - u B))
    have h1 : ∀ B, Real.sqrt (w B) * (u B - e B) * (Real.sqrt (w B) * (u' B - u B))
        = w B * (u B - e B) * (u' B - u B) := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u B - e B) * (u' B - u B)) * hs
    have h2 : ∀ B, (Real.sqrt (w B) * (u B - e B)) ^ 2 = w B * (u B - e B) ^ 2 := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u B - e B) ^ 2) * hs
    have h3 : ∀ B, (Real.sqrt (w B) * (u' B - u B)) ^ 2 = w B * (u' B - u B) ^ 2 := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u' B - u B) ^ 2) * hs
    simp only [h1, h2, h3] at h
    exact h
  rw [hP']
  nlinarith [hinc, hCS,
    sq_nonneg (2 * (∑ B, w B * (u B - e B) * (u' B - u B)) + ∑ B, w B * (u' B - u B) ^ 2),
    sq_nonneg (4 * (∑ B, w B * (u B - e B) * (u' B - u B)) + ∑ B, w B * (u' B - u B) ^ 2)]
```

Proof. Let $Y = \sum w_B(u'_B - u_B)^2$. Expanding termwise gives

$$\sum w_B(u'_B - e_B)^2 = P + 2L + Y.$$

Hence

$$R := F(u') - F(u) - 4PL = 2PY + (2L + Y)^2.$$

Cauchy–Schwarz gives $L^2 \le PY$. Therefore

$$Y^2 \le 2(2L + Y)^2 + 8L^2 \le 2(2L + Y)^2 + 8PY.$$

By Lemma 3.25,

$$\begin{aligned} S^2 &\le 72PY + 18Y^2 \\ &\le 216PY + 36(2L + Y)^2 \\ &\le 108R. \end{aligned}$$

□

Lemma 3.27 (`local_ineq_combine`). *Let $m \in \mathbb{N}$, let $p, S_0, a, F_v, L_v \in \mathbb{R}$, and let $c, b, F_U, L : [m] \to \mathbb{R}$. Assume:*

- *$p \ge 0$, $c \ge 0$ and $p + \sum_j c_j = 1$;*
- *$pL_v + \sum_j c_j L_j = 0$;*

- $a \le 108(F_v^2 - S_0^2 - 4S_0L_v)$;
- $b_j \le 108(F_{U,j}^2 - S_0^2 - 4S_0L_j)$ *for all* j.

Then

$$pa + \sum_j c_j b_j \le 108\Big(pF_v^2 + \sum_j c_j F_{U,j}^2 - S_0^2\Big).$$

Lean: `local_ineq_combine` — helper of local_ineq, proved by the captain after a hand-back

```
theorem local_ineq_combine {m : ℕ} (p S0 a Fv Lv : ℝ) (c b FU L : Fin m → ℝ)
    (hp : 0 ≤ p) (hc : ∀ j, 0 ≤ c j) (hsum : p + ∑ j, c j = 1)
    (hlin : p * Lv + ∑ j, c j * L j = 0)
    (ha : a ≤ 108 * (Fv ^ 2 - S0 ^ 2 - 4 * S0 * Lv))
    (hb : ∀ j, b j ≤ 108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j)) :
    p * a + ∑ j, c j * b j ≤ 108 * (p * Fv ^ 2 + ∑ j, c j * FU j ^ 2 - S0 ^ 2) := by
  have h1 : ∑ j, c j * b j ≤ ∑ j, c j * (108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j)) :=
    Finset.sum_le_sum fun j _ => mul_le_mul_of_nonneg_left (hb j) (hc j)
  have h2 : ∑ j, c j * (108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j))
      = 108 * ∑ j, c j * FU j ^ 2 - 108 * S0 ^ 2 * ∑ j, c j - 432 * S0 * ∑ j, c j * L j := by
    simp only [Finset.mul_sum, ← Finset.sum_sub_distrib]
    exact Finset.sum_congr rfl fun j _ => by ring
  have hsc : ∑ j, c j = 1 - p := by linarith
  have hlin' : ∑ j, c j * L j = -(p * Lv) := by linarith
  rw [hsc, hlin'] at h2
  have h3 := mul_le_mul_of_nonneg_left ha hp
  nlinarith [h1, h2, h3]
```

Proof. Multiply the hypotheses by $p \ge 0$ and $c_j \ge 0$ and add them. Use $\sum c_j = 1 - p$ and $\sum c_j L_j = -pL_v$. The two terms $\mp 432\, pS_0L_v$ cancel, and the coefficients of S_0^2 add up to -108. □

Lemma 3.28 (`local_ineq`)**.** *Let* ι *be finite,* $w \ge 0$ *and* ϕ *as in Lemma 3.25. Let* $e, u, v : \iota \to \mathbb{R}$, $U : [m] \to (\iota \to \mathbb{R})$, $p \in \mathbb{R}$ *and* $c : [m] \to \mathbb{R}$. *Assume:*

- $p \ge 0$, $c \ge 0$ *and* $p + \sum_j c_j = 1$;
- $u_B = pv_B + \sum_j c_j U_{j,B}$ *for all* B;
- $e_B \in \{0, 1\}$ *and* $u_B, v_B, U_{j,B} \in [0, 1]$ *for all* j, B.

Write $F(\cdot)$ *as in Lemma 3.26. Then*

$$p\Big(\sum_B w_B|\phi(v_B) - \phi(u_B)|\Big)^2 + \sum_j c_j\Big(\sum_B w_B|\phi(U_{j,B}) - \phi(u_B)|\Big)^2 \le 108\Big(pF(v) + \sum_j c_j F(U_j) - F(u)\Big).$$

Lean: `local_ineq` — proved by the captain after a hand-back

```
theorem local_ineq {ι : Type*} [Fintype ι] {m : ℕ} (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u v : ι → ℝ) (U : Fin m → ι → ℝ) (p : ℝ) (c : Fin m → ℝ)
    (hp : 0 ≤ p) (hc : ∀ j, 0 ≤ c j) (hsum : p + ∑ j, c j = 1)
    (hbal : ∀ B, u B = p * v B + ∑ j, c j * U j B)
    (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hv : ∀ B, 0 ≤ v B ∧ v B ≤ 1)
    (hU : ∀ j B, 0 ≤ U j B ∧ U j B ≤ 1) :
```

```
    p * (∑ B, w B * |φ (v B) - φ (u B)|) ^ 2 + ∑ j, c j * (∑ B, w B * |φ (U j B) - φ (u B)|) ^ 2
      ≤ 108 * (p * (∑ B, w B * (v B - e B) ^ 2) ^ 2
          + ∑ j, c j * (∑ B, w B * (U j B - e B) ^ 2) ^ 2
          - (∑ B, w B * (u B - e B) ^ 2) ^ 2) := by
  have hinner : ∀ B, p * (v B - u B) + ∑ j, c j * (U j B - u B) = 0 := by
    intro B
    have h1 : ∑ j, c j * (U j B - u B) = ∑ j, c j * U j B - (∑ j, c j) * u B := by
      rw [Finset.sum_mul, ← Finset.sum_sub_distrib]
      exact Finset.sum_congr rfl fun j _ => by ring
    have hs : ∑ j, c j = 1 - p := by linarith
    rw [h1, hs]
    have hb := hbal B
    linear_combination (-1 : ℝ) * hb
  have hlin : p * (∑ B, w B * (u B - e B) * (v B - u B))
      + ∑ j, c j * (∑ B, w B * (u B - e B) * (U j B - u B)) = 0 := by
    have hpt : ∀ B, w B * (u B - e B) * (p * (v B - u B) + ∑ j, c j * (U j B - u B))
        = p * (w B * (u B - e B) * (v B - u B))
          + ∑ j, c j * (w B * (u B - e B) * (U j B - u B)) := by
      intro B
      rw [mul_add, Finset.mul_sum]
      congr 1
      · ring
      · exact Finset.sum_congr rfl fun j _ => by ring
    have hz : ∑ B, w B * (u B - e B) * (p * (v B - u B) + ∑ j, c j * (U j B - u B)) = 0 :=
      Finset.sum_eq_zero fun B _ => by rw [hinner B, mul_zero]
    rw [Finset.sum_congr rfl fun B _ => hpt B, Finset.sum_add_distrib, ← Finset.mul_sum,
      Finset.sum_comm] at hz
    have hz2 : ∑ j, c j * (∑ B, w B * (u B - e B) * (U j B - u B))
        = ∑ j, ∑ B, c j * (w B * (u B - e B) * (U j B - u B)) :=
      Finset.sum_congr rfl fun j _ => by rw [Finset.mul_sum]
    rw [hz2]
    exact hz
  exact local_ineq_combine p (∑ B, w B * (u B - e B) ^ 2) _ (∑ B, w B * (v B - e B) ^ 2)
    (∑ B, w B * (u B - e B) * (v B - u B)) c
    (fun j => (∑ B, w B * |φ (U j B) - φ (u B)|) ^ 2)
    (fun j => ∑ B, w B * (U j B - e B) ^ 2)
    (fun j => ∑ B, w B * (u B - e B) * (U j B - u B))
    hp hc hsum hlin (flat_ineq w hw φ hφ e u v he hu hv)
    (fun j => flat_ineq w hw φ hφ e u (U j) he hu (hU j))
```

Proof. From the balance relation and $\sum c_j = 1 - p$,

$$p(v_B - u_B) + \sum_j c_j (U_{j,B} - u_B) = 0 \qquad \text{for each } B.$$

Multiply by $w_B(u_B - e_B)$ and sum over B. With $L_v = \sum_B w_B(u_B - e_B)(v_B - u_B)$ and $L_j = \sum_B w_B(u_B - e_B)(U_{j,B} - u_B)$ this gives $pL_v + \sum_j c_j L_j = 0$.

Apply Lemma 3.26 to the pair (u, v) and to each pair (u, U_j). Then conclude with Lemma 3.27, taking $S_0 = \sum_B w_B(u_B - e_B)^2$. □

Lemma 3.29 (`global_sum`). *Let $\pi : [n] \to \mathbb{R}$ with $\pi \geq 0$, let $A, G \in \mathbb{R}^{[n]\times[n]}$ and $p, q \in \mathbb{R}$ with $p \geq 0$. Assume:*

- *$G \geq 0$ entrywise;*
- *$\sum_i \pi_i G_{ij} = \pi_j$ for all j;*

- $G_{kj} = p\delta_{kj} + q\sum_i G_{ki}A_{ij}$ *for all* k, j.

Let $J^d : [n] \to \mathbb{R}$ *and* $J^h, F^z, F^h : [n] \times [n] \to \mathbb{R}$ *with* $F^h \geq 0$, *and suppose that for all* k, i

$$pJ^d_i + q\sum_j A_{ij}J^h_{ij} \leq 108\Big(pF^z_{ki} + q\sum_j A_{ij}F^h_{kj} - F^h_{ki}\Big).$$

Then

$$\sum_i \pi_i\Big(pJ^d_i + q\sum_j A_{ij}J^h_{ij}\Big) \leq 108\,p\sum_k\sum_i \pi_k G_{ki}F^z_{ki}.$$

Lean: `global_sum` — proved by the captain after a hand-back

```
theorem global_sum {n : ℕ} (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A G : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ) (hp : 0 ≤ p)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (hGr : ∀ k j, G k j = p * (1 : Matrix (Fin n) (Fin n) ℝ) k j + q * ∑ i, G k i * A i j)
    (Jd : Fin n → ℝ) (Jh Fz Fh : Fin n → Fin n → ℝ) (hFh : ∀ k i, 0 ≤ Fh k i)
    (hloc : ∀ k i, p * Jd i + q * ∑ j, A i j * Jh i j
        ≤ 108 * (p * Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) :
    ∑ i, π i * (p * Jd i + q * ∑ j, A i j * Jh i j)
      ≤ 108 * p * ∑ k, ∑ i, π k * G k i * Fz k i := by
  have key : ∀ k, ∑ i, G k i * (q * ∑ j, A i j * Fh k j) = ∑ j, G k j * Fh k j - p * Fh k k := by
    intro k
    have e1 : ∑ i, G k i * (q * ∑ j, A i j * Fh k j) = ∑ j, (q * ∑ i, G k i * A i j) * Fh k j := by
      simp only [Finset.mul_sum, Finset.sum_mul]
      rw [Finset.sum_comm]
      refine Finset.sum_congr rfl (fun j _ => Finset.sum_congr rfl (fun i _ => by ring))
    have e2 : ∀ j, G k j * Fh k j = p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j + (q * ∑ i, G k i * A i
↪  j) * Fh k j := by
      intro j
      conv_lhs => rw [hGr k j]
      ring
    have e3 : ∑ j, p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j = p * Fh k k := by
      simp [Matrix.one_apply, Finset.sum_ite_eq]
    have e4 : ∑ j, G k j * Fh k j = ∑ j, (p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j + (q * ∑ i, G
↪  k i * A i j) * Fh k j) :=
      Finset.sum_congr rfl (fun j _ => e2 j)
    rw [e1, e4, Finset.sum_add_distrib, e3]
    ring
  have hk : ∀ k, ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) ≤ 108 * p * ∑ i, π k * G k i *
↪  Fz k i := by
    intro k
    have s1 : ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) ≤ ∑ i, π k * G k i * (108 * (p *
↪  Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) :=
      Finset.sum_le_sum (fun i _ => mul_le_mul_of_nonneg_left (hloc k i) (mul_nonneg (hπ k) (hG0 k i)))
    have s2 : ∑ i, π k * G k i * (108 * (p * Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) = 108 * π k * (p
↪  * ∑ i, G k i * Fz k i + ∑ i, G k i * (q * ∑ j, A i j * Fh k j) - ∑ i, G k i * Fh k i) := by
      rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib, Finset.mul_sum]
      refine Finset.sum_congr rfl (fun i _ => by ring)
    have s3 : ∑ i, π k * G k i * Fz k i = π k * ∑ i, G k i * Fz k i := by
      rw [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun i _ => by ring)
    rw [s2, key k] at s1
    rw [s3]
    nlinarith [mul_nonneg (mul_nonneg (hπ k) hp) (hFh k k)]
  calc ∑ i, π i * (p * Jd i + q * ∑ j, A i j * Jh i j)
      = ∑ i, ∑ k, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) := by
```

```
      refine Finset.sum_congr rfl (fun i _ => ?_)
      rw [← hGπ i, Finset.sum_mul]
  _ = ∑ k, ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) := Finset.sum_comm
  _ ≤ ∑ k, 108 * p * ∑ i, π k * G k i * Fz k i := Finset.sum_le_sum (fun k _ => hk k)
  _ = 108 * p * ∑ k, ∑ i, π k * G k i * Fz k i := by rw [Finset.mul_sum]
```

Proof. Fix k. Using the identity for G,

$$\sum_i G_{ki}\Big(q\sum_j A_{ij}F^h_{kj}\Big) = \sum_j \Big(q\sum_i G_{ki}A_{ij}\Big)F^h_{kj} = \sum_j G_{kj}F^h_{kj} - pF^h_{kk}.$$

Multiply the hypothesis for (k,i) by $\pi_k G_{ki} \ge 0$ and sum over i. The potential terms telescope by the display, so the right side becomes

$$108\,\pi_k\Big(p\sum_i G_{ki}F^z_{ki} - pF^h_{kk}\Big) \le 108\,p\sum_i \pi_k G_{ki}F^z_{ki},$$

since $\pi_k, p, F^h_{kk} \ge 0$.

Finally, by stationarity,

$$\sum_i \pi_i(\cdots)_i = \sum_i\sum_k \pi_k G_{ki}(\cdots)_i.$$

Exchange the two sums and sum the bounds over k. □

Lemma 3.30 (h_props)**.** *Let ι be finite, $G, A \in \mathbb{R}^{[n]\times[n]}$ and $p, q \in \mathbb{R}$. Assume:*

- *$G \ge 0$ and $\sum_j G_{ij} = 1$ for all i;*
- *$G_{ij} = p\delta_{ij} + q\sum_l A_{il}G_{lj}$ for all i, j;*
- *$z : [n] \to (\iota \to \mathbb{R})$ takes values in $[0,1]$;*
- *$h_{iB} = \sum_j G_{ij}z_{jB}$ for all i, B.*

Then $h_{iB} \in [0,1]$ and $h_{iB} = pz_{iB} + \sum_j (qA_{ij})h_{jB}$ for all i, B.

Lean: h_props — proved by AIProver, job 1, session 0

```
theorem h_props {n : ℕ} {ι : Type*} [Fintype ι] (G A : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hGl : ∀ i j, G i j = p * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
    (z : Fin n → ι → ℝ) (hz : ∀ i B, 0 ≤ z i B ∧ z i B ≤ 1)
    (h : Fin n → ι → ℝ) (hh : ∀ i B, h i B = ∑ j, G i j * z j B) :
    (∀ i B, 0 ≤ h i B ∧ h i B ≤ 1) ∧
      (∀ i B, h i B = p * z i B + ∑ j, (q * A i j) * h j B) := by
  have h_nonneg : ∀ i B, 0 ≤ h i B := by
    intro i B
    rw [hh i B]
    refine Finset.sum_nonneg (fun j _ => ?_)
    have hzj := hz j B
    have hGij := hG0 i j
    nlinarith
  have h_le_one : ∀ i B, h i B ≤ 1 := by
    intro i B
    rw [hh i B]
    calc
```

```
      ∑ j, G i j * z j B ≤ ∑ j, G i j * 1 := by
        refine Finset.sum_le_sum (fun j _ => ?_)
        have hzj := hz j B
        have hGij := hG0 i j
        nlinarith
      _ = ∑ j, G i j := by simp
      _ = 1 := hGrow i
  have h_harmonic : ∀ i B, h i B = p * z i B + ∑ j, (q * A i j) * h j B := by
    intro i B
    rw [hh i B]
    have hGl_i := hGl i
    calc
      ∑ k, G i k * z k B = ∑ k, (p * (1 : Matrix (Fin n) (Fin n) ℝ) i k + q * ∑ l, A i l * G l k) * z k B
↪   := by
        refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hGl_i k]
      _ = ∑ k, (p * (1 : Matrix (Fin n) (Fin n) ℝ) i k * z k B + q * (∑ l, A i l * G l k) * z k B) := by
        refine Finset.sum_congr rfl (fun k _ => ?_)
        ring
      _ = (∑ k, p * (1 : Matrix (Fin n) (Fin n) ℝ) i k * z k B) + (∑ k, q * (∑ l, A i l * G l k) * z k
↪   B) := by
        rw [Finset.sum_add_distrib]
      _ = p * z i B + (∑ k, q * (∑ l, A i l * G l k) * z k B) := by
        simp [Matrix.one_apply]
      _ = p * z i B + (∑ l, ∑ k, q * A i l * G l k * z k B) := by
        rw [Finset.sum_comm]
        simp [Finset.mul_sum, mul_comm, mul_left_comm]
      _ = p * z i B + (∑ l, q * A i l * (∑ k, G l k * z k B)) := by
        simp [Finset.mul_sum, mul_assoc]
      _ = p * z i B + ∑ l, (q * A i l) * h l B := by
        refine congrArg (fun t => p * z i B + t) ?_
        refine Finset.sum_congr rfl (fun l _ => ?_)
        simp [hh l B, mul_assoc]
  refine ⟨fun i B => ⟨h_nonneg i B, h_le_one i B⟩, h_harmonic⟩
```

Proof. h_{iB} is a convex combination of numbers in $[0,1]$, by the nonnegativity and row sums of G.

For the second identity, substitute the identity for G_{ik} into $\sum_k G_{ik} z_{kB}$. This gives $p z_{iB} + \sum_k q(\sum_l A_{il} G_{lk}) z_{kB}$. Exchanging the finite sums gives $p z_{iB} + \sum_l q A_{il} h_{lB}$. □

Lemma 3.31 (`cut_coord_upper_shift`). *Let $a : [n] \to \mathbb{R}$ and $m < c$ be reals. Assume $a_i \geq m$ for all i, and $a_i < c \Rightarrow a_i = m$. Let $b' : \mathcal{P}([n]) \to \mathbb{R}$ satisfy:*

- $b' \geq 0$;
- *if $b'_B \neq 0$, $j \in B$ and $\max(a_j, c) \leq \max(a_i, c)$, then $i \in B$;*
- $\max(a_i, c) - \max(a_j, c) = \sum_B b'_B(\mathbf{1}_B(i) - \mathbf{1}_B(j))$ *for all i, j.*

Then there is $b : \mathcal{P}([n]) \to \mathbb{R}$ with:

- $b \geq 0$;
- *if $b_B \neq 0$, $j \in B$ and $a_j \leq a_i$, then $i \in B$;*
- $a_i - a_j = \sum_B b_B(\mathbf{1}_B(i) - \mathbf{1}_B(j))$ *for all i, j.*

Lean: cut_coord_upper_shift — proved by the captain after a hand-back

```
theorem cut_coord_upper_shift {n : ℕ} (a : Fin n → ℝ) (m c : ℝ) (hmc : m < c)
    (hm : ∀ i, m ≤ a i) (hc : ∀ i, a i < c → a i = m) (b' : Finset (Fin n) → ℝ)
    (hb0 : ∀ B, 0 ≤ b' B)
    (hbup : ∀ B, b' B ≠ 0 → ∀ i j, j ∈ B → max (a j) c ≤ max (a i) c → i ∈ B)
    (hbsum : ∀ i j, max (a i) c - max (a j) c
      = ∑ B, b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) := by
  obtain ⟨Bstar, hBstar_mem⟩ : ∃ S : Finset (Fin n), ∀ i, i ∈ S ↔ c ≤ a i :=
    ⟨Finset.univ.filter (fun i => c ≤ a i), fun i => by simp⟩
  have hidx : ∀ k, a k = max (a k) c + (c - m) * ((if k ∈ Bstar then (1:ℝ) else 0) - 1) := by
    intro k
    by_cases hk : c ≤ a k
    · rw [if_pos ((hBstar_mem k).2 hk), max_eq_left hk]
      ring
    · have hk2 : a k < c := lt_of_not_ge hk
      have hk3 : a k = m := hc k hk2
      rw [if_neg (fun h => hk ((hBstar_mem k).1 h)), max_eq_right hk2.le]
      rw [hk3]
      ring
  refine ⟨fun B => b' B + (if B = Bstar then c - m else 0), ?_, ?_, ?_⟩
  · intro B
    show 0 ≤ b' B + (if B = Bstar then c - m else 0)
    have h1 := hb0 B
    split_ifs <;> linarith
  · intro B hB i j hj hle
    by_cases hB' : B = Bstar
    · rw [hB'] at hj ⊢
      exact (hBstar_mem i).2 (le_trans ((hBstar_mem j).1 hj) hle)
    · have hb'ne : b' B ≠ 0 := by
        intro h0
        apply hB
        show b' B + (if B = Bstar then c - m else 0) = 0
        rw [if_neg hB', h0, add_zero]
      exact hbup B hb'ne i j hj (max_le (le_trans hle (le_max_left _ _)) (le_max_right _ _))
  · intro i j
    show a i - a j = ∑ B, (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
    have hsplit : ∀ B : Finset (Fin n), (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
        = b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
          + (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else
↪  0)) else 0) := by
      intro B
      by_cases hB : B = Bstar
      · rw [if_pos hB, if_pos hB, hB]
        ring
      · rw [if_neg hB, if_neg hB]
        ring
    have hsingle : (∑ B : Finset (Fin n), (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) -
↪  (if j ∈ Bstar then (1:ℝ) else 0)) else 0))
        = (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else 0)) := by
      rw [Finset.sum_eq_single Bstar (fun B _ hB => if_neg hB) (fun h => absurd (Finset.mem_univ _) h),
↪  if_pos rfl]
    have hsum : (∑ B : Finset (Fin n), (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)))
```

```
        = ∑ B : Finset (Fin n), (b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
          + (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else
↪  0)) else 0)) :=
      Finset.sum_congr rfl (fun B _ => hsplit B)
    rw [hsum, Finset.sum_add_distrib]
    linear_combination hidx i - hidx j + hbsum i j - hsingle
```

Proof. Let $B^* = \{i : c \le a_i\}$. For every k,

$$a_k = \max(a_k, c) + (c - m)(\mathbf{1}_{B^*}(k) - 1).$$

Indeed, if $c \le a_k$ both sides equal a_k; otherwise $a_k = m$ and the right side is $c - (c - m)$.

Set $b_B = b'_B + (c-m)[B = B^*]$. Then $b \ge 0$. The decomposition of $a_i - a_j$ follows by subtracting the identity for j from that for i and using the hypothesis on b'.

Upward closedness: suppose $b_B \ne 0$, $j \in B$ and $a_j \le a_i$.

- If $B = B^*$, then $c \le a_j \le a_i$, so $i \in B^*$.
- Otherwise $b'_B \ne 0$ and $\max(a_j, c) \le \max(a_i, c)$, so the hypothesis gives $i \in B$.

□

Lemma 3.32 (cut_coord_upper_reduce)**.** *Let $a : [n] \to \mathbb{R}$ be non-constant, i.e. $a_i \ne a_j$ for some i, j. Then there are reals $m < c$ such that:*

- *$a_i \ge m$ for all i;*
- *$a_i < c \Rightarrow a_i = m$;*
- *the set of values of $i \mapsto \max(a_i, c)$ has strictly smaller cardinality than the set of values of a.*

Lean: cut_coord_upper_reduce — proved by solver 2 (aiprover), round 0

```
theorem cut_coord_upper_reduce {n : ℕ} (a : Fin n → ℝ) (hne : ∃ i j, a i ≠ a j) :
    ∃ m c : ℝ, m < c ∧ (∀ i, m ≤ a i) ∧ (∀ i, a i < c → a i = m) ∧
      (Finset.univ.image (fun i => max (a i) c)).card < (Finset.univ.image a).card := by
  let s := Finset.image a Finset.univ
  have hsne : s.Nonempty := by
    rcases hne with ⟨i, j, hne'⟩
    exact ⟨a i, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
  let m := s.min' hsne
  have hm_le : ∀ i, m ≤ a i := by
    intro i
    apply Finset.min'_le _ _ (Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩)
  have hm_mem : m ∈ s := Finset.min'_mem _ hsne
  by_cases hall : ∀ i, a i = m
  · exfalso
    rcases hne with ⟨i, j, h⟩
    rw [hall i, hall j] at h
    exact h rfl
  have h_exists_gt : ∃ i, m < a i := by
    push_neg at hall
    rcases hall with ⟨i, hi⟩
    exact ⟨i, lt_of_le_of_ne (hm_le i) (Ne.symm hi)⟩
  let s' := s.erase m
```

```
  have hs'_nonempty : s'.Nonempty := by
    rcases h_exists_gt with ⟨i, hi⟩
    have hi_mem : a i ∈ s := Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩
    have hne_mem : a i ≠ m := by linarith
    exact ⟨a i, Finset.mem_erase.mpr ⟨hne_mem, hi_mem⟩⟩
  let c := s'.min' hs'_nonempty
  have hc_mem_s' : c ∈ s' := Finset.min'_mem _ hs'_nonempty
  have hc_ne_m : c ≠ m := Finset.ne_of_mem_erase hc_mem_s'
  have hc_mem_s : c ∈ s := Finset.mem_of_mem_erase hc_mem_s'
  have hc_mem_image : c ∈ Finset.image a Finset.univ := hc_mem_s
  have hmc : m < c := by
    have hle := Finset.min'_le _ _ hc_mem_s
    by_contra! h
    have heq : c = m := le_antisymm h hle
    exact hc_ne_m heq

  have h_lt_eq : ∀ i, a i < c → a i = m := by
    intro i hi_lt
    by_contra! h_ne
    have hi_mem' : a i ∈ s' := Finset.mem_erase.mpr ⟨h_ne, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
    have hle' : c ≤ a i := Finset.min'_le _ _ hi_mem'
    linarith




  have h_max_image_eq_erase : Finset.image (fun i => max (a i) c) Finset.univ = s.erase m := by
    ext x
    constructor
    · intro hx
      rcases Finset.mem_image.mp hx with ⟨i, _, rfl⟩
      rw [Finset.mem_erase]
      by_cases hle : a i ≤ c
      · rw [max_eq_right hle]
        exact ⟨hc_ne_m, hc_mem_image⟩
      · rw [max_eq_left (by linarith)]
        refine ⟨by
          intro h_eq
          rw [h_eq] at hle
          linarith, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
    · intro hx
      rcases Finset.mem_erase.mp hx with ⟨hx_ne_m, hx_mem⟩
      rcases Finset.mem_image.mp hx_mem with ⟨i, _, rfl⟩
      by_cases hle : a i ≤ c
      ·

        have heq : a i = c := by
          by_contra! h_lt
          have : a i < c := lt_of_le_of_ne hle h_lt
          have : a i = m := h_lt_eq i this
          exact hx_ne_m this
        have hmax : max (a i) c = a i := by rw [heq, max_self]
        exact Finset.mem_image.mpr ⟨i, Finset.mem_univ _, hmax⟩
      ·
        have hmax : max (a i) c = a i := max_eq_left (by linarith)
        exact Finset.mem_image.mpr ⟨i, Finset.mem_univ _, hmax⟩
  have h_max_image_card_lt : (Finset.image (fun i => max (a i) c) Finset.univ).card < (Finset.image a
↪  Finset.univ).card := by
    rw [h_max_image_eq_erase]
```

```
    exact Finset.card_erase_lt_of_mem hm_mem
  exact ⟨m, c, hmc, hm_le, h_lt_eq, h_max_image_card_lt⟩
```

Proof. Let V be the set of values of a, $m = \min V$, and $c = \min(V \setminus \{m\})$. The set $V \setminus \{m\}$ is nonempty because a is non-constant. Then $m < c$, and any value below c equals m.

The values of $\max(a, c)$ form exactly $V \setminus \{m\}$:

- $\max(a_i, c)$ equals c if $a_i \le c$, and equals $a_i > c > m$ otherwise; in both cases it lies in $V \setminus \{m\}$;
- a value $a_i \neq m$ satisfies $a_i \ge c$, so $\max(a_i, c) = a_i$.

Removing $m \in V$ lowers the cardinality. □

Lemma 3.33 (cut_coord_upper). *For every $a : [n] \to \mathbb{R}$ there is $b : \mathcal{P}([n]) \to \mathbb{R}$ with the three properties in the conclusion of Lemma 3.31.*

Lean: cut_coord_upper — proved by the captain after a hand-back

```
theorem cut_coord_upper {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) := by
  suffices H : ∀ k : ℕ, ∀ a : Fin n → ℝ, (Finset.univ.image a).card ≤ k →
      ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
        (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
        (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) from
    H _ a le_rfl
  intro k
  induction k with
  | zero =>
    intro a ha
    by_cases hall : ∀ i j, a i = a j
    · exact ⟨fun _ => 0, fun _ => le_rfl, fun B hB => absurd rfl hB, fun i j => by simp [hall i j]⟩
    · push_neg at hall
      obtain ⟨m, c, -, -, -, hcard⟩ := cut_coord_upper_reduce a hall
      omega
  | succ k ih =>
    intro a ha
    by_cases hall : ∀ i j, a i = a j
    · exact ⟨fun _ => 0, fun _ => le_rfl, fun B hB => absurd rfl hB, fun i j => by simp [hall i j]⟩
    · push_neg at hall
      obtain ⟨m, c, hmc, hm, hc, hcard⟩ := cut_coord_upper_reduce a hall
      obtain ⟨b', hb0, hbup, hbsum⟩ := ih (fun i => max (a i) c) (by omega)
      exact cut_coord_upper_shift a m c hmc hm hc b' hb0 hbup hbsum
```

Proof. We prove by induction on k that the claim holds for all a taking at most k values.

If a is constant, take $b = 0$. Otherwise Lemma 3.32 gives $m < c$.

- For $k = 0$, the cardinality inequality is impossible.
- For $k + 1$, the function $\max(a, c)$ takes at most k values. The induction hypothesis gives b' for it, and Lemma 3.31 produces b.

□

Lemma 3.34 (cut_coord_core)**.** *For every $a : [n] \to \mathbb{R}$ there is $b : \mathcal{P}([n]) \to \mathbb{R}$ with $b \geq 0$ such that, for all i, j,*

$$a_i - a_j = \sum_B b_B(\mathbf{1}_B(i) - \mathbf{1}_B(j)), \qquad |a_i - a_j| = \sum_B b_B|\mathbf{1}_B(i) - \mathbf{1}_B(j)|.$$

Lean: cut_coord_core — proved by the captain after a hand-back

```
theorem cut_coord_core {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) ∧
      (∀ i j, |a i - a j| = ∑ B, b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) := by
  obtain ⟨b, hb0, hup, hdiff⟩ := cut_coord_upper a
  refine ⟨b, hb0, hdiff, ?_⟩
  have key : ∀ i j, a j ≤ a i → ∀ B : Finset (Fin n),
      b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
        = b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| := by
    intro i j hij B
    by_cases hb : b B = 0
    · simp [hb]
    · have h := hup B hb i j
      by_cases hj : j ∈ B
      · have hi := h hj hij
        simp [hi, hj]
      · by_cases hi : i ∈ B <;> simp [hi, hj]
  intro i j
  rcases le_total (a j) (a i) with h | h
  · rw [abs_of_nonneg (sub_nonneg.mpr h), hdiff i j]
    exact Finset.sum_congr rfl (fun B _ => key i j h B)
  · rw [abs_sub_comm (a i) (a j), abs_of_nonneg (sub_nonneg.mpr h), hdiff j i]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    rw [key j i h B, abs_sub_comm]
```

Proof. Take b from Lemma 3.33. Suppose $a_j \leq a_i$. For each B,

$$b_B(\mathbf{1}_B(i) - \mathbf{1}_B(j)) = b_B|\mathbf{1}_B(i) - \mathbf{1}_B(j)|.$$

This is trivial if $b_B = 0$. If $b_B \neq 0$ and $j \in B$, then $i \in B$ and both sides vanish. If $j \notin B$, the difference $\mathbf{1}_B(i)$ is nonnegative.

Hence $|a_i - a_j| = a_i - a_j = \sum_B b_B|\cdots|$. The case $a_i \leq a_j$ follows by symmetry. □

Lemma 3.35 (cut_coord)**.** *For every $a : [n] \to \mathbb{R}$ there is $b : \mathcal{P}([n]) \to \mathbb{R}$ satisfying:*

- *$b \geq 0$;*
- *$b_B \leq 2\sum_i |a_i|$ for all B;*
- *both identities of Lemma 3.34.*

Lean: cut_coord — proved by the captain after a hand-back

```
theorem cut_coord {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧ (∀ B, b B ≤ 2 * ∑ i, |a i|) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) ∧
      (∀ i j, |a i - a j| = ∑ B, b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) := by
```

```
classical
obtain ⟨b, hb0, hsgn, habs⟩ := cut_coord_core a
refine ⟨fun B => if (∃ i, i ∈ B) ∧ (∃ j, j ∉ B) then b B else 0, ?_, ?_, ?_, ?_⟩
· intro B
  dsimp only
  split_ifs
  · exact hb0 B
  · exact le_refl 0
· intro B
  dsimp only
  have hS : 0 ≤ ∑ i, |a i| := Finset.sum_nonneg (fun i _ => abs_nonneg _)
  split_ifs with h
  · obtain ⟨⟨i, hi⟩, ⟨j, hj⟩⟩ := h
    have h1 := habs i j
    have hle : b B ≤ |a i - a j| := by
      rw [h1]
      have := Finset.single_le_sum
        (f := fun B' => b B' * |(if i ∈ B' then (1:ℝ) else 0) - (if j ∈ B' then (1:ℝ) else 0)|)
        (fun B' _ => mul_nonneg (hb0 B') (abs_nonneg _)) (Finset.mem_univ B)
      simpa [hi, hj] using this
    have h2 : |a i - a j| ≤ |a i| + |a j| :=
      abs_sub_le_iff.mpr ⟨by linarith [le_abs_self (a i), neg_abs_le (a j)],
        by linarith [le_abs_self (a j), neg_abs_le (a i)]⟩
    have hi' := Finset.single_le_sum (f := fun k => |a k|) (fun k _ => abs_nonneg _) (Finset.mem_univ i)
    have hj' := Finset.single_le_sum (f := fun k => |a k|) (fun k _ => abs_nonneg _) (Finset.mem_univ j)
    simp only at hi' hj'
    linarith
  · linarith
· intro i j
  rw [hsgn i j]
  refine Finset.sum_congr rfl (fun B _ => ?_)
  dsimp only
  by_cases h : (∃ i, i ∈ B) ∧ (∃ j, j ∉ B)
  · rw [if_pos h]
  · rw [if_neg h]
    have hd : (if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0) = 0 := by
      by_cases hi : i ∈ B
      · have hj : j ∈ B := by
          by_contra hj
          exact h ⟨⟨i, hi⟩, ⟨j, hj⟩⟩
        simp [hi, hj]
      · have hj : j ∉ B := by
          intro hj
          exact h ⟨⟨j, hj⟩, ⟨i, hi⟩⟩
        simp [hi, hj]
    rw [hd]
    ring
· intro i j
  rw [habs i j]
  refine Finset.sum_congr rfl (fun B _ => ?_)
  dsimp only
  by_cases h : (∃ i, i ∈ B) ∧ (∃ j, j ∉ B)
  · rw [if_pos h]
  · rw [if_neg h]
    have hd : (if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0) = 0 := by
      by_cases hi : i ∈ B
      · have hj : j ∈ B := by
          by_contra hj
          exact h ⟨⟨i, hi⟩, ⟨j, hj⟩⟩
```

```
          simp [hi, hj]
        · have hj : j ∉ B := by
            intro hj
            exact h ⟨⟨j, hj⟩, ⟨i, hi⟩⟩
          simp [hi, hj]
      rw [hd, abs_zero]
      ring
```

Proof. Take b from Lemma 3.34. Replace b_B by 0 unless B is nonempty and proper, i.e. unless some $i \in B$ and some $j \notin B$.

For the excluded B, $\mathbf{1}_B(i) - \mathbf{1}_B(j) = 0$ for all i, j, so both identities persist.

For a proper nonempty B, pick $i \in B$ and $j \notin B$. The absolute identity gives

$$b_B \le |a_i - a_j| \le |a_i| + |a_j| \le 2\sum |a|.$$

□

Lemma 3.36 (cut_l1_ext)**.** *If $f, g \in \ell_1$ satisfy $f(k) = g(k)$ for all k, then $f = g$.*

Lean: cut_l1_ext — proved by AIProver, job 1, session 0

```
theorem cut_l1_ext (f g : L1Space) (h : ∀ k, f k = g k) : f = g := by
  ext k
  exact h k
```

Proof. Elements of ℓ_1 are determined by their coordinates. □

Lemma 3.37 (cut_l1_apply)**.** *Let ι be finite, $x_0 \in \ell_1$, $c : \iota \to \mathbb{R}$ and $\beta : \iota \to \ell_1$. For every k,*

$$\Big(x_0 + \sum_B c_B \beta_B\Big)(k) = x_0(k) + \sum_B c_B \beta_B(k).$$

Lean: cut_l1_apply — proved by AIProver, job 1, session 0

```
theorem cut_l1_apply {ι : Type*} [Fintype ι] (x0 : L1Space) (c : ι → ℝ) (β : ι → L1Space) (k : ℕ) :
    (x0 + ∑ B, c B • β B) k = x0 k + ∑ B, c B * β B k := by
  simp
```

Proof. Evaluation at k is linear. □

Lemma 3.38 (assembly_real_split)**.** *For arbitrary π, A, $p, q \in \mathbb{R}$, $a : [n] \to \mathbb{R}$ and $b : [n] \times [n] \to \mathbb{R}$,*

$$\sum_i \pi_i \Big(p a_i + q \sum_j A_{ij} b_{ij}\Big) = p \sum_i \pi_i a_i + q \sum_{i,j} \pi_i A_{ij} b_{ij}.$$

Lean: assembly_real_split — helper of assembly_real, proved by the captain after a hand-back

```
theorem assembly_real_split {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ)
    (a : Fin n → ℝ) (b : Fin n → Fin n → ℝ) :
    ∑ i, π i * (p * a i + q * ∑ j, A i j * b i j)
      = p * ∑ i, π i * a i + q * ∑ i, ∑ j, π i * A i j * b i j := by
  rw [Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun i _ => ?_)
  have e : ∑ j, π i * A i j * b i j = π i * ∑ j, A i j * b i j := by
    rw [Finset.mul_sum]
    exact Finset.sum_congr rfl (fun j _ => by ring)
  rw [e]
  ring
```

Proof. Linearity of finite sums. □

Lemma 3.39 (assembly_real_finish)**.** *Let $q, \tau, S_1, S_2, R \in \mathbb{R}$ with $q < 1$ and $\tau(1-q) = q$. If $(1-q)S_1 + qS_2 \le 108(1-q)R$, then $S_1 + \tau S_2 \le 108R$.*

Lean: assembly_real_finish — helper of assembly_real, proved by the captain after a hand-back

```
theorem assembly_real_finish (q τ S1 S2 R : ℝ) (hq1 : q < 1) (hτ : τ * (1 - q) = q)
    (h : (1 - q) * S1 + q * S2 ≤ 108 * (1 - q) * R) : S1 + τ * S2 ≤ 108 * R := by
  have hpos : 0 < 1 - q := by linarith
  have e : (1 - q) * (S1 + τ * S2) = (1 - q) * S1 + q * S2 := by
    linear_combination S2 * hτ
  have h2 : (1 - q) * (S1 + τ * S2) ≤ (1 - q) * (108 * R) := by
    rw [e]; linarith
  exact le_of_mul_le_mul_left h2 hpos
```

Proof. We have $(1-q)(S_1 + \tau S_2) = (1-q)S_1 + qS_2$. Divide by $1 - q > 0$. □

Lemma 3.40 (assembly_real)**.** *Assume the following:*

- *$\pi \ge 0$ and A is stochastic;*
- *ι is finite and $w : \iota \to \mathbb{R}$ satisfies $w \ge 0$;*
- *$z : [n] \to (\iota \to \mathbb{R})$ with $z_{iB} \in \{0, 1\}$;*
- *$0 \le q < 1$ and $\tau(1-q) = q$;*
- *$G \in \mathbb{R}^{[n]\times[n]}$ satisfies $G \ge 0$, $\sum_j G_{ij} = 1$, $\sum_i \pi_i G_{ij} = \pi_j$, and both identities of Lemmas 3.22 and 3.23.*

Write $\phi(r) = 3r^2 - 2r^3$. Then there exists $h : [n] \to (\iota \to \mathbb{R})$ with

$$\sum_i \pi_i \Big(\sum_B w_B |z_{iB} - \phi(h_{iB})| \Big)^2 + \tau \sum_{i,j} \pi_i A_{ij} \Big(\sum_B w_B |\phi(h_{iB}) - \phi(h_{jB})| \Big)^2 \le 108 \sum_{i,j} \pi_i G_{ij} \Big(\sum_B w_B |z_{iB} - z_{jB}| \Big)^2 .$$

Lean: `assembly_real` — proved by the captain after a hand-back

```
theorem assembly_real {n : ℕ} {ι : Type*} [Fintype ι] (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B) (z : Fin n → ι → ℝ) (hz : ∀ i B, z i B = 0 ∨ z i B = 1)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (τ : ℝ) (hτ : τ * (1 - q) = q)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (hGl : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
    (hGr : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j) :
    ∃ h : Fin n → ι → ℝ,
      (∑ i, π i * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
        + τ * (∑ i, ∑ j, π i * A i j *
            (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
        ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ B, w B * |z i B - z j B|) ^ 2 := by
  have hp0 : 0 ≤ 1 - q := by linarith
  have hz01 : ∀ i B, 0 ≤ z i B ∧ z i B ≤ 1 := by
    intro i B
    rcases hz i B with e | e <;> rw [e] <;> norm_num
  obtain ⟨h, hh⟩ : ∃ h : Fin n → ι → ℝ, ∀ i B, h i B = ∑ j, G i j * z j B :=
    ⟨fun i B => ∑ j, G i j * z j B, fun _ _ => rfl⟩
  obtain ⟨hhb, hhbal⟩ := h_props G A (1 - q) q hG0 hGrow hGl z hz01 h hh
  obtain ⟨φ, hφ⟩ : ∃ φ : ℝ → ℝ, ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3 := ⟨_, fun _ => rfl⟩
  have hrow : ∀ i, (1 - q) + ∑ j, q * A i j = 1 := by
    intro i
    rw [← Finset.mul_sum, hA.2 i]
    ring
  have hloc : ∀ k i, (1 - q) * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2
     + q * ∑ j, A i j * (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2
      ≤ 108 * ((1 - q) * (∑ B, w B * |z k B - z i B|) ^ 2
        + q * ∑ j, A i j * (∑ B, w B * (h j B - z k B) ^ 2) ^ 2
        - (∑ B, w B * (h i B - z k B) ^ 2) ^ 2) := by
    intro k i
    have key := local_ineq w hw φ hφ (z k) (h i) (z i) h (1 - q) (fun j => q * A i j)
      hp0 (fun j => mul_nonneg hq0 (hA.1 i j)) (hrow i) (hhbal i) (hz k)
      (hhb i) (hz01 i) hhb
    have e1 : ∀ B, |3 * z i B ^ 2 - 2 * z i B ^ 3 - (3 * h i B ^ 2 - 2 * h i B ^ 3)|
        = |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)| := by
      intro B
      rcases hz i B with e | e <;> rw [e] <;> norm_num
    have e2 : ∀ j B, |3 * h j B ^ 2 - 2 * h j B ^ 3 - (3 * h i B ^ 2 - 2 * h i B ^ 3)|
        = |3 * h i B ^ 2 - 2 * h i B ^ 3 - (3 * h j B ^ 2 - 2 * h j B ^ 3)| :=
      fun j B => abs_sub_comm _ _
    have e3 : ∀ B, (z i B - z k B) ^ 2 = |z k B - z i B| := by
      intro B
      rcases hz i B with e | e <;> rcases hz k B with e' | e' <;> rw [e, e'] <;> norm_num
    simp only [hφ, e1, e2, e3] at key
    have c1 : ∀ (f : Fin n → ℝ), ∑ j, q * A i j * f j = q * ∑ j, A i j * f j := by
      intro f
      rw [Finset.mul_sum]
      exact Finset.sum_congr rfl (fun j _ => by ring)
    rw [c1, c1] at key
    exact key
  have gs := global_sum π hπ A G (1 - q) q hp0 hG0 hGπ hGr
    (fun i => (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
    (fun i j => (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
    (fun k i => (∑ B, w B * |z k B - z i B|) ^ 2)
    (fun k i => (∑ B, w B * (h i B - z k B) ^ 2) ^ 2)
    (fun k i => sq_nonneg _) hloc
```

```
  have hL := assembly_real_split π A (1 - q) q
    (fun i => (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
    (fun i j => (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
  refine ⟨h, assembly_real_finish q τ _ _ _ hq1 hτ ?_⟩
  linarith [gs, hL]
```

Proof. Let $h_{iB} = \sum_j G_{ij} z_{jB}$. By Lemma 3.30 with $p = 1 - q$, using the left identity, $h_{iB} \in [0, 1]$ and

$$h_{iB} = (1 - q) z_{iB} + \sum_j (q A_{ij}) h_{jB}.$$

The weights $1 - q$ and $q A_{ij}$ are nonnegative and sum to 1 because A is stochastic.

Fix k, i and apply Lemma 3.28 with $e = z_k$, $u = h_i$, $v = z_i$, $U_j = h_j$, $p = 1 - q$ and $c_j = q A_{ij}$. For bits, $\phi(z_{iB}) = z_{iB}$ and $(z_{iB} - z_{kB})^2 = |z_{kB} - z_{iB}|$. Hence the hypothesis of Lemma 3.29 holds with

$$J_i^d = \Big(\sum_B w_B |z_{iB} - \phi(h_{iB})|\Big)^2, \qquad J_{ij}^h = \Big(\sum_B w_B |\phi(h_{iB}) - \phi(h_{jB})|\Big)^2,$$

$$F_{ki}^z = \Big(\sum_B w_B |z_{kB} - z_{iB}|\Big)^2, \qquad F_{ki}^h = \Big(\sum_B w_B (h_{iB} - z_{kB})^2\Big)^2 \geq 0.$$

Lemma 3.29 applies with the right identity and stationarity. It gives

$$\sum_i \pi_i \big((1 - q) J_i^d + q \textstyle\sum_j A_{ij} J_{ij}^h\big) \leq 108 (1 - q) R,$$

where R is the double sum on the right of the claim. Split the left side by Lemma 3.38 and conclude with Lemma 3.39. □

Lemma 3.41 (cut_l1_norm_combo). *Let ι be finite, $f \in \ell_1$, $\beta : \iota \to \ell_1$ and $d : \iota \to \mathbb{R}$. Suppose $\beta_B(k) \geq 0$ for all B, k and $|f(k)| = \sum_B \beta_B(k) d_B$ for all k. Then*

$$\|f\| = \sum_B \|\beta_B\| d_B.$$

Lean: cut_l1_norm_combo — proved by AIProver, job 1, session 0

```
theorem cut_l1_norm_combo {ι : Type*} [Fintype ι] (f : L1Space) (β : ι → L1Space) (d : ι → ℝ)
    (hβ : ∀ B k, 0 ≤ β B k) (hf : ∀ k, |f k| = ∑ B, β B k * d B) :
    ‖f‖ = ∑ B, ‖β B‖ * d B := by
  have hp : 0 < (1 : ENNReal).toReal := by norm_num
  have hnorm_f : ‖f‖ = ∑' k, |f k| := by
    rw [lp.norm_eq_tsum_rpow hp f]
    simp
  have hnorm_β : ∀ B, ‖β B‖ = ∑' k, |β B k| := by
    intro B
    rw [lp.norm_eq_tsum_rpow hp (β B)]
    simp
  have habs_β : ∀ B k, |β B k| = β B k := by
    intro B k
    rw [abs_of_nonneg (hβ B k)]
  rw [hnorm_f]
```

```
  have h_summable_β : ∀ B, Summable (fun k : ℕ => β B k) := by
    intro B
    have h_sum_abs : Summable (fun k : ℕ => |β B k|) := by
      have h := (lp.memℓp (β B)).summable hp
      simpa using h
    have h_eq : (fun k : ℕ => |β B k|) = (fun k : ℕ => β B k) := by
      ext k
      exact habs_β B k
    rwa [h_eq] at h_sum_abs

  have h_summable_mul : ∀ B, Summable (fun k : ℕ => β B k * d B) := by
    intro B
    exact Summable.mul_right (d B) (h_summable_β B)

  have h_exchange := Summable.tsum_finsetSum (s := Finset.univ)
    (fun B _ => h_summable_mul B)

  calc
    (∑' k, |f k|) = (∑' k, ∑ B, β B k * d B) := by
      refine tsum_congr (fun k => ?_)
      rw [hf k]
    _ = (∑' k, ∑ B ∈ (Finset.univ : Finset ι), β B k * d B) := rfl
    _ = (∑ B ∈ (Finset.univ : Finset ι), ∑' k, β B k * d B) := by
      rw [h_exchange]
    _ = (∑ B, ∑' k, β B k * d B) := rfl
    _ = (∑ B, (∑' k, β B k) * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      rw [tsum_mul_right]
    _ = (∑ B, (∑' k, |β B k|) * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      simp_rw [habs_β B]
    _ = (∑ B, ‖β B‖ * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      rw [hnorm_β B]
```

Proof. We have $\|f\| = \sum_k |f(k)| = \sum_k \sum_B \beta_B(k) d_B$. Each series $\sum_k \beta_B(k)$ converges, because $\beta_B \in \ell_1$. Hence the finite sum and the series can be exchanged, giving $\sum_B d_B \sum_k \beta_B(k)$. Finally $\sum_k \beta_B(k) = \sum_k |\beta_B(k)| = \|\beta_B\|$, since $\beta_B \geq 0$. □

Lemma 3.42 (cut_l1_vectors_summable_fsum)**.** *Let $f : \iota \to (\mathbb{N} \to \mathbb{R})$ with each f_i summable, and let $s \subseteq \iota$ be finite. Then $k \mapsto \sum_{i \in s} f_i(k)$ is summable.*

Lean: cut_l1_vectors_summable_fsum — helper of cut_l1_vectors, proved by the captain after a hand-back

```
theorem cut_l1_vectors_summable_fsum {ι : Type*} (s : Finset ι) (f : ι → ℕ → ℝ)
    (hf : ∀ i, Summable (f i)) : Summable (fun k => ∑ i ∈ s, f i k) := by
  classical
  refine Finset.induction_on s ?_ ?_
  · simp only [Finset.sum_empty]
    exact summable_zero
  · intro a s ha ih
    simp only [Finset.sum_insert ha]
    exact (hf a).add ih
```

Proof. Induction on s; sums of summable sequences are summable. □

Lemma 3.43 (cut_l1_vectors). *For every n and $x : [n] \to \ell_1$ there is $\beta : \mathcal{P}([n]) \to \ell_1$ such that $\beta_B(k) \geq 0$ for all B, k, and for all i, j, k:*

$$x_i(k) = x_j(k) + \sum_B (\mathbf{1}_B(i) - \mathbf{1}_B(j))\beta_B(k), \qquad |(x_i - x_j)(k)| = \sum_B \beta_B(k)|\mathbf{1}_B(i) - \mathbf{1}_B(j)|.$$

Lean: cut_l1_vectors — proved by the captain after a hand-back

```
theorem cut_l1_vectors (n : ℕ) (x : Fin n → L1Space) :
    ∃ β : Finset (Fin n) → L1Space, (∀ B k, 0 ≤ β B k) ∧
      (∀ i j k, x i k = x j k + ∑ B, ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) * β B
↪  k) ∧
      (∀ i j k, |(x i - x j) k| = ∑ B, β B k * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else
↪  0)|) := by
  choose b hb0 hble hdiff habs using fun k : ℕ => cut_coord (fun i : Fin n => (x i : ℕ → ℝ) k)
  have hx : ∀ i, Summable (fun k => |(x i : ℕ → ℝ) k|) := by
    intro i
    have h := (memℓp_gen_iff (p := (1:ENNReal)) (by simp)).1 (lp.memℓp (x i))
    simpa [Real.norm_eq_abs] using h
  have hS : Summable (fun k => 2 * ∑ i, |(x i : ℕ → ℝ) k|) :=
    (cut_l1_vectors_summable_fsum Finset.univ (fun i k => |(x i : ℕ → ℝ) k|) hx).mul_left 2
  have hmem : ∀ B, Memℓp (fun k => b k B) 1 := by
    intro B
    apply (memℓp_gen_iff (p := (1:ENNReal)) (by simp)).2
    have hs : Summable (fun k => b k B) := by
      refine Summable.of_nonneg_of_le (fun k => hb0 k B) (fun k => ?_) hS
      exact hble k B
    have e : (fun k => ‖b k B‖ ^ (1:ENNReal).toReal) = fun k => b k B := by
      funext k
      simp [Real.norm_eq_abs, abs_of_nonneg (hb0 k B)]
    rw [e]
    exact hs
  obtain ⟨β, hβ⟩ : ∃ β : Finset (Fin n) → L1Space, ∀ B k, β B k = b k B :=
    ⟨fun B => ⟨fun k => b k B, hmem B⟩, fun _ _ => rfl⟩
  refine ⟨β, fun B k => by rw [hβ]; exact hb0 k B, ?_, ?_⟩
  · intro i j k
    have h : (x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k
        = ∑ B, b k B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) := hdiff k i j
    have e : ∑ B, ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) * β B k
        = ∑ B, b k B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) :=
      Finset.sum_congr rfl (fun B _ => by rw [hβ, mul_comm])
    rw [e]
    linarith [h]
  · intro i j k
    have h : |(x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k|
        = ∑ B, b k B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| := habs k i j
    have hsub : (x i - x j) k = (x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k := by
      first
      | rfl
      | simp [lp.coeFn_sub]
    rw [hsub, h]
    exact Finset.sum_congr rfl (fun B _ => by rw [hβ])
```

Proof. For each coordinate k, apply Lemma 3.35 to $a = (x_i(k))_i$ and choose the resulting $b^{(k)}$. Put $\beta_B(k) = b^{(k)}_B$.

Then $0 \le \beta_B(k) \le 2\sum_i |x_i(k)|$. The right side is summable in k by Lemma 3.42, since each $x_i \in \ell_1$. Hence $\beta_B \in \ell_1$.

The two identities are those of Lemma 3.35 at coordinate k, using $(x_i - x_j)(k) = x_i(k) - x_j(k)$. □

Lemma 3.44 (cut_l1)**.** *Let $n > 0$ and $x : [n] \to \ell_1$. Then there exist $w : \mathcal{P}([n]) \to \mathbb{R}$, $z : [n] \to (\mathcal{P}([n]) \to \mathbb{R})$ and $T : \mathbb{R}^{\mathcal{P}([n])} \to \ell_1$ such that:*

- *$w \ge 0$ and $z_{iB} \in \{0, 1\}$;*
- *$T(z_i) = x_i$;*
- *$\|T(u) - T(u')\| \le \sum_B w_B |u_B - u'_B|$ for all u, u';*
- *$\|x_i - x_j\| = \sum_B w_B |z_{iB} - z_{jB}|$ for all i, j.*

Lean: cut_l1 — proved by the captain after a hand-back

```
theorem cut_l1 (n : ℕ) (hn : 0 < n) (x : Fin n → L1Space) :
    ∃ (w : Finset (Fin n) → ℝ) (z : Fin n → Finset (Fin n) → ℝ)
      (T : (Finset (Fin n) → ℝ) → L1Space),
      (∀ B, 0 ≤ w B) ∧ (∀ i B, z i B = 0 ∨ z i B = 1) ∧ (∀ i, T (z i) = x i) ∧
      (∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|) ∧
      (∀ i j, ‖x i - x j‖ = ∑ B, w B * |z i B - z j B|) := by
  obtain ⟨β, hβ0, hβd, hβa⟩ := cut_l1_vectors n x
  obtain ⟨ind, hind⟩ : ∃ ind : Fin n → Finset (Fin n) → ℝ,
      ∀ i B, ind i B = if i ∈ B then (1:ℝ) else 0 := ⟨_, fun _ _ => rfl⟩
  let i0 : Fin n := ⟨0, hn⟩
  obtain ⟨T, hT⟩ : ∃ T : (Finset (Fin n) → ℝ) → L1Space,
      ∀ u, T u = x i0 + ∑ B, (u B - ind i0 B) • β B := ⟨_, fun _ => rfl⟩
  refine ⟨fun B => ‖β B‖, ind, T, fun B => norm_nonneg _, ?_, ?_, ?_, ?_⟩
  · intro i B
    rw [hind]
    split_ifs <;> simp
  · intro i
    rw [hT]
    apply cut_l1_ext
    intro k
    rw [cut_l1_apply]
    simp only [hind]
    exact (hβd i i0 k).symm
  · intro u u'
    rw [hT, hT, add_sub_add_left_eq_sub, ← Finset.sum_sub_distrib]
    refine (norm_sum_le _ _).trans (le_of_eq ?_)
    refine Finset.sum_congr rfl (fun B _ => ?_)
    rw [← sub_smul, sub_sub_sub_cancel_right, norm_smul, Real.norm_eq_abs, mul_comm]
  · intro i j
    exact cut_l1_norm_combo (x i - x j) β (fun B => |ind i B - ind j B|) hβ0
      (fun k => by simp only [hind]; exact hβa i j k)
```

Proof. Take β from Lemma 3.43 and let i_0 be the first index (it exists because $n > 0$). Set

$$w_B = \|\beta_B\|, \qquad z_{iB} = \mathbf{1}_B(i), \qquad T(u) = x_{i_0} + \sum_B (u_B - z_{i_0 B})\beta_B.$$

We check the properties in turn.

- $T(z_i) = x_i$: compare coordinates, using Lemma 3.37, the signed identity with $j = i_0$, and Lemma 3.36.
- Contraction: $T(u) - T(u') = \sum_B (u_B - u'_B)\beta_B$, and the triangle inequality bounds its norm by $\sum_B |u_B - u'_B| \|\beta_B\|$.
- Distance identity: Lemma 3.41 with $f = x_i - x_j$ and $d_B = |z_{iB} - z_{jB}|$.

□

Lemma 3.45 (assembly). *Assume the hypotheses of Lemma 3.40 on π, A, w, z, q, τ and G. Assume moreover that $x : [n] \to \ell_1$ and $T : \mathbb{R}^\iota \to \ell_1$ satisfy:*

- $T(z_i) = x_i$;
- $\|T(u) - T(u')\| \le \sum_B w_B |u_B - u'_B|$;
- $\|x_i - x_j\| = \sum_B w_B |z_{iB} - z_{jB}|$.

Then there is $y : [n] \to \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + \tau \sum_{i,j} \pi_i A_{ij} \|y_i - y_j\|^2 \le 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2.$$

Lean: `assembly` — proved by the captain after a hand-back

```
theorem assembly {n : ℕ} {ι : Type*} [Fintype ι] (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (x : Fin n → L1Space)
    (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B) (z : Fin n → ι → ℝ) (hz : ∀ i B, z i B = 0 ∨ z i B = 1)
    (T : (ι → ℝ) → L1Space) (hTz : ∀ i, T (z i) = x i)
    (hT : ∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|)
    (hdist : ∀ i j, ‖x i - x j‖ = ∑ B, w B * |z i B - z j B|)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (τ : ℝ) (hτ : τ * (1 - q) = q)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (hGl : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
    (hGr : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j) :
    ∃ y : Fin n → L1Space,
      (∑ i, π i * ‖x i - y i‖ ^ 2) + τ * (∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2)
        ≤ 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by
  obtain ⟨h, hh⟩ := assembly_real π hπ A hA w hw z hz q hq0 hq1 τ hτ G hG0 hGrow hGπ hGl hGr
  refine ⟨fun i => T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3), ?_⟩
  have hq1' : 0 < 1 - q := by linarith
  have hτ0 : 0 ≤ τ := by
    by_contra hneg
    push_neg at hneg
    nlinarith [mul_neg_of_neg_of_pos hneg hq1']
  have h1 : ∀ i, ‖x i - T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)‖ ^ 2
      ≤ (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2 := by
    intro i
    have h' : ‖x i - T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)‖
        ≤ ∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)| := by
      have := hT (z i) (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)
      rw [hTz i] at this
      exact this
    exact pow_le_pow_left₀ (norm_nonneg _) h' 2
  have h2 : ∀ i j, ‖T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3) - T (fun B => 3 * h j B ^ 2 - 2 * h j B ^ 3)‖ ^ 2
```

```
      ≤ (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2 := by
    intro i j
    have h' : ‖T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3) - T (fun B => 3 * h j B ^ 2 - 2 * h j B ^ 3)‖
        ≤ ∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)| :=
      hT _ _
    exact pow_le_pow_left₀ (norm_nonneg _) h' 2
  calc _ ≤ (∑ i, π i * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
         + τ * (∑ i, ∑ j, π i * A i j *
             (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2) :=
       add_le_add (Finset.sum_le_sum fun i _ => mul_le_mul_of_nonneg_left (h1 i) (hπ i))
         (mul_le_mul_of_nonneg_left
           (Finset.sum_le_sum fun i _ => Finset.sum_le_sum fun j _ =>
             mul_le_mul_of_nonneg_left (h2 i j) (mul_nonneg (hπ i) (hA.1 i j))) hτ0)
    _ ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ B, w B * |z i B - z j B|) ^ 2 := hh
    _ = 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by simp only [hdist]
```

Proof. Take h from Lemma 3.40 and set $y_i = T(\phi(h_i))$, where ϕ acts coordinatewise.

By the contraction property and $T(z_i) = x_i$:

- $\|x_i - y_i\| \le \sum_B w_B |z_{iB} - \phi(h_{iB})|$;
- $\|y_i - y_j\| \le \sum_B w_B |\phi(h_{iB}) - \phi(h_{jB})|$.

Square these bounds and multiply by the nonnegative weights π_i and $\pi_i A_{ij}$. Also $\tau \ge 0$, since $\tau(1-q) = q \ge 0$ and $1 - q > 0$.

Summing, the left side is bounded by the left side of Lemma 3.40. The right side of that lemma equals $108 \sum \pi_i G_{ij} \|x_i - x_j\|^2$ by the distance identity. □

3.3 Proof of the theorem

Lean: `solution` — main proof from the captain's sketch

```
theorem solution
    (n t : ℕ) (hn : 0 < n) (ht : 0 < t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (hrev : IsReversibleWrt π A)
    (x : Fin n → L1Space) :
    ∃ y : Fin n → L1Space,
      (∑ i, π i * ‖x i - y i‖ ^ 2)
        + (t : ℝ) * (∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2)
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  obtain ⟨w, z, T, hw, hz, hTz, hT, hdist⟩ := cut_l1 n hn x
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t : ℝ) / ((t : ℝ) + 1) := ⟨_, rfl⟩
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1 : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hqt : q * ((t : ℝ) + 1) = t := by rw [hq]; field_simp
  have hq1 : q < 1 := by nlinarith [hqt, htpos]
  have hτ : (t : ℝ) * (1 - q) = q := by linear_combination (-1 : ℝ) * hqt
  obtain ⟨G, hG⟩ : ∃ G : Matrix (Fin n) (Fin n) ℝ,
      ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j :=
    ⟨fun i j => ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j, fun i j => rfl⟩
  obtain ⟨y, hy⟩ := assembly π hπ.1 A hA x w hw z hz T hTz hT hdist q hq0 hq1 (t : ℝ) hτ G
```

```
    (res_nonneg A hA q hq0 hq1 G hG) (res_rowsum A hA q hq0 hq1 G hG)
    (res_stationary π A hA hrev q hq0 hq1 G hG) (res_left A hA q hq0 hq1 G hG)
    (res_right A hA q hq0 hq1 G hG)
  have hcomp := res_comparison t ht π hπ A hA hrev x q hq G hG
  refine ⟨y, ?_⟩
  linarith
```

Proof. Let $n, t > 0$, π, A and x be as in the target statement.

Cut decomposition. Lemma 3.44 gives w, z and T with $\iota = \mathcal{P}([n])$.

Parameters. Put $q = t/(t+1)$. Then $0 \le q < 1$ and $q(t+1) = t$, so $\tau = t$ satisfies $\tau(1-q) = q$.

Resolvent. Define $G_{ij} = \sum_{s=0}^{\infty}(1-q)q^s(A^s)_{ij}$. By Lemmas 3.18, 3.19 and 3.21 to 3.23:

- $G \ge 0$ and $\sum_j G_{ij} = 1$;
- $\pi G = \pi$, which uses reversibility through Lemma 2.6;
- G satisfies both resolvent identities.

Construction. Lemma 3.45, with $\pi \ge 0$ and these data, yields $y : [n] \to \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_{i,j} \pi_i A_{ij} \|y_i - y_j\|^2 \le 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2.$$

Comparison. Lemma 3.17 bounds the right side by

$$108 \cdot 28 \sum_{i,j} \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} (A^s)_{ij} \Big) \|x_i - x_j\|^2.$$

Since $108 \cdot 28 = 3024$, this is the asserted inequality. □

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. `L1Space` is `lp` of $\mathbb{N} \to \mathbb{R}$ with exponent 1. Its norm is $\sum_k |x(k)|$, so this is the real Banach space ℓ_1. It is indexed from 0 rather than 1, which is immaterial.

`IsProbVec` requires nonnegative entries summing to one. Zero entries are allowed, as in the source.

`IsStochasticMatrix` requires nonnegative entries and every row sum equal to 1.

`IsReversibleWrt` is detailed balance: $\pi_i a_{ij} = \pi_j a_{ji}$ for all i, j.

All four definitions match the source exactly. The auxiliary `setting_consistent` exhibits an instance satisfying all three predicates, so the hypotheses are not vacuous.

Statement. Quantification is over the source's objects:

- n, t positive naturals;
- π a probability vector on $\mathrm{Fin}\, n$;
- A stochastic and reversible with respect to π;

- $x : \mathrm{Fin}\, n \to \ell_1$.

The hypothesis $0 < n$ is automatic anyway, since a probability vector on an empty index set cannot exist.

The conclusion asserts the existence of $y : \mathrm{Fin}\, n \to \ell_1$ satisfying

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_{i,j} \pi_i a_{ij} \|y_i - y_j\|^2 \le 3024 \sum_{i,j} \pi_i \Big(\tfrac{1}{t} \sum_{s \in [1,t]} (A^s)_{ij} \Big) \|x_i - x_j\|^2.$$

This is the source inequality verbatim. In particular, A^s is the matrix power, the Cesàro average runs over $s = 1, \ldots, t$, and the constant is 3024.

No extra hypotheses are present, and nothing the source proves is assumed. The statement is not trivially satisfiable: for example, $y = x$ does not in general make the left side small relative to the right.

Proof. The formal proof follows the source's architecture, with the probabilistic parts replaced by deterministic matrix identities.

- **Cuts.** `cut_coord_upper`, `cut_coord_core`, `cut_coord`, `cut_l1_vectors` and `cut_l1` carry out Lemma 1.
  - They build, coordinatewise, nonnegative cut weights indexed by all subsets of $\mathrm{Fin}\, n$, with nested-cut sign control.
  - The bound $b_B(k) \le 2 \sum_i |x_i(k)|$ gives membership in ℓ_1.
  - The affine map $T(u) = x_{i_0} + \sum_B (u_B - \mathbf{1}_{i_0 \in B}) \beta_B$ is shown to satisfy $T(z_i) = x_i$, contractivity, and $\|x_i - x_j\| = \sum_B w_B |z_i(B) - z_j(B)|$.
  - Zero weights are allowed, so the "all points coincide" case needs no separate treatment.
- **Flat map.** `flat_scalar`, `flat_increment` and `flat_ineq` prove Lemma 2 with the constant 108. They use the same scalar bound $6|a||\delta| + 3\delta^2$ and Cauchy–Schwarz.
- **Martingale step.** The source's Corollary 3 and the stopped-walk martingale are replaced by a deterministic argument.
  - `res_left` and `res_right` establish the resolvent identities $G = pI + qAG = pI + qGA$ for $G = \sum_s p q^s A^s$.
  - `h_props` establishes the harmonic identity (2) for $h = Gz$.
  - `local_ineq` is the one-step conditional-expectation version of Lemma 2: the linear term cancels by the balance relation.
  - `global_sum` sums these local inequalities with weights $\pi_k G_{ki}$. The right resolvent identity then telescopes the potentials, and the initial nonnegative potential is discarded.

  This yields $B + tE_A \le 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2$ (`assembly_real`, `assembly`). This is exactly the bound the source obtains from (stopped-bound) and (jump-identity). It is a legitimate deterministic rendering of the expectation computation.
- **Comparison.** `markov_subadd`, `comp_block`, `comp_Dt`, `comp_geom_sq`, `comp_geom_mod`, `comp_real` and `res_comparison` prove Lemma 4 with constant 28, phrased directly in terms of $D(s) = \sum \pi_i (A^s)_{ij} \|x_i - x_j\|^2$.

  - Subadditivity of $\sqrt{D}$ uses stationarity of π under A^r, derived from reversibility.
  - The bound $\mathbb{E}J^2 \le 3$ and the bound $\mathbb{P}(R = r) \le 2/t$ are proved via explicit series.
- **Assembly.** The final `linarith` combines the two bounds: $108 \cdot 28 = 3024$.

No lemma is weaker than the step it represents, and no step is assumed rather than proved.

Concerns. None

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

/-- The real Banach space `ℓ₁` of absolutely summable real sequences (indexed by `ℕ`). -/
abbrev L1Space : Type := lp (fun _ : ℕ => ℝ) 1

/-- `π` is a probability vector on `[n]`: nonnegative entries summing to one. -/
def IsProbVec {n : ℕ} (π : Fin n → ℝ) : Prop :=
  (∀ i, 0 ≤ π i) ∧ ∑ i, π i = 1

/-- `A` is a stochastic matrix: nonnegative entries and all row sums equal to one. -/
def IsStochasticMatrix {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  (∀ i j, 0 ≤ A i j) ∧ ∀ i, ∑ j, A i j = 1

/-- `A` is reversible with respect to `π`: `π i * a_ij = π j * a_ji` for all `i, j`. -/
def IsReversibleWrt {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  ∀ i j, π i * A i j = π j * A j i

theorem setting_consistent :
    IsProbVec (fun _ : Fin 1 => (1:ℝ)) ∧ IsStochasticMatrix (1 : Matrix (Fin 1) (Fin 1) ℝ) ∧
      IsReversibleWrt (fun _ : Fin 1 => (1:ℝ)) (1 : Matrix (Fin 1) (Fin 1) ℝ) := by
  refine ⟨⟨fun _ => zero_le_one, by simp⟩, ⟨fun i j => ?_, fun i => ?_⟩, fun i j => ?_⟩
  · rw [Matrix.one_apply]
    split_ifs <;> norm_num
  · rw [Fin.sum_univ_one, Subsingleton.elim i 0, Matrix.one_apply_eq]
  · have h : i = j := Subsingleton.elim i j
    subst h
    rfl

theorem setting_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (j : Fin n) :
    ∑ i, π i * A i j = π j := by
  calc ∑ i, π i * A i j = ∑ i, π j * A j i := Finset.sum_congr rfl (fun i _ => hrev i j)
    _ = π j * ∑ i, A j i := by rw [Finset.mul_sum]
    _ = π j := by rw [hA.2 j, mul_one]

theorem setting_stationary_pow {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (s : ℕ) (j : Fin n) :
    ∑ i, π i * (A ^ s) i j = π j := by
  induction s generalizing j with
  | zero => simp [Matrix.one_apply]
  | succ k ih =>
    rw [pow_succ]
```

```
    simp only [Matrix.mul_apply, Finset.mul_sum]
    rw [Finset.sum_comm]
    calc ∑ l, ∑ i, π i * ((A ^ k) i l * A l j)
         = ∑ l, (∑ i, π i * (A ^ k) i l) * A l j := by
           refine Finset.sum_congr rfl (fun l _ => ?_)
           rw [Finset.sum_mul]
           refine Finset.sum_congr rfl (fun i _ => ?_)
           ring
      _ = ∑ l, π l * A l j := by simp only [ih]
      _ = ∑ l, π j * A j l := Finset.sum_congr rfl (fun l _ => hrev l j)
      _ = π j * ∑ l, A j l := by rw [Finset.mul_sum]
      _ = π j := by rw [hA.2 j, mul_one]



theorem stoch_pow {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (s : ℕ) :
    IsStochasticMatrix (A ^ s) := by
  induction s with
  | zero =>
    refine ⟨?_, ?_⟩
    · intro i j
      by_cases h : i = j
      · subst h; simp
      · simp [h]
    · intro i
      simp [Matrix.one_apply]
  | succ s ih =>
    rw [pow_succ]
    rcases hA with ⟨hA_nonneg, hA_row⟩
    rcases ih with ⟨hB_nonneg, hB_row⟩
    refine ⟨?_, ?_⟩
    · intro i j
      simp only [Matrix.mul_apply]
      refine Finset.sum_nonneg (fun k _ => ?_)
      have hk1 : 0 ≤ (A ^ s) i k := hB_nonneg i k
      have hk2 : 0 ≤ A k j := hA_nonneg k j
      nlinarith
    · intro i
      simp only [Matrix.mul_apply]
      calc
       ∑ j, ∑ k : Fin n, ((A ^ s) i k) * A k j = ∑ k : Fin n, ((A ^ s) i k) * (∑ j : Fin n, A k j) := by
          rw [Finset.sum_comm]
          refine Finset.sum_congr rfl (fun k _ => ?_)
          rw [Finset.mul_sum]
        _ = ∑ k : Fin n, ((A ^ s) i k) * 1 := by
          refine Finset.sum_congr rfl (fun k _ => ?_)
          rw [hA_row k]
        _ = ∑ k : Fin n, (A ^ s) i k := by simp
        _ = 1 := hB_row i

theorem weighted_minkowski {ι : Type*} [Fintype ι] (c f g F : ι → ℝ)
    (hc : ∀ i, 0 ≤ c i) (hf : ∀ i, 0 ≤ f i) (hg : ∀ i, 0 ≤ g i) (hF0 : ∀ i, 0 ≤ F i)
    (hF : ∀ i, F i ≤ f i + g i) :
   Real.sqrt (∑ i, c i * F i ^ 2) ≤ Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2) := by
  have h_sum_cF_sq : 0 ≤ ∑ i, c i * F i ^ 2 :=
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_sum_cf_sq : 0 ≤ ∑ i, c i * f i ^ 2 :=
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_sum_cg_sq : 0 ≤ ∑ i, c i * g i ^ 2 :=
```

```
    Finset.sum_nonneg fun i _ => mul_nonneg (hc i) (sq_nonneg _)
  have h_nonneg_prod : 0 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) :=
    mul_nonneg h_sum_cf_sq h_sum_cg_sq


  have h_cauchy : (∑ i, c i * f i * g i) ^ 2 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) := by
    set u := fun i => Real.sqrt (c i) * f i with hu
    set v := fun i => Real.sqrt (c i) * g i with hv
    have h_cs := Finset.sum_mul_sq_le_sq_mul_sq (Finset.univ : Finset ι) u v

    have hu_sq : ∀ i, u i ^ 2 = c i * f i ^ 2 := by
      intro i
      dsimp [u]
      calc
        (Real.sqrt (c i) * f i) ^ 2 = (Real.sqrt (c i)) ^ 2 * (f i) ^ 2 := by ring
        _ = c i * (f i) ^ 2 := by rw [Real.sq_sqrt (hc i)]
    have hv_sq : ∀ i, v i ^ 2 = c i * g i ^ 2 := by
      intro i
      dsimp [v]
      calc
        (Real.sqrt (c i) * g i) ^ 2 = (Real.sqrt (c i)) ^ 2 * (g i) ^ 2 := by ring
        _ = c i * (g i) ^ 2 := by rw [Real.sq_sqrt (hc i)]
    have huv : ∀ i, u i * v i = c i * f i * g i := by
      intro i
      dsimp [u, v]
      calc
        (Real.sqrt (c i) * f i) * (Real.sqrt (c i) * g i) = (Real.sqrt (c i) * Real.sqrt (c i)) * (f i * g i)
↪  := by ring
        _ = c i * (f i * g i) := by rw [Real.mul_self_sqrt (hc i)]
        _ = c i * f i * g i := by ring
    have hsum_u_sq : (∑ i : ι, u i ^ 2) = ∑ i, c i * f i ^ 2 :=
      Finset.sum_congr rfl (fun i _ => by rw [hu_sq i])
    have hsum_v_sq : (∑ i : ι, v i ^ 2) = ∑ i, c i * g i ^ 2 :=
      Finset.sum_congr rfl (fun i _ => by rw [hv_sq i])
    have hsum_uv : (∑ i : ι, u i * v i) = ∑ i, c i * f i * g i :=
      Finset.sum_congr rfl (fun i _ => by rw [huv i])
    simpa [hsum_uv, hsum_u_sq, hsum_v_sq] using h_cs
  have h_bound : ∑ i, c i * f i * g i ≤ Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2)) := by
    have h_sq : (∑ i, c i * f i * g i) ^ 2 ≤ (Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2))) ^
↪  2 := by
      calc
        (∑ i, c i * f i * g i) ^ 2 ≤ (∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2) := h_cauchy
        _ = (Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2))) ^ 2 := by
          rw [Real.sq_sqrt h_nonneg_prod]
    have h_nonneg_sqrt : 0 ≤ Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i * g i ^ 2)) :=
      Real.sqrt_nonneg _
    have h_nonneg_sum : 0 ≤ ∑ i, c i * f i * g i :=
      Finset.sum_nonneg fun i _ => mul_nonneg (mul_nonneg (hc i) (hf i)) (hg i)
    nlinarith
  have h_goal_sq : (∑ i, c i * F i ^ 2) ≤ (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^
↪  2)) ^ 2 := by
    calc
      (∑ i, c i * F i ^ 2) ≤ (∑ i : ι, c i * ((f i + g i) ^ 2)) := by
        refine Finset.sum_le_sum fun i _ => ?_
        have hF_i : F i ^ 2 ≤ (f i + g i) ^ 2 := by
          have hF_nonneg : 0 ≤ F i := hF0 i
          have h_fg_nonneg : 0 ≤ f i + g i := add_nonneg (hf i) (hg i)
          have h_le : F i ≤ f i + g i := hF i
          nlinarith
```

```
        exact mul_le_mul_of_nonneg_left hF_i (hc i)
      _ = ∑ i : ι, c i * (f i ^ 2 + 2 * f i * g i + g i ^ 2) := by
        refine Finset.sum_congr rfl (fun i _ => ?_)
        ring
      _ = (∑ i, c i * f i ^ 2) + (∑ i, c i * g i ^ 2) + 2 * (∑ i, c i * f i * g i) := by
        simp only [Finset.sum_add_distrib, mul_add, Finset.mul_sum]
        ring
      _ ≤ (∑ i, c i * f i ^ 2) + (∑ i, c i * g i ^ 2) + 2 * Real.sqrt ((∑ i, c i * f i ^ 2) * (∑ i, c i
↪  * g i ^ 2)) := by
        nlinarith
      _ = (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2)) ^ 2 := by
        rw [add_sq]
        rw [Real.sq_sqrt h_sum_cf_sq, Real.sq_sqrt h_sum_cg_sq]
        rw [Real.sqrt_mul h_sum_cf_sq (∑ i, c i * g i ^ 2)]
        ring
  have h_sq_sqrt_cF : (Real.sqrt (∑ i, c i * F i ^ 2)) ^ 2 = ∑ i, c i * F i ^ 2 :=
    Real.sq_sqrt h_sum_cF_sq
  have h_nonneg_sqrt_sum : 0 ≤ Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt (∑ i, c i * g i ^ 2) := by
    positivity
  have h_sq_goal : (Real.sqrt (∑ i, c i * F i ^ 2)) ^ 2 ≤ (Real.sqrt (∑ i, c i * f i ^ 2) + Real.sqrt
↪  (∑ i, c i * g i ^ 2)) ^ 2 := by
    rw [h_sq_sqrt_cF]
    exact h_goal_sq
  nlinarith

theorem markov_subadd_expand {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (f : Fin n → Fin n → ℝ) (r s : ℕ) :
    ∑ i, ∑ j, π i * (A ^ (r + s)) i j * f i j
      = ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.2 := by
  rw [pow_add]









  have hLHS : (∑ i, ∑ j, π i * ((A ^ r * A ^ s) i j) * f i j) =
            (∑ i, ∑ j, ∑ k, π i * ((A ^ r) i k * (A ^ s) k j) * f i j) := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    simp [Matrix.mul_apply, Finset.mul_sum, Finset.sum_mul, mul_assoc]
  rw [hLHS]


  rw [← Finset.univ_product_univ (α := Fin n), Finset.sum_product]

  refine Finset.sum_congr rfl (fun i' hi' => ?_)
  rw [← Finset.univ_product_univ (α := Fin n), Finset.sum_product]
```

```
  simp
  rw [Finset.sum_comm]
  refine Finset.sum_congr rfl (fun x _ => ?_)
  refine Finset.sum_congr rfl (fun y _ => ?_)
  ring

theorem markov_subadd_left {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (f : Fin n → Fin n → ℝ) (r s : ℕ) :
    ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.1
      = ∑ i, ∑ k, π i * (A ^ r) i k * f i k := by

  have h_pow_rowsum (k : Fin n) : ∑ j : Fin n, (A ^ s) k j = 1 := by
    induction s with
    | zero => simp [Matrix.one_apply]
    | succ s ih =>
      rw [pow_succ]
      calc
        ∑ j : Fin n, ((A ^ s) * A) k j = ∑ j : Fin n, (∑ l : Fin n, (A ^ s) k l * A l j) := rfl
        _ = (∑ l : Fin n, (A ^ s) k l * (∑ j : Fin n, A l j)) := by
          rw [Finset.sum_comm]; simp [Finset.mul_sum]
        _ = ∑ l : Fin n, (A ^ s) k l * 1 := by simp [hA.2]
        _ = ∑ l : Fin n, (A ^ s) k l := by simp
        _ = 1 := ih
  have h_triple_sum {α β γ δ : Type _} [AddCommMonoid δ] [Fintype α] [Fintype β] [Fintype γ] [DecidableEq
↪  α] [DecidableEq β] [DecidableEq γ] (g : α → β → γ → δ) :
    (∑ x : α × β × γ, g x.1 x.2.1 x.2.2) = ∑ a : α, ∑ b : β, ∑ c : γ, g a b c := by
    calc
      (∑ x : α × β × γ, g x.1 x.2.1 x.2.2) = (∑ x : α × (β × γ), g x.1 x.2.1 x.2.2) := rfl
      _ = ∑ a : α, ∑ bc : β × γ, g a bc.1 bc.2 := by
       simpa using Finset.sum_product (s := (Finset.univ : Finset α)) (t := (Finset.univ : Finset (β × γ)))
          (f := fun (x : α × (β × γ)) => g x.1 x.2.1 x.2.2)
      _ = ∑ a : α, ∑ b : β, ∑ c : γ, g a b c := by
        refine Finset.sum_congr rfl (fun a _ => ?_)
        simpa using Finset.sum_product (s := (Finset.univ : Finset β)) (t := (Finset.univ : Finset γ))
          (f := fun (bc : β × γ) => g a bc.1 bc.2)
  calc
    ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.1 p.2.1
    = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f i k := by
      simpa using h_triple_sum (fun i k j => (π i * (A ^ r) i k * (A ^ s) k j) * f i k)
    _ = ∑ i : Fin n, ∑ k : Fin n, (π i * (A ^ r) i k * (∑ j : Fin n, (A ^ s) k j)) * f i k := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      calc
```

```
        ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f i k
          = (∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j)) * f i k := by
        rw [Finset.sum_mul]
      _ = (π i * (A ^ r) i k * (∑ j : Fin n, (A ^ s) k j)) * f i k := by
        simp [Finset.mul_sum, mul_assoc]
  _ = ∑ i : Fin n, ∑ k : Fin n, (π i * (A ^ r) i k * 1) * f i k := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    refine Finset.sum_congr rfl (fun k _ => ?_)
    rw [h_pow_rowsum k]
  _ = ∑ i : Fin n, ∑ k : Fin n, π i * (A ^ r) i k * f i k := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    refine Finset.sum_congr rfl (fun k _ => ?_)
    ring

theorem markov_subadd_right {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
  (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A) (f : Fin n → Fin n → ℝ) (r s : ℕ) :
  ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2
    = ∑ k, ∑ j, π k * (A ^ s) k j * f k j := by
  classical

  have h_reorder : ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f
↪  p.2.1 p.2.2
              = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f k j := by
    calc
      ∑ p : Fin n × Fin n × Fin n, (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2
        = ∑ p ∈ (Finset.univ : Finset (Fin n × Fin n × Fin n)),
          (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2 := by simp
      _ = ∑ p ∈ (Finset.univ : Finset (Fin n)) ×[U+02E2] ((Finset.univ : Finset (Fin n)) ×[U+02E2]
↪  (Finset.univ : Finset (Fin n))),
          (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * f p.2.1 p.2.2 := by simp
      _ = ∑ x ∈ (Finset.univ : Finset (Fin n)),
            ∑ yz ∈ ((Finset.univ : Finset (Fin n)) ×[U+02E2] (Finset.univ : Finset (Fin n))),
            (π x * (A ^ r) x yz.1 * (A ^ s) yz.1 yz.2) * f yz.1 yz.2 := by
        rw [Finset.sum_product]
      _ = ∑ x ∈ (Finset.univ : Finset (Fin n)),
            ∑ y ∈ (Finset.univ : Finset (Fin n)), ∑ z ∈ (Finset.univ : Finset (Fin n)),
            (π x * (A ^ r) x y * (A ^ s) y z) * f y z := by
        refine Finset.sum_congr rfl (fun x _ => ?_)
        rw [Finset.sum_product]
      _ = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n,
            (π i * (A ^ r) i k * (A ^ s) k j) * f k j := by simp
  rw [h_reorder]
  calc
    ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k * (A ^ s) k j) * f k j
      = ∑ i : Fin n, ∑ k : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k) * ((A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun i _ => ?_)
      refine Finset.sum_congr rfl (fun k _ => ?_)
      refine Finset.sum_congr rfl (fun j _ => ?_)
      ring
    _ = ∑ k : Fin n, ∑ i : Fin n, ∑ j : Fin n, (π i * (A ^ r) i k) * ((A ^ s) k j * f k j) := by
      rw [Finset.sum_comm]
    _ = ∑ k : Fin n, (∑ i : Fin n, π i * (A ^ r) i k) * (∑ j : Fin n, (A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun k _ => ?_)
      rw [Finset.sum_mul]
      refine Finset.sum_congr rfl (fun i _ => ?_)
      simp [Finset.mul_sum, mul_assoc]
    _ = ∑ k : Fin n, (π k) * (∑ j : Fin n, (A ^ s) k j * f k j) := by
      refine Finset.sum_congr rfl (fun k _ => ?_)
      rw [setting_stationary_pow π A hA hrev r k]
```

```
    _ = ∑ k, ∑ j, π k * (A ^ s) k j * f k j := by
      simp [Finset.mul_sum, mul_assoc]

theorem comp_geom_sq_sqsum (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) :
    HasSum (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) (q * (1 + q) / (1 - q) ^ 3) := by
  have hq_norm : ‖q‖ < 1 := by
    rw [Real.norm_eq_abs, abs_of_nonneg hq0]; exact hq1
  have hsum2 : Summable (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) :=
    summable_pow_mul_geometric_of_norm_lt_one 2 hq_norm
  have hshift : HasSum (fun s : ℕ => ((s + 1 : ℕ) : ℝ) ^ 2 * q ^ (s + 1))
      (q * ((∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) + 2 * (q / (1 - q) ^ 2) + (1 - q)⁻¹)) := by
    have h := ((hsum2.hasSum.add ((hasSum_coe_mul_geometric_of_norm_lt_one hq_norm).mul_left 2)).add
      (hasSum_geometric_of_lt_one hq0 hq1)).mul_left q
    convert h using 1
    all_goals (try (funext s; push_cast; ring))
  have hT : (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s)
      = q * ((∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) + 2 * (q / (1 - q) ^ 2) + (1 - q)⁻¹) := by
    rw [← hshift.tsum_eq]
    rw [hsum2.tsum_eq_zero_add]
    simp
  have hpos : 0 < 1 - q := by linarith
  have hr : (1 - q)⁻¹ * (1 - q) = 1 := inv_mul_cancel₀ hpos.ne'
  have hval : (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) = q * (1 + q) / (1 - q) ^ 3 := by
    rw [div_eq_mul_inv, ← inv_pow]
    rw [div_eq_mul_inv, ← inv_pow] at hT
    generalize (1 - q)⁻¹ = r at hT hr ⊢
    generalize (∑' s : ℕ, (s : ℝ) ^ 2 * q ^ s) = T at hT ⊢
    linear_combination r * hT - (T + q * r ^ 2) * hr
  rw [← hval]
  exact hsum2.hasSum

theorem markov_subadd {n : ℕ} (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (d : Fin n → Fin n → ℝ) (hd0 : ∀ i j, 0 ≤ d i j) (htri : ∀ i j k, d i k ≤ d i j + d j k)
    (D : ℕ → ℝ) (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * d i j ^ 2) (r s : ℕ) :
    Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := by
  have hc : ∀ p : Fin n × Fin n × Fin n, 0 ≤ π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 :=
    fun p => mul_nonneg (mul_nonneg (hπ _) ((stoch_pow A hA r).1 _ _)) ((stoch_pow A hA s).1 _ _)
  have e1 : D (r + s) = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.1 p.2.2 ^ 2 := by
    rw [hD]
    exact markov_subadd_expand π A (fun i j => d i j ^ 2) r s
  have e2 : D r = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.1 p.2.1 ^ 2 := by
    rw [hD]
    exact (markov_subadd_left π A hA (fun i j => d i j ^ 2) r s).symm
  have e3 : D s = ∑ p : Fin n × Fin n × Fin n,
      (π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2) * d p.2.1 p.2.2 ^ 2 := by
    rw [hD]
    exact (markov_subadd_right π A hA hrev (fun i j => d i j ^ 2) r s).symm
  rw [e1, e2, e3]
  exact weighted_minkowski
    (fun p : Fin n × Fin n × Fin n => π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2)
    (fun p => d p.1 p.2.1) (fun p => d p.2.1 p.2.2) (fun p => d p.1 p.2.2)
    hc (fun p => hd0 _ _) (fun p => hd0 _ _) (fun p => hd0 _ _) (fun p => htri _ _ _)

theorem comp_geom_mod_pow_le (t : ℕ) (ht : 0 < t) : ((t : ℝ) / ((t : ℝ) + 1)) ^ t ≤ 1 / 2 := by
  have hpos : 0 < (t : ℝ) := by exact_mod_cast ht
  have h_denom_pos : 0 < (t : ℝ) + 1 := by linarith
```

```
  rcases Nat.eq_or_lt_of_le (Nat.one_le_of_lt ht) with (rfl | ht2)
  ·
    norm_num
  ·
    have h_div_nonneg : 0 ≤ 1 / (t : ℝ) := div_nonneg (by norm_num) (by exact_mod_cast ht.le)


    have h_bern : (2 : ℝ) ≤ ((1 : ℝ) + 1 / (t : ℝ)) ^ t := by
      have h_ge_neg2 : (-2 : ℝ) ≤ 1 / (t : ℝ) := by linarith
      have h_raw := one_add_mul_le_pow (a := 1 / (t : ℝ)) h_ge_neg2 t

      have h_mul : (t : ℝ) * (1 / (t : ℝ)) = 1 := by
        field_simp [ne_of_gt hpos]
      have h_two : (1 : ℝ) + (t : ℝ) * (1 / (t : ℝ)) = (2 : ℝ) := by
        rw [h_mul]
        norm_num
      rw [h_two] at h_raw
      exact h_raw
    have h_base_pos : 0 < (1 : ℝ) + 1 / (t : ℝ) := by linarith


    have h_eq : (t : ℝ) / ((t : ℝ) + 1) = ((1 : ℝ) + 1 / (t : ℝ))⁻¹ := by
      calc
        (t : ℝ) / ((t : ℝ) + 1) = ((t : ℝ) / (t : ℝ)) / (((t : ℝ) + 1) / (t : ℝ)) := by
          field_simp [ne_of_gt hpos]
        _ = 1 / (((t : ℝ) + 1) / (t : ℝ)) := by field_simp [ne_of_gt hpos]
        _ = 1 / ((t : ℝ) / (t : ℝ) + 1 / (t : ℝ)) := by ring
        _ = 1 / (1 + 1 / (t : ℝ)) := by field_simp [ne_of_gt hpos]
        _ = ((1 : ℝ) + 1 / (t : ℝ))⁻¹ := by simp


    have h_pow_inv : (((1 : ℝ) + 1 / (t : ℝ))⁻¹) ^ t = (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ := by
      rw [inv_pow]

    have h_inv_pos : 0 < ((1 : ℝ) + 1 / (t : ℝ)) ^ t := pow_pos h_base_pos _
    have h_inv : (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ ≤ (2 : ℝ)⁻¹ := by
      rw [inv_le_inv₀ h_inv_pos (by norm_num : 0 < (2 : ℝ))]
      exact h_bern
    calc
      ((t : ℝ) / ((t : ℝ) + 1)) ^ t = (((1 : ℝ) + 1 / (t : ℝ))⁻¹) ^ t := by rw [h_eq]
      _ = (((1 : ℝ) + 1 / (t : ℝ)) ^ t)⁻¹ := by rw [h_pow_inv]
      _ ≤ (2 : ℝ)⁻¹ := h_inv
      _ = 1 / 2 := by norm_num

theorem comp_geom_mod_hasSum (t : ℕ) (ht : 0 < t) (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (f : ℕ → ℝ) :
    HasSum (fun s : ℕ => q ^ s * f (s % t)) ((∑ r ∈ Finset.range t, q ^ r * f r) / (1 - q ^ t)) := by
  have hqt : q ^ t < 1 := pow_lt_one₀ hq0 hq1 (by omega)
  have hsum_abs : Summable (fun s : ℕ => |q ^ s * f (s % t)|) := by
    have hM : ∀ r ∈ Finset.range t, |f r| ≤ ∑ r ∈ Finset.range t, |f r| := fun r hr =>
      Finset.single_le_sum (f := fun r => |f r|) (fun i _ => abs_nonneg _) hr
    have h_bound : ∀ s : ℕ, |q ^ s * f (s % t)| ≤ (∑ r ∈ Finset.range t, |f r|) * q ^ s := by
      intro s
      rw [abs_mul, abs_of_nonneg (pow_nonneg hq0 s), mul_comm (q ^ s)]
      exact mul_le_mul_of_nonneg_right (hM _ (Finset.mem_range.mpr (Nat.mod_lt s ht))) (pow_nonneg hq0 s)
    exact Summable.of_nonneg_of_le (fun s => abs_nonneg _) h_bound
      ((summable_geometric_of_lt_one hq0 hq1).mul_left _)
  have hsum : Summable (fun s : ℕ => q ^ s * f (s % t)) := Summable.of_abs hsum_abs
  set g := fun s : ℕ => q ^ s * f (s % t) with hg
  have hT := hsum.hasSum
```

```
  have h1 := (hasSum_nat_add_iff' t).mpr hT
  have h2 : HasSum (fun n => g (n + t)) (q ^ t * ∑' s, g s) := by
    have : (fun n => g (n + t)) = fun n => q ^ t * g n := by
      funext n
      simp only [hg]
      rw [Nat.add_mod_right, pow_add]
      ring
    rw [this]
    exact hT.mul_left (q ^ t)
  have h3 := h1.unique h2
  have hP : ∑ i ∈ Finset.range t, g i = ∑ r ∈ Finset.range t, q ^ r * f r := by
    refine Finset.sum_congr rfl (fun r hr => ?_)
    simp only [hg]
    rw [Nat.mod_eq_of_lt (Finset.mem_range.mp hr)]
  have hne : (1 - q ^ t) ≠ 0 := by linarith
  have hval : ∑' s, g s = (∑ r ∈ Finset.range t, q ^ r * f r) / (1 - q ^ t) := by
    rw [eq_div_iff hne]
    linear_combination h3 + hP
  rw [← hval]
  exact hT

theorem comp_block (t : ℕ) (D : ℕ → ℝ) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∀ j r : ℕ, D (j * t + r) ≤ 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by
  intro j r
  have h_base : ∀ j : ℕ, Real.sqrt (D (j * t + r)) ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    intro j'
    induction j' with
    | zero => simp
    | succ k ih =>
      have h_eq : (k.succ : ℕ) * t + r = (k * t + r) + t := by
        rw [Nat.succ_mul]
        omega
      rw [h_eq]
      calc
        Real.sqrt (D ((k * t + r) + t))
            ≤ Real.sqrt (D (k * t + r)) + Real.sqrt (D t) := hsub (k * t + r) t
        _ ≤ ((k : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) + Real.sqrt (D t) := by
          gcongr
        _ = ((k : ℝ) + 1) * Real.sqrt (D t) + Real.sqrt (D r) := by ring
        _ = (k.succ : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by simp
  have h_sqrt := h_base j
  have hR_nonneg : 0 ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    have h1 : 0 ≤ Real.sqrt (D t) := Real.sqrt_nonneg _
    have h2 : 0 ≤ Real.sqrt (D r) := Real.sqrt_nonneg _
    nlinarith
  have h_sq : D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
    calc
      D (j * t + r) = (Real.sqrt (D (j * t + r))) ^ 2 := by
        rw [Real.sq_sqrt (hDnn (j * t + r))]
      _ ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
        gcongr
  have h_sq_sqrt_t : (Real.sqrt (D t)) ^ 2 = D t := by
    rw [Real.sq_sqrt (hDnn t)]
  have h_sq_sqrt_r : (Real.sqrt (D r)) ^ 2 = D r := by
    rw [Real.sq_sqrt (hDnn r)]
  have h_expand : ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 ≤
      2 * ((j : ℝ) ^ 2) * D t + 2 * D r := by
    calc
```

```
      ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2
          = ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 +
            2 * ((j : ℝ) * Real.sqrt (D t)) * Real.sqrt (D r) := by ring
      _ ≤ ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 +
            (((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2) := by
        gcongr
        nlinarith [sq_nonneg ((j : ℝ) * Real.sqrt (D t) - Real.sqrt (D r))]
      _ = 2 * (((j : ℝ) * Real.sqrt (D t)) ^ 2) + 2 * (Real.sqrt (D r)) ^ 2 := by ring
      _ = 2 * ((j : ℝ) ^ 2) * (Real.sqrt (D t)) ^ 2 + 2 * (Real.sqrt (D r)) ^ 2 := by ring
      _ = 2 * ((j : ℝ) ^ 2) * D t + 2 * D r := by rw [h_sq_sqrt_t, h_sq_sqrt_r]
  nlinarith

theorem comp_Dt (t : ℕ) (ht : 0 < t) (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have htne : (t : ℝ) ≠ 0 := ne_of_gt htpos
  have key : ∀ a b : ℕ, D (a + b) ≤ 2 * D a + 2 * D b := by
    intro a b
    have h1 := hsub a b
    have ha := Real.sq_sqrt (hDnn a)
    have hb := Real.sq_sqrt (hDnn b)
    have hab := Real.sq_sqrt (hDnn (a + b))
    have s1 := Real.sqrt_nonneg (D (a + b))
    have s2 := Real.sqrt_nonneg (D a)
    have s3 := Real.sqrt_nonneg (D b)
    have hm := mul_le_mul h1 h1 s1 (add_nonneg s2 s3)
    nlinarith [sq_nonneg (Real.sqrt (D a) - Real.sqrt (D b))]
  have hIcc : Finset.Icc 1 t = (Finset.range t).image (fun r => r + 1) := by
    ext s
    simp only [Finset.mem_Icc, Finset.mem_image, Finset.mem_range]
    constructor
    · intro h
      exact ⟨s - 1, by omega, by omega⟩
    · rintro ⟨r, hr, rfl⟩
      omega
  have hconv : ∑ s ∈ Finset.Icc 1 t, D s = ∑ r ∈ Finset.range t, D (r + 1) := by
    rw [hIcc, Finset.sum_image]
    intro a _ b _ h
    simpa using h
  have hstep : ∀ r ∈ Finset.range t, D t ≤ 2 * D (r + 1) + 2 * D (t - 1 - r) := by
    intro r hr
    have hr' := Finset.mem_range.mp hr
    have he : (r + 1) + (t - 1 - r) = t := by omega
    have h := key (r + 1) (t - 1 - r)
    rwa [he] at h
  have hsum := Finset.sum_le_sum hstep
  rw [Finset.sum_const, Finset.card_range, nsmul_eq_mul, Finset.sum_add_distrib,
    ← Finset.mul_sum, ← Finset.mul_sum, Finset.sum_range_reflect] at hsum
  have h2 := Finset.sum_range_succ' D t
  have h3 := Finset.sum_range_succ D t
  have hDt := hDnn t
  rw [hconv]
  have hfin : (t : ℝ) * D t ≤ 4 * ∑ r ∈ Finset.range t, D (r + 1) := by
    linarith
  calc D t = (1 / (t : ℝ)) * ((t : ℝ) * D t) := by
        rw [← mul_assoc, one_div_mul_cancel htne, one_mul]
    _ ≤ (1 / (t : ℝ)) * (4 * ∑ r ∈ Finset.range t, D (r + 1)) := by
        apply mul_le_mul_of_nonneg_left hfin
```

```
        positivity
    _ = 4 * ((1 / (t : ℝ)) * ∑ r ∈ Finset.range t, D (r + 1)) := by ring

theorem comp_geom_sq (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1)) :
    Summable (fun s : ℕ => q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ 3 := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1r : (1 : ℝ) ≤ t := by exact_mod_cast ht
  have htne : (t : ℝ) ≠ 0 := ne_of_gt htpos
  have ht1ne : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have h1q : 1 - q = 1 / ((t : ℝ) + 1) := by
    rw [hq]
    field_simp <;> ring
  have hpos : 0 < 1 - q := by rw [h1q]; positivity
  have hq1 : q < 1 := by linarith
  have hpt : ∀ s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ (1 / (t : ℝ) ^ 2) * ((s : ℝ) ^ 2 * q ^ s) := by
    intro s
    have h1 : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) / t := Nat.cast_div_le
    have h0 : (0 : ℝ) ≤ ((s / t : ℕ) : ℝ) := Nat.cast_nonneg _
    have h2 : (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / t) ^ 2 := by nlinarith
    have hqs : 0 ≤ q ^ s := pow_nonneg hq0 s
    calc q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ q ^ s * ((s : ℝ) / t) ^ 2 :=
          mul_le_mul_of_nonneg_left h2 hqs
      _ = (1 / (t : ℝ) ^ 2) * ((s : ℝ) ^ 2 * q ^ s) := by ring
  have hS := comp_geom_sq_sqsum q hq0 hq1
  have hsum : Summable (fun s : ℕ => q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) :=
    Summable.of_nonneg_of_le (fun s => mul_nonneg (pow_nonneg hq0 s) (sq_nonneg _)) hpt
      (hS.mul_left (1 / (t : ℝ) ^ 2)).summable
  refine ⟨hsum, ?_⟩
  have hle := hasSum_le hpt hsum.hasSum (hS.mul_left (1 / (t : ℝ) ^ 2))
  have hval : (1 - q) * (1 / (t : ℝ) ^ 2 * (q * (1 + q) / (1 - q) ^ 3)) = (2 * (t : ℝ) + 1) / t := by
    rw [h1q, hq]
    field_simp <;> ring
  calc (1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      ≤ (1 - q) * (1 / (t : ℝ) ^ 2 * (q * (1 + q) / (1 - q) ^ 3)) :=
        mul_le_mul_of_nonneg_left hle hpos.le
    _ = (2 * (t : ℝ) + 1) / t := hval
    _ ≤ 3 := by rw [div_le_iff₀ htpos]; linarith

theorem comp_geom_mod (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (f : ℕ → ℝ) (hf : ∀ r, 0 ≤ f r) :
    Summable (fun s : ℕ => q ^ s * f (s % t)) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * f (s % t) ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1 : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hqt : q * ((t : ℝ) + 1) = t := by rw [hq]; field_simp
  have hq1 : q < 1 := by nlinarith [hqt, htpos]
  have hqt' : (1 - q) * (t : ℝ) = q := by linear_combination (-1 : ℝ) * hqt
  have hpt : q ^ t ≤ 1 / 2 := by rw [hq]; exact comp_geom_mod_pow_le t ht
  have hS := comp_geom_mod_hasSum t ht q hq0 hq1 f
  refine ⟨hS.summable, ?_⟩
  rw [hS.tsum_eq]
  have hP : ∑ r ∈ Finset.range t, q ^ r * f r ≤ ∑ r ∈ Finset.range t, f r :=
    Finset.sum_le_sum (fun r _ => mul_le_of_le_one_left (hf r) (pow_le_one₀ hq0 hq1.le))
  have hP0 : 0 ≤ ∑ r ∈ Finset.range t, q ^ r * f r :=
    Finset.sum_nonneg (fun r _ => mul_nonneg (pow_nonneg hq0 r) (hf r))
  set P := ∑ r ∈ Finset.range t, q ^ r * f r with hPdef
```

```
  set F := ∑ r ∈ Finset.range t, f r with hFdef
  have hF0 : 0 ≤ F := le_trans hP0 hP
  have hden : 1 / 2 ≤ 1 - q ^ t := by linarith
  have hdpos : 0 < 1 - q ^ t := by linarith
  have hX : P / (1 - q ^ t) ≤ 2 * P := by
    rw [div_le_iff₀ hdpos]
    nlinarith [mul_le_mul_of_nonneg_left hden hP0]
  have h1 : 0 ≤ 1 - q := by linarith
  have hE : (1 - q) * (2 * F) * (t : ℝ) = 2 * F * q := by
    linear_combination (2 * F) * hqt'
  calc (1 - q) * (P / (1 - q ^ t)) ≤ (1 - q) * (2 * F) :=
       mul_le_mul_of_nonneg_left (by linarith) h1
    _ ≤ 2 / (t : ℝ) * F := by
       rw [div_mul_eq_mul_div, le_div_iff₀ htpos]
       rw [hE]
       nlinarith [mul_nonneg hF0 h1]

theorem comp_real_range (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s) (t : ℕ) :
    ∑ r ∈ Finset.range t, D r ≤ ∑ s ∈ Finset.Icc 1 t, D s := by
  have key : ∀ m : ℕ, ∑ r ∈ Finset.range (m + 1), D r = D 0 + ∑ s ∈ Finset.Icc 1 m, D s := by
    intro m
    induction m with
    | zero => simp [Finset.Icc_eq_empty]
    | succ k ih =>
      rw [Finset.sum_range_succ, ih, Finset.sum_Icc_succ_top (show 1 ≤ k + 1 by omega)]
      ring
  have h := key t
  rw [Finset.sum_range_succ, hD0] at h
  linarith [hDnn t]

theorem comp_real (t : ℕ) (ht : 0 < t) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (D : ℕ → ℝ) (hD0 : D 0 = 0) (hDnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    Summable (fun s : ℕ => q ^ s * D s) ∧
      (1 - q) * ∑' s : ℕ, q ^ s * D s ≤ 28 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have hq1 : 0 ≤ 1 - q := by
    rw [hq, sub_nonneg, div_le_one (by positivity)]
    linarith
  obtain ⟨hs1, hb1⟩ := comp_geom_sq t ht q hq
  obtain ⟨hs2, hb2⟩ := comp_geom_mod t ht q hq D hDnn
  have hDt := comp_Dt t ht D hD0 hDnn hsub
  have hblk := comp_block t D hDnn hsub
  have hle : ∀ s : ℕ, q ^ s * D s ≤
      2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)) := by
    intro s
    have hqs : 0 ≤ q ^ s := pow_nonneg hq0 s
    calc q ^ s * D s = q ^ s * D (s / t * t + s % t) := by rw [Nat.div_add_mod']
      _ ≤ q ^ s * (2 * ((s / t : ℕ) : ℝ) ^ 2 * D t + 2 * D (s % t)) :=
          mul_le_mul_of_nonneg_left (hblk _ _) hqs
      _ = 2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)) := by ring
  have hH : HasSum (fun s : ℕ => 2 * D t * (q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (q ^ s * D (s % t)))
      (2 * D t * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2 + 2 * ∑' s : ℕ, q ^ s * D (s % t)) :=
    (hs1.hasSum.mul_left (2 * D t)).add (hs2.hasSum.mul_left 2)
  have hf : Summable (fun s : ℕ => q ^ s * D s) :=
    Summable.of_nonneg_of_le (fun s => mul_nonneg (pow_nonneg hq0 s) (hDnn s)) hle hH.summable
  refine ⟨hf, ?_⟩
  have htsum := hasSum_le hle hf.hasSum hH
```

```
  have hrange := comp_real_range D hD0 hDnn t
  have key : (1 - q) * ∑' s : ℕ, q ^ s * D s ≤
      2 * D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
        + 2 * ((1 - q) * ∑' s : ℕ, q ^ s * D (s % t)) := by
    calc (1 - q) * ∑' s : ℕ, q ^ s * D s
        ≤ (1 - q) * (2 * D t * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2
            + 2 * ∑' s : ℕ, q ^ s * D (s % t)) := mul_le_mul_of_nonneg_left htsum hq1
      _ = 2 * D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
        + 2 * ((1 - q) * ∑' s : ℕ, q ^ s * D (s % t)) := by ring
  have h1 : D t * ((1 - q) * ∑' s : ℕ, q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ≤ D t * 3 :=
    mul_le_mul_of_nonneg_left hb1 (hDnn t)
  have h2 : (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r
      ≤ 2 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
    have hinv : (0 : ℝ) ≤ 1 / (t : ℝ) := by positivity
    have := mul_le_mul_of_nonneg_left hrange hinv
    have he : (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r
        = 2 * ((1 / (t : ℝ)) * ∑ r ∈ Finset.range t, D r) := by ring
    rw [he]
    linarith
  nlinarith [key, h1, hb2, h2, hDt]

theorem res_summable {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (i j : Fin n) :
    Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) := by
  rcases hA with ⟨hA_nonneg, hA_rowsum⟩
  have h_entry_le_one : ∀ i j, A i j ≤ 1 := by
    intro i j
    have hle : A i j ≤ ∑ k : Fin n, A i k :=
      Finset.single_le_sum (by
        intro k hk
        exact hA_nonneg i k) (Finset.mem_univ j)
    rw [hA_rowsum i] at hle
    exact hle
  have h_pow_rowsum : ∀ s i, ∑ j : Fin n, (A ^ s) i j = 1 := by
    intro s i
    induction s with
    | zero =>
      simp [Matrix.one_apply]
    | succ s ih =>
      rw [pow_succ]
      simp_rw [Matrix.mul_apply]
      rw [Finset.sum_comm]
      simp_rw [← Finset.mul_sum]
      simp_rw [hA_rowsum]
      simp [ih]
  have h_pow_nonneg : ∀ s i j, 0 ≤ (A ^ s) i j := by
    intro s
    induction s with
    | zero =>
      intro i j
      simp [Matrix.one_apply]
      split_ifs
      · exact zero_le_one
      · exact le_refl 0
    | succ s ih =>
      intro i j
      rw [pow_succ, Matrix.mul_apply]
      refine Finset.sum_nonneg (fun k _ => mul_nonneg (ih i k) (hA_nonneg k j))
  have h_pow_le_one : ∀ s i j, (A ^ s) i j ≤ 1 := by
```

```
    intro s i j
    have h := Finset.single_le_sum (fun k hk => h_pow_nonneg s i k) (Finset.mem_univ j)
    rw [h_pow_rowsum s i] at h
    exact h

  have hq_nonneg : 0 ≤ q := hq0
  have h_geo_summable : Summable (fun s : ℕ => q ^ s) :=
    summable_geometric_of_lt_one hq_nonneg hq1
  have h_geo_bound : Summable (fun s : ℕ => (1 - q) * q ^ s) := by
    apply Summable.mul_left (1 - q) h_geo_summable
  have h_nonneg_term : ∀ s : ℕ, 0 ≤ (1 - q) * q ^ s * (A ^ s) i j := by
    intro s
    have h1q_nonneg : 0 ≤ 1 - q := by linarith
    have hq_pow_nonneg : 0 ≤ q ^ s := pow_nonneg hq_nonneg _
    have hprod_nonneg : 0 ≤ (1 - q) * q ^ s := mul_nonneg h1q_nonneg hq_pow_nonneg
    exact mul_nonneg hprod_nonneg (h_pow_nonneg s i j)
  refine Summable.of_nonneg_of_le h_nonneg_term (fun s => ?_) h_geo_bound
  have h1q_nonneg : 0 ≤ 1 - q := by linarith
  have hq_pow_nonneg : 0 ≤ q ^ s := pow_nonneg hq_nonneg _
  have hprod_nonneg : 0 ≤ (1 - q) * q ^ s := mul_nonneg h1q_nonneg hq_pow_nonneg
  calc
    (1 - q) * q ^ s * (A ^ s) i j ≤ (1 - q) * q ^ s * 1 :=
      mul_le_mul_of_nonneg_left (h_pow_le_one s i j) hprod_nonneg
    _ = (1 - q) * q ^ s := by ring

theorem res_comparison {n : ℕ} (t : ℕ) (ht : 0 < t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (x : Fin n → L1Space) (q : ℝ) (hq : q = (t : ℝ) / ((t : ℝ) + 1))
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hq1 : q < 1 := by
    rw [hq, div_lt_one (by positivity)]
    linarith
  have hd0 : ∀ i j, 0 ≤ ‖x i - x j‖ := fun i j => norm_nonneg _
  have htri : ∀ i j k, ‖x i - x k‖ ≤ ‖x i - x j‖ + ‖x j - x k‖ := by
    intro i j k
    calc ‖x i - x k‖ = ‖(x i - x j) + (x j - x k)‖ := by rw [sub_add_sub_cancel]
      _ ≤ ‖x i - x j‖ + ‖x j - x k‖ := norm_add_le _ _
  obtain ⟨D, hD⟩ : ∃ D : ℕ → ℝ, ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2 :=
    ⟨_, fun _ => rfl⟩
  have hD0 : D 0 = 0 := by
    rw [hD]
    refine Finset.sum_eq_zero (fun i _ => Finset.sum_eq_zero (fun j _ => ?_))
    rw [pow_zero, Matrix.one_apply]
    split_ifs with h
    · subst h; simp
    · simp
  have hDnn : ∀ s, 0 ≤ D s := by
    intro s
    rw [hD]
    refine Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => ?_))
    have h1 := hπ.1 i
    have h2 := (stoch_pow A hA s).1 i j
    positivity
  have hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
```

```
    fun r s => markov_subadd π hπ.1 A hA hrev (fun i j => ‖x i - x j‖) hd0 htri D hD r s
  obtain ⟨_, hbound⟩ := comp_real t ht q hq D hD0 hDnn hsub
  have hterm : ∀ i j, HasSum (fun s : ℕ => π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      (π i * G i j * ‖x i - x j‖ ^ 2) := by
    intro i j
    have h : HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) (G i j) := by
      rw [hG i j]
      exact (res_summable A hA q hq0 hq1 i j).hasSum
    exact (h.mul_left (π i)).mul_right _
  have hall : HasSum (fun s : ℕ => ∑ i, ∑ j, π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      (∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2) :=
    hasSum_sum (fun i _ => hasSum_sum (fun j _ => hterm i j))
  have hfun : (fun s : ℕ => ∑ i, ∑ j, π i * ((1 - q) * q ^ s * (A ^ s) i j) * ‖x i - x j‖ ^ 2)
      = fun s => (1 - q) * (q ^ s * D s) := by
    funext s
    rw [hD]
    simp only [Finset.mul_sum]
    exact Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => by ring))
  rw [hfun] at hall
  have hL : ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 = (1 - q) * ∑' s, q ^ s * D s := by
    rw [← hall.tsum_eq, tsum_mul_left]
  have hR : (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s
      = ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
    have hthis : ∀ i j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
        = ∑ s ∈ Finset.Icc 1 t, (1 / (t : ℝ)) * (π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
      intro i j
      simp only [Finset.mul_sum, Finset.sum_mul]
      refine Finset.sum_congr rfl (fun s _ => ?_)
      ring
    simp only [hthis]
    simp only [hD, Finset.mul_sum]
    refine Finset.sum_comm.trans ?_
    refine Finset.sum_congr rfl (fun i _ => ?_)
    exact Finset.sum_comm
  first
    | (rw [hL, ← hR]; exact hbound)
    | linarith [hbound, hL, hR]

theorem res_nonneg {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, 0 ≤ G i j := by
  intro i j
  rw [hG i j]
  have h1mq : 0 ≤ 1 - q := by linarith
  refine tsum_nonneg (fun s => ?_)
  have hq_pow : 0 ≤ q ^ s := pow_nonneg hq0 s
  have hA_pow_nonneg : ∀ (k : ℕ), ∀ i j, 0 ≤ (A ^ k) i j := by
    intro k
    induction k with
    | zero =>
      intro i j
      rw [pow_zero, Matrix.one_apply]
      split_ifs <;> norm_num
    | succ k ih =>
      intro i j
      rw [pow_succ, Matrix.mul_apply]
      refine Finset.sum_nonneg (fun l _ => ?_)
      have hAk : 0 ≤ (A ^ k) i l := ih i l
```

```
      have hAkl : 0 ≤ A l j := hA.1 l j
      nlinarith
  have hAs : 0 ≤ (A ^ s) i j := hA_pow_nonneg s i j
  have hprod : 0 ≤ (1 - q) * q ^ s * (A ^ s) i j := by
    apply mul_nonneg
    · exact mul_nonneg h1mq hq_pow
    · exact hAs
  exact hprod

theorem res_rowsum {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i, ∑ j, G i j = 1 := by
  intro i
  have hs : ∀ j, HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) (G i j) := by
    intro j
    rw [hG]
    exact (res_summable A hA q hq0 hq1 i j).hasSum
  have htot : HasSum (fun s : ℕ => ∑ j, (1 - q) * q ^ s * (A ^ s) i j) (∑ j, G i j) := by
    first
    | exact hasSum_sum (fun j _ => hs j)
    | exact HasSum.finsetSum (fun j _ => hs j)
    | exact hasSum_finsetSum (fun j _ => hs j)
  have hgeom : HasSum (fun s : ℕ => ∑ j, (1 - q) * q ^ s * (A ^ s) i j) 1 := by
    have h1 : ∀ s : ℕ, ∑ j, (1 - q) * q ^ s * (A ^ s) i j = (1 - q) * q ^ s := by
      intro s
      rw [← Finset.mul_sum, (stoch_pow A hA s).2 i, mul_one]
    simp only [h1]
    have hg := hasSum_geometric_of_lt_one hq0 hq1
    have h2 := hg.mul_left (1 - q)
    have hne : 1 - q ≠ 0 := by linarith
    rwa [mul_inv_cancel₀ hne] at h2
  exact htot.unique hgeom

theorem res_stationary_hasSum_fin {n : ℕ} (f : Fin n → ℕ → ℝ) (a : Fin n → ℝ)
    (h : ∀ i, HasSum (f i) (a i)) : HasSum (fun s => ∑ i, f i s) (∑ i, a i) := by
  classical
  have key : ∀ S : Finset (Fin n), HasSum (fun s => ∑ i ∈ S, f i s) (∑ i ∈ S, a i) := by
    intro S
    refine Finset.induction_on S ?_ ?_
    · simp only [Finset.sum_empty]
      exact hasSum_zero
    · intro i S hi ih
      simp only [Finset.sum_insert hi]
      exact (h i).add ih
  exact key Finset.univ

theorem res_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochasticMatrix A) (hrev : IsReversibleWrt π A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ j, ∑ i, π i * G i j = π j := by
  intro j
  have hq' : (1 - q) ≠ 0 := ne_of_gt (by linarith)
  have hs : ∀ i, HasSum (fun s : ℕ => π i * ((1 - q) * q ^ s * (A ^ s) i j)) (π i * G i j) := by
    intro i
    rw [hG i j]
    exact ((res_summable A hA q hq0 hq1 i j).hasSum).mul_left (π i)
  have htot : HasSum (fun s : ℕ => ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j))
```

```
      (∑ i, π i * G i j) := res_stationary_hasSum_fin _ _ hs
  have hgeo : HasSum (fun s : ℕ => (1 - q) * q ^ s * π j) (π j) := by
    have := (hasSum_geometric_of_lt_one hq0 hq1).mul_left ((1 - q) * π j)
    convert this using 1
    · funext s; ring
    · rw [mul_comm (1 - q) (π j), mul_assoc, mul_inv_cancel₀ hq', mul_one]
  have heq : (fun s : ℕ => ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j))
      = fun s : ℕ => (1 - q) * q ^ s * π j := by
    funext s
    have hp := setting_stationary_pow π A hA hrev s j
    calc ∑ i, π i * ((1 - q) * q ^ s * (A ^ s) i j)
        = (1 - q) * q ^ s * ∑ i, π i * (A ^ s) i j := by
          rw [Finset.mul_sum]
          refine Finset.sum_congr rfl (fun i _ => ?_)
          ring
      _ = (1 - q) * q ^ s * π j := by rw [hp]
  rw [heq] at htot
  exact htot.unique hgeo

theorem res_left {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j := by
  intro i j
  have hs : ∀ l, Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) l j) :=
    fun l => res_summable A hA q hq0 hq1 l j
  have hs' : ∀ l ∈ (Finset.univ : Finset (Fin n)),
      Summable (fun s : ℕ => A i l * ((1 - q) * q ^ s * (A ^ s) l j)) :=
    fun l _ => (hs l).mul_left (A i l)
  have hswap : ∑' s : ℕ, ∑ l, A i l * ((1 - q) * q ^ s * (A ^ s) l j)
      = ∑ l, ∑' s : ℕ, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    first
    | exact Summable.tsum_finsetSum hs'
    | exact tsum_sum hs'
    | exact (hasSum_sum (fun l hl => (hs' l hl).hasSum)).tsum_eq
  have hsplit : ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j
      = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j
        + ∑' s : ℕ, (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j := by
    rw [(hs i).tsum_eq_zero_add]
    simp only [pow_zero, mul_one]
  have hstep : ∑' s : ℕ, (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j
      = q * ∑' s : ℕ, ∑ l, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    rw [← tsum_mul_left]
    refine tsum_congr (fun s => ?_)
    rw [pow_succ', pow_succ', Matrix.mul_apply, Finset.mul_sum, Finset.mul_sum]
    refine Finset.sum_congr rfl (fun l _ => ?_)
    ring
  have hR : ∀ l, A i l * G l j = ∑' s : ℕ, A i l * ((1 - q) * q ^ s * (A ^ s) l j) := by
    intro l
    rw [hG l j, tsum_mul_left]
  rw [hG i j, hsplit, hstep, hswap, Finset.sum_congr rfl (fun l _ => hR l)]

theorem res_right {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j) :
    ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j := by
  intro i j
  have hs : ∀ l, Summable (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i l) :=
    fun l => res_summable A hA q hq0 hq1 i l
```

```
  have h1 : HasSum (fun s : ℕ => ∑ l, (1 - q) * q ^ s * (A ^ s) i l * A l j)
      (∑ l, G i l * A l j) := by
    refine hasSum_sum (fun l _ => ?_)
    show HasSum (fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i l * A l j) (G i l * A l j)
    rw [hG i l]
    exact (hs l).hasSum.mul_right (A l j)
  have h2 : HasSum (fun s : ℕ => (1 - q) * q ^ (s + 1) * (A ^ (s + 1)) i j)
      (q * ∑ l, G i l * A l j) := by
    have := h1.mul_left q
    convert this using 1
    funext s
    rw [pow_succ q s, pow_succ A s, Matrix.mul_apply]
    simp only [Finset.mul_sum]
    refine Finset.sum_congr rfl (fun l _ => ?_)
    ring
  have h3 := (hasSum_nat_add_iff (f := fun s : ℕ => (1 - q) * q ^ s * (A ^ s) i j) 1).mp h2
  rw [hG i j, h3.tsum_eq]
  simp only [Finset.sum_range_one, pow_zero, mul_one]
  ring

theorem flat_scalar (r r' d : ℝ) (hr : 0 ≤ r ∧ r ≤ 1) (hr' : 0 ≤ r' ∧ r' ≤ 1) (hd : d = 0 ∨ d = 1) :
    |(3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3)|
      ≤ 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
  rcases hr with ⟨hr0, hr1⟩
  rcases hr' with ⟨hr0', hr1'⟩
  set δ := r' - r with hδ
  have hφdiff : (3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3) = δ * (6 * r * (1 - r) + δ * (3 - 4 * r -
↪  2 * r')) := by
    ring
  rw [hφdiff, abs_mul]

  have hA_nonneg : 0 ≤ 6 * r * (1 - r) := by
    nlinarith
  have hA_bound : 6 * r * (1 - r) ≤ 6 * |r - d| := by
    rcases hd with (hd0 | hd1)
    · rw [hd0, sub_zero]
      rw [abs_of_nonneg hr0]
      nlinarith
    · rw [hd1]
      have habs_eq : |r - 1| = 1 - r := by
        have h := abs_of_nonpos (show r - 1 ≤ 0 from by nlinarith)
        linarith
      rw [habs_eq]
      nlinarith
  have h_inner_bound : |3 - 4 * r - 2 * r'| ≤ 3 := by
    have hlow : -3 ≤ 3 - 4 * r - 2 * r' := by nlinarith
    have hhigh : 3 - 4 * r - 2 * r' ≤ 3 := by nlinarith
    exact abs_le.mpr ⟨hlow, hhigh⟩
  have h_abs_δ_le_one : |δ| ≤ 1 := by
    have hδ_low : -1 ≤ δ := by nlinarith
    have hδ_high : δ ≤ 1 := by nlinarith
    exact abs_le.mpr ⟨hδ_low, hδ_high⟩
  have hB_bound : |δ * (3 - 4 * r - 2 * r')| ≤ 3 * |δ| := by
    rw [abs_mul]
    have h_nonneg_absδ : 0 ≤ |δ| := abs_nonneg _
    nlinarith
  have h_sum_bound : |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤ 6 * |r - d| + 3 * |δ| := by
    calc
      |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤
```

```
          |6 * r * (1 - r)| + |δ * (3 - 4 * r - 2 * r')| := abs_add _ _
      _ = 6 * r * (1 - r) + |δ * (3 - 4 * r - 2 * r')| := by rw [abs_of_nonneg hA_nonneg]
      _ ≤ 6 * r * (1 - r) + 3 * |δ| := by nlinarith
      _ ≤ 6 * |r - d| + 3 * |δ| := by nlinarith
  calc
    |δ| * |6 * r * (1 - r) + δ * (3 - 4 * r - 2 * r')| ≤ |δ| * (6 * |r - d| + 3 * |δ|) := by
      have hpos_absδ : 0 ≤ |δ| := abs_nonneg _
      have h_nonneg_nlinarith : 0 ≤ 6 * |r - d| + 3 * |δ| := by
        positivity
      nlinarith
    _ = |δ| * (6 * |r - d|) + |δ| * (3 * |δ|) := by ring
    _ = 6 * |r - d| * |δ| + 3 * (|δ| * |δ|) := by ring
    _ = 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
      have h_sq : |δ| * |δ| = δ ^ 2 := by
        calc
          |δ| * |δ| = |δ| * |δ| := rfl
          _ = δ * δ := abs_mul_abs_self δ
          _ = δ ^ 2 := by ring
      rw [h_sq, hδ]

theorem flat_increment {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u u' : ι → ℝ) (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hu' : ∀ B, 0 ≤ u' B ∧ u' B ≤ 1) :
    (∑ B, w B * |φ (u' B) - φ (u B)|) ^ 2
      ≤ 72 * (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2)
        + 18 * (∑ B, w B * (u' B - u B) ^ 2) ^ 2 := by
  have hS : ∀ B, w B * |φ (u' B) - φ (u B)|
      ≤ w B * (6 * |u B - e B| * |u' B - u B| + 3 * (u' B - u B) ^ 2) := by
    intro B
    have h1 := flat_scalar (u B) (u' B) (e B) (hu B) (hu' B) (he B)
    rw [hφ (u' B), hφ (u B)]
    exact mul_le_mul_of_nonneg_left h1 (hw B)
  have hsum : ∑ B, w B * |φ (u' B) - φ (u B)|
      ≤ 6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) + 3 * (∑ B, w B * (u' B - u B) ^ 2) := by
    calc ∑ B, w B * |φ (u' B) - φ (u B)|
          ≤ ∑ B, w B * (6 * |u B - e B| * |u' B - u B| + 3 * (u' B - u B) ^ 2) :=
            Finset.sum_le_sum (fun B _ => hS B)
      _ = 6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) + 3 * (∑ B, w B * (u' B - u B) ^ 2) := by
            rw [Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
            exact Finset.sum_congr rfl (fun B _ => by ring)
  have hCS : (∑ B, w B * (|u B - e B| * |u' B - u B|)) ^ 2
      ≤ (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2) := by
    have h : (∑ B, (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)) ^ 2
        ≤ (∑ B, (Real.sqrt (w B) * |u B - e B|) ^ 2) * (∑ B, (Real.sqrt (w B) * |u' B - u B|) ^ 2) :=
      Finset.sum_mul_sq_le_sq_mul_sq _ _ _
    have k1 : ∑ B, (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)
        = ∑ B, w B * (|u B - e B| * |u' B - u B|) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      have hs := Real.mul_self_sqrt (hw B)
      calc (Real.sqrt (w B) * |u B - e B|) * (Real.sqrt (w B) * |u' B - u B|)
            = (Real.sqrt (w B) * Real.sqrt (w B)) * (|u B - e B| * |u' B - u B|) := by ring
        _ = w B * (|u B - e B| * |u' B - u B|) := by rw [hs]
    have k2 : ∑ B, (Real.sqrt (w B) * |u B - e B|) ^ 2 = ∑ B, w B * (u B - e B) ^ 2 := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      have hs := Real.mul_self_sqrt (hw B)
      calc (Real.sqrt (w B) * |u B - e B|) ^ 2
            = (Real.sqrt (w B) * Real.sqrt (w B)) * |u B - e B| ^ 2 := by ring
        _ = w B * (u B - e B) ^ 2 := by rw [hs, sq_abs]
```

```
    have k3 : ∑ B, (Real.sqrt (w B) * |u' B - u B|) ^ 2 = ∑ B, w B * (u' B - u B) ^ 2 := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      have hs := Real.mul_self_sqrt (hw B)
      calc (Real.sqrt (w B) * |u' B - u B|) ^ 2
            = (Real.sqrt (w B) * Real.sqrt (w B)) * |u' B - u B| ^ 2 := by ring
        _ = w B * (u' B - u B) ^ 2 := by rw [hs, sq_abs]
    rw [k1, k2, k3] at h
    exact h
  have hX0 : 0 ≤ ∑ B, w B * (|u B - e B| * |u' B - u B|) :=
    Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (mul_nonneg (abs_nonneg _) (abs_nonneg _)))
  have hY0 : 0 ≤ ∑ B, w B * (u' B - u B) ^ 2 :=
    Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (sq_nonneg _))
  have hP0 : 0 ≤ ∑ B, w B * (u B - e B) ^ 2 :=
    Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (sq_nonneg _))
  have hS0 : 0 ≤ ∑ B, w B * |φ (u' B) - φ (u B)| :=
    Finset.sum_nonneg (fun B _ => mul_nonneg (hw B) (abs_nonneg _))
  have hmul := mul_le_mul hsum hsum hS0 (le_trans hS0 hsum)
  nlinarith [hmul, hCS, hX0, hY0, hP0,
    sq_nonneg (6 * (∑ B, w B * (|u B - e B| * |u' B - u B|)) - 3 * (∑ B, w B * (u' B - u B) ^ 2))]

theorem flat_ineq {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u u' : ι → ℝ) (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hu' : ∀ B, 0 ≤ u' B ∧ u' B ≤ 1) :
    (∑ B, w B * |φ (u' B) - φ (u B)|) ^ 2
      ≤ 108 * ((∑ B, w B * (u' B - e B) ^ 2) ^ 2 - (∑ B, w B * (u B - e B) ^ 2) ^ 2
          - 4 * (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u B - e B) * (u' B - u B))) := by
  have hinc := flat_increment w hw φ hφ e u u' he hu hu'
  have hP' : ∑ B, w B * (u' B - e B) ^ 2
      = ∑ B, w B * (u B - e B) ^ 2 + 2 * ∑ B, w B * (u B - e B) * (u' B - u B)
        + ∑ B, w B * (u' B - u B) ^ 2 := by
    rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_add_distrib]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    ring
  have hCS : (∑ B, w B * (u B - e B) * (u' B - u B)) ^ 2
      ≤ (∑ B, w B * (u B - e B) ^ 2) * (∑ B, w B * (u' B - u B) ^ 2) := by
    have h := Finset.sum_mul_sq_le_sq_mul_sq Finset.univ
      (fun B => Real.sqrt (w B) * (u B - e B)) (fun B => Real.sqrt (w B) * (u' B - u B))
    have h1 : ∀ B, Real.sqrt (w B) * (u B - e B) * (Real.sqrt (w B) * (u' B - u B))
        = w B * (u B - e B) * (u' B - u B) := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u B - e B) * (u' B - u B)) * hs
    have h2 : ∀ B, (Real.sqrt (w B) * (u B - e B)) ^ 2 = w B * (u B - e B) ^ 2 := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u B - e B) ^ 2) * hs
    have h3 : ∀ B, (Real.sqrt (w B) * (u' B - u B)) ^ 2 = w B * (u' B - u B) ^ 2 := by
      intro B
      have hs := Real.mul_self_sqrt (hw B)
      linear_combination ((u' B - u B) ^ 2) * hs
    simp only [h1, h2, h3] at h
    exact h
  rw [hP']
  nlinarith [hinc, hCS,
    sq_nonneg (2 * (∑ B, w B * (u B - e B) * (u' B - u B)) + ∑ B, w B * (u' B - u B) ^ 2),
    sq_nonneg (4 * (∑ B, w B * (u B - e B) * (u' B - u B)) + ∑ B, w B * (u' B - u B) ^ 2)]

theorem local_ineq_combine {m : ℕ} (p S0 a Fv Lv : ℝ) (c b FU L : Fin m → ℝ)
```

```
    (hp : 0 ≤ p) (hc : ∀ j, 0 ≤ c j) (hsum : p + ∑ j, c j = 1)
    (hlin : p * Lv + ∑ j, c j * L j = 0)
    (ha : a ≤ 108 * (Fv ^ 2 - S0 ^ 2 - 4 * S0 * Lv))
    (hb : ∀ j, b j ≤ 108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j)) :
    p * a + ∑ j, c j * b j ≤ 108 * (p * Fv ^ 2 + ∑ j, c j * FU j ^ 2 - S0 ^ 2) := by
  have h1 : ∑ j, c j * b j ≤ ∑ j, c j * (108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j)) :=
    Finset.sum_le_sum fun j _ => mul_le_mul_of_nonneg_left (hb j) (hc j)
  have h2 : ∑ j, c j * (108 * (FU j ^ 2 - S0 ^ 2 - 4 * S0 * L j))
      = 108 * ∑ j, c j * FU j ^ 2 - 108 * S0 ^ 2 * ∑ j, c j - 432 * S0 * ∑ j, c j * L j := by
    simp only [Finset.mul_sum, ← Finset.sum_sub_distrib]
    exact Finset.sum_congr rfl fun j _ => by ring
  have hsc : ∑ j, c j = 1 - p := by linarith
  have hlin' : ∑ j, c j * L j = -(p * Lv) := by linarith
  rw [hsc, hlin'] at h2
  have h3 := mul_le_mul_of_nonneg_left ha hp
  nlinarith [h1, h2, h3]

theorem local_ineq {ι : Type*} [Fintype ι] {m : ℕ} (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e u v : ι → ℝ) (U : Fin m → ι → ℝ) (p : ℝ) (c : Fin m → ℝ)
    (hp : 0 ≤ p) (hc : ∀ j, 0 ≤ c j) (hsum : p + ∑ j, c j = 1)
    (hbal : ∀ B, u B = p * v B + ∑ j, c j * U j B)
    (he : ∀ B, e B = 0 ∨ e B = 1)
    (hu : ∀ B, 0 ≤ u B ∧ u B ≤ 1) (hv : ∀ B, 0 ≤ v B ∧ v B ≤ 1)
    (hU : ∀ j B, 0 ≤ U j B ∧ U j B ≤ 1) :
    p * (∑ B, w B * |φ (v B) - φ (u B)|) ^ 2 + ∑ j, c j * (∑ B, w B * |φ (U j B) - φ (u B)|) ^ 2
      ≤ 108 * (p * (∑ B, w B * (v B - e B) ^ 2) ^ 2
          + ∑ j, c j * (∑ B, w B * (U j B - e B) ^ 2) ^ 2
          - (∑ B, w B * (u B - e B) ^ 2) ^ 2) := by
  have hinner : ∀ B, p * (v B - u B) + ∑ j, c j * (U j B - u B) = 0 := by
    intro B
    have h1 : ∑ j, c j * (U j B - u B) = ∑ j, c j * U j B - (∑ j, c j) * u B := by
      rw [Finset.sum_mul, ← Finset.sum_sub_distrib]
      exact Finset.sum_congr rfl fun j _ => by ring
    have hs : ∑ j, c j = 1 - p := by linarith
    rw [h1, hs]
    have hb := hbal B
    linear_combination (-1 : ℝ) * hb
  have hlin : p * (∑ B, w B * (u B - e B) * (v B - u B))
      + ∑ j, c j * (∑ B, w B * (u B - e B) * (U j B - u B)) = 0 := by
    have hpt : ∀ B, w B * (u B - e B) * (p * (v B - u B) + ∑ j, c j * (U j B - u B))
        = p * (w B * (u B - e B) * (v B - u B))
          + ∑ j, c j * (w B * (u B - e B) * (U j B - u B)) := by
      intro B
      rw [mul_add, Finset.mul_sum]
      congr 1
      · ring
      · exact Finset.sum_congr rfl fun j _ => by ring
    have hz : ∑ B, w B * (u B - e B) * (p * (v B - u B) + ∑ j, c j * (U j B - u B)) = 0 :=
      Finset.sum_eq_zero fun B _ => by rw [hinner B, mul_zero]
    rw [Finset.sum_congr rfl fun B _ => hpt B, Finset.sum_add_distrib, ← Finset.mul_sum,
      Finset.sum_comm] at hz
    have hz2 : ∑ j, c j * (∑ B, w B * (u B - e B) * (U j B - u B))
        = ∑ j, ∑ B, c j * (w B * (u B - e B) * (U j B - u B)) :=
      Finset.sum_congr rfl fun j _ => by rw [Finset.mul_sum]
    rw [hz2]
    exact hz
  exact local_ineq_combine p (∑ B, w B * (u B - e B) ^ 2) _ (∑ B, w B * (v B - e B) ^ 2)
    (∑ B, w B * (u B - e B) * (v B - u B)) c
```

```
    (fun j => (∑ B, w B * |φ (U j B) - φ (u B)|) ^ 2)
    (fun j => ∑ B, w B * (U j B - e B) ^ 2)
    (fun j => ∑ B, w B * (u B - e B) * (U j B - u B))
    hp hc hsum hlin (flat_ineq w hw φ hφ e u v he hu hv)
    (fun j => flat_ineq w hw φ hφ e u (U j) he hu (hU j))

theorem global_sum {n : ℕ} (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A G : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ) (hp : 0 ≤ p)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (hGr : ∀ k j, G k j = p * (1 : Matrix (Fin n) (Fin n) ℝ) k j + q * ∑ i, G k i * A i j)
    (Jd : Fin n → ℝ) (Jh Fz Fh : Fin n → Fin n → ℝ) (hFh : ∀ k i, 0 ≤ Fh k i)
    (hloc : ∀ k i, p * Jd i + q * ∑ j, A i j * Jh i j
        ≤ 108 * (p * Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) :
    ∑ i, π i * (p * Jd i + q * ∑ j, A i j * Jh i j)
      ≤ 108 * p * ∑ k, ∑ i, π k * G k i * Fz k i := by
  have key : ∀ k, ∑ i, G k i * (q * ∑ j, A i j * Fh k j) = ∑ j, G k j * Fh k j - p * Fh k k := by
    intro k
    have e1 : ∑ i, G k i * (q * ∑ j, A i j * Fh k j) = ∑ j, (q * ∑ i, G k i * A i j) * Fh k j := by
      simp only [Finset.mul_sum, Finset.sum_mul]
      rw [Finset.sum_comm]
      refine Finset.sum_congr rfl (fun j _ => Finset.sum_congr rfl (fun i _ => by ring))
    have e2 : ∀ j, G k j * Fh k j = p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j + (q * ∑ i, G k i * A i
↪  j) * Fh k j := by
      intro j
      conv_lhs => rw [hGr k j]
      ring
    have e3 : ∑ j, p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j = p * Fh k k := by
      simp [Matrix.one_apply, Finset.sum_ite_eq]
    have e4 : ∑ j, G k j * Fh k j = ∑ j, (p * (1 : Matrix (Fin n) (Fin n) ℝ) k j * Fh k j + (q * ∑ i, G
↪  k i * A i j) * Fh k j) :=
      Finset.sum_congr rfl (fun j _ => e2 j)
    rw [e1, e4, Finset.sum_add_distrib, e3]
    ring
  have hk : ∀ k, ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) ≤ 108 * p * ∑ i, π k * G k i *
↪  Fz k i := by
    intro k
    have s1 : ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) ≤ ∑ i, π k * G k i * (108 * (p *
↪  Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) :=
      Finset.sum_le_sum (fun i _ => mul_le_mul_of_nonneg_left (hloc k i) (mul_nonneg (hπ k) (hG0 k i)))
    have s2 : ∑ i, π k * G k i * (108 * (p * Fz k i + q * ∑ j, A i j * Fh k j - Fh k i)) = 108 * π k * (p
↪  * ∑ i, G k i * Fz k i + ∑ i, G k i * (q * ∑ j, A i j * Fh k j) - ∑ i, G k i * Fh k i) := by
      rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib, Finset.mul_sum]
      refine Finset.sum_congr rfl (fun i _ => by ring)
    have s3 : ∑ i, π k * G k i * Fz k i = π k * ∑ i, G k i * Fz k i := by
      rw [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun i _ => by ring)
    rw [s2, key k] at s1
    rw [s3]
    nlinarith [mul_nonneg (mul_nonneg (hπ k) hp) (hFh k k)]
  calc ∑ i, π i * (p * Jd i + q * ∑ j, A i j * Jh i j)
      = ∑ i, ∑ k, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) := by
        refine Finset.sum_congr rfl (fun i _ => ?_)
        rw [← hGπ i, Finset.sum_mul]
    _ = ∑ k, ∑ i, π k * G k i * (p * Jd i + q * ∑ j, A i j * Jh i j) := Finset.sum_comm
    _ ≤ ∑ k, 108 * p * ∑ i, π k * G k i * Fz k i := Finset.sum_le_sum (fun k _ => hk k)
    _ = 108 * p * ∑ k, ∑ i, π k * G k i * Fz k i := by rw [Finset.mul_sum]

theorem h_props {n : ℕ} {ι : Type*} [Fintype ι] (G A : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
```

```
    (hGl : ∀ i j, G i j = p * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
    (z : Fin n → ι → ℝ) (hz : ∀ i B, 0 ≤ z i B ∧ z i B ≤ 1)
    (h : Fin n → ι → ℝ) (hh : ∀ i B, h i B = ∑ j, G i j * z j B) :
    (∀ i B, 0 ≤ h i B ∧ h i B ≤ 1) ∧
      (∀ i B, h i B = p * z i B + ∑ j, (q * A i j) * h j B) := by
  have h_nonneg : ∀ i B, 0 ≤ h i B := by
    intro i B
    rw [hh i B]
    refine Finset.sum_nonneg (fun j _ => ?_)
    have hzj := hz j B
    have hGij := hG0 i j
    nlinarith
  have h_le_one : ∀ i B, h i B ≤ 1 := by
    intro i B
    rw [hh i B]
    calc
      ∑ j, G i j * z j B ≤ ∑ j, G i j * 1 := by
        refine Finset.sum_le_sum (fun j _ => ?_)
        have hzj := hz j B
        have hGij := hG0 i j
        nlinarith
      _ = ∑ j, G i j := by simp
      _ = 1 := hGrow i
  have h_harmonic : ∀ i B, h i B = p * z i B + ∑ j, (q * A i j) * h j B := by
    intro i B
    rw [hh i B]
    have hGl_i := hGl i
    calc
      ∑ k, G i k * z k B = ∑ k, (p * (1 : Matrix (Fin n) (Fin n) ℝ) i k + q * ∑ l, A i l * G l k) * z k B
↪   := by
        refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hGl_i k]
      _ = ∑ k, (p * (1 : Matrix (Fin n) (Fin n) ℝ) i k * z k B + q * (∑ l, A i l * G l k) * z k B) := by
        refine Finset.sum_congr rfl (fun k _ => ?_)
        ring
      _ = (∑ k, p * (1 : Matrix (Fin n) (Fin n) ℝ) i k * z k B) + (∑ k, q * (∑ l, A i l * G l k) * z k
↪   B) := by
        rw [Finset.sum_add_distrib]
      _ = p * z i B + (∑ k, q * (∑ l, A i l * G l k) * z k B) := by
        simp [Matrix.one_apply]
      _ = p * z i B + (∑ l, ∑ k, q * A i l * G l k * z k B) := by
        rw [Finset.sum_comm]
        simp [Finset.mul_sum, mul_comm, mul_left_comm]
      _ = p * z i B + (∑ l, q * A i l * (∑ k, G l k * z k B)) := by
        simp [Finset.mul_sum, mul_assoc]
      _ = p * z i B + ∑ l, (q * A i l) * h l B := by
        refine congrArg (fun t => p * z i B + t) ?_
        refine Finset.sum_congr rfl (fun l _ => ?_)
        simp [hh l B, mul_assoc]
  refine ⟨fun i B => ⟨h_nonneg i B, h_le_one i B⟩, h_harmonic⟩

theorem cut_coord_upper_shift {n : ℕ} (a : Fin n → ℝ) (m c : ℝ) (hmc : m < c)
    (hm : ∀ i, m ≤ a i) (hc : ∀ i, a i < c → a i = m) (b' : Finset (Fin n) → ℝ)
    (hb0 : ∀ B, 0 ≤ b' B)
    (hbup : ∀ B, b' B ≠ 0 → ∀ i j, j ∈ B → max (a j) c ≤ max (a i) c → i ∈ B)
    (hbsum : ∀ i j, max (a i) c - max (a j) c
      = ∑ B, b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
```

```
    (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) := by
  obtain ⟨Bstar, hBstar_mem⟩ : ∃ S : Finset (Fin n), ∀ i, i ∈ S ↔ c ≤ a i :=
    ⟨Finset.univ.filter (fun i => c ≤ a i), fun i => by simp⟩
  have hidx : ∀ k, a k = max (a k) c + (c - m) * ((if k ∈ Bstar then (1:ℝ) else 0) - 1) := by
    intro k
    by_cases hk : c ≤ a k
    · rw [if_pos ((hBstar_mem k).2 hk), max_eq_left hk]
      ring
    · have hk2 : a k < c := lt_of_not_ge hk
      have hk3 : a k = m := hc k hk2
      rw [if_neg (fun h => hk ((hBstar_mem k).1 h)), max_eq_right hk2.le]
      rw [hk3]
      ring
  refine ⟨fun B => b' B + (if B = Bstar then c - m else 0), ?_, ?_, ?_⟩
  · intro B
    show 0 ≤ b' B + (if B = Bstar then c - m else 0)
    have h1 := hb0 B
    split_ifs <;> linarith
  · intro B hB i j hj hle
    by_cases hB' : B = Bstar
    · rw [hB'] at hj ⊢
      exact (hBstar_mem i).2 (le_trans ((hBstar_mem j).1 hj) hle)
    · have hb'ne : b' B ≠ 0 := by
        intro h0
        apply hB
        show b' B + (if B = Bstar then c - m else 0) = 0
        rw [if_neg hB', h0, add_zero]
      exact hbup B hb'ne i j hj (max_le (le_trans hle (le_max_left _ _)) (le_max_right _ _))
  · intro i j
    show a i - a j = ∑ B, (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
    have hsplit : ∀ B : Finset (Fin n), (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
        = b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
          + (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else
↪  0)) else 0) := by
      intro B
      by_cases hB : B = Bstar
      · rw [if_pos hB, if_pos hB, hB]
        ring
      · rw [if_neg hB, if_neg hB]
        ring
    have hsingle : (∑ B : Finset (Fin n), (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) -
↪  (if j ∈ Bstar then (1:ℝ) else 0)) else 0))
        = (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else 0)) := by
      rw [Finset.sum_eq_single Bstar (fun B _ hB => if_neg hB) (fun h => absurd (Finset.mem_univ _) h),
↪  if_pos rfl]
    have hsum : (∑ B : Finset (Fin n), (b' B + (if B = Bstar then c - m else 0)) *
        ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)))
        = ∑ B : Finset (Fin n), (b' B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
          + (if B = Bstar then (c - m) * ((if i ∈ Bstar then (1:ℝ) else 0) - (if j ∈ Bstar then (1:ℝ) else
↪  0)) else 0)) :=
      Finset.sum_congr rfl (fun B _ => hsplit B)
    rw [hsum, Finset.sum_add_distrib]
    linear_combination hidx i - hidx j + hbsum i j - hsingle

theorem cut_coord_upper_reduce {n : ℕ} (a : Fin n → ℝ) (hne : ∃ i j, a i ≠ a j) :
    ∃ m c : ℝ, m < c ∧ (∀ i, m ≤ a i) ∧ (∀ i, a i < c → a i = m) ∧
      (Finset.univ.image (fun i => max (a i) c)).card < (Finset.univ.image a).card := by
```

```
let s := Finset.image a Finset.univ
have hsne : s.Nonempty := by
  rcases hne with ⟨i, j, hne'⟩
  exact ⟨a i, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
let m := s.min' hsne
have hm_le : ∀ i, m ≤ a i := by
  intro i
  apply Finset.min'_le _ _ (Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩)
have hm_mem : m ∈ s := Finset.min'_mem _ hsne
by_cases hall : ∀ i, a i = m
· exfalso
  rcases hne with ⟨i, j, h⟩
  rw [hall i, hall j] at h
  exact h rfl
have h_exists_gt : ∃ i, m < a i := by
  push_neg at hall
  rcases hall with ⟨i, hi⟩
  exact ⟨i, lt_of_le_of_ne (hm_le i) (Ne.symm hi)⟩
let s' := s.erase m
have hs'_nonempty : s'.Nonempty := by
  rcases h_exists_gt with ⟨i, hi⟩
  have hi_mem : a i ∈ s := Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩
  have hne_mem : a i ≠ m := by linarith
  exact ⟨a i, Finset.mem_erase.mpr ⟨hne_mem, hi_mem⟩⟩
let c := s'.min' hs'_nonempty
have hc_mem_s' : c ∈ s' := Finset.min'_mem _ hs'_nonempty
have hc_ne_m : c ≠ m := Finset.ne_of_mem_erase hc_mem_s'
have hc_mem_s : c ∈ s := Finset.mem_of_mem_erase hc_mem_s'
have hc_mem_image : c ∈ Finset.image a Finset.univ := hc_mem_s
have hmc : m < c := by
  have hle := Finset.min'_le _ _ hc_mem_s
  by_contra! h
  have heq : c = m := le_antisymm h hle
  exact hc_ne_m heq

have h_lt_eq : ∀ i, a i < c → a i = m := by
  intro i hi_lt
  by_contra! h_ne
  have hi_mem' : a i ∈ s' := Finset.mem_erase.mpr ⟨h_ne, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
  have hle' : c ≤ a i := Finset.min'_le _ _ hi_mem'
  linarith




have h_max_image_eq_erase : Finset.image (fun i => max (a i) c) Finset.univ = s.erase m := by
  ext x
  constructor
  · intro hx
    rcases Finset.mem_image.mp hx with ⟨i, _, rfl⟩
    rw [Finset.mem_erase]
    by_cases hle : a i ≤ c
    · rw [max_eq_right hle]
      exact ⟨hc_ne_m, hc_mem_image⟩
    · rw [max_eq_left (by linarith)]
      refine ⟨by
        intro h_eq
        rw [h_eq] at hle
        linarith, Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩⟩
```

```
    · intro hx
      rcases Finset.mem_erase.mp hx with ⟨hx_ne_m, hx_mem⟩
      rcases Finset.mem_image.mp hx_mem with ⟨i, _, rfl⟩
      by_cases hle : a i ≤ c
      ·

        have heq : a i = c := by
          by_contra! h_lt
          have : a i < c := lt_of_le_of_ne hle h_lt
          have : a i = m := h_lt_eq i this
          exact hx_ne_m this
        have hmax : max (a i) c = a i := by rw [heq, max_self]
        exact Finset.mem_image.mpr ⟨i, Finset.mem_univ _, hmax⟩
      ·
        have hmax : max (a i) c = a i := max_eq_left (by linarith)
        exact Finset.mem_image.mpr ⟨i, Finset.mem_univ _, hmax⟩
  have h_max_image_card_lt : (Finset.image (fun i => max (a i) c) Finset.univ).card < (Finset.image a
↪  Finset.univ).card := by
    rw [h_max_image_eq_erase]
    exact Finset.card_erase_lt_of_mem hm_mem
  exact ⟨m, c, hmc, hm_le, h_lt_eq, h_max_image_card_lt⟩

theorem cut_coord_upper {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) := by
  suffices H : ∀ k : ℕ, ∀ a : Fin n → ℝ, (Finset.univ.image a).card ≤ k →
      ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
        (∀ B, b B ≠ 0 → ∀ i j, j ∈ B → a j ≤ a i → i ∈ B) ∧
        (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) from
    H _ a le_rfl
  intro k
  induction k with
  | zero =>
    intro a ha
    by_cases hall : ∀ i j, a i = a j
    · exact ⟨fun _ => 0, fun _ => le_rfl, fun B hB => absurd rfl hB, fun i j => by simp [hall i j]⟩
    · push_neg at hall
      obtain ⟨m, c, -, -, -, hcard⟩ := cut_coord_upper_reduce a hall
      omega
  | succ k ih =>
    intro a ha
    by_cases hall : ∀ i j, a i = a j
    · exact ⟨fun _ => 0, fun _ => le_rfl, fun B hB => absurd rfl hB, fun i j => by simp [hall i j]⟩
    · push_neg at hall
      obtain ⟨m, c, hmc, hm, hc, hcard⟩ := cut_coord_upper_reduce a hall
      obtain ⟨b', hb0, hbup, hbsum⟩ := ih (fun i => max (a i) c) (by omega)
      exact cut_coord_upper_shift a m c hmc hm hc b' hb0 hbup hbsum

theorem cut_coord_core {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) ∧
      (∀ i j, |a i - a j| = ∑ B, b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) := by
  obtain ⟨b, hb0, hup, hdiff⟩ := cut_coord_upper a
  refine ⟨b, hb0, hdiff, ?_⟩
  have key : ∀ i j, a j ≤ a i → ∀ B : Finset (Fin n),
      b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))
        = b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| := by
    intro i j hij B
```

```
    by_cases hb : b B = 0
    · simp [hb]
    · have h := hup B hb i j
      by_cases hj : j ∈ B
      · have hi := h hj hij
        simp [hi, hj]
      · by_cases hi : i ∈ B <;> simp [hi, hj]
  intro i j
  rcases le_total (a j) (a i) with h | h
  · rw [abs_of_nonneg (sub_nonneg.mpr h), hdiff i j]
    exact Finset.sum_congr rfl (fun B _ => key i j h B)
  · rw [abs_sub_comm (a i) (a j), abs_of_nonneg (sub_nonneg.mpr h), hdiff j i]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    rw [key j i h B, abs_sub_comm]

theorem cut_coord {n : ℕ} (a : Fin n → ℝ) :
    ∃ b : Finset (Fin n) → ℝ, (∀ B, 0 ≤ b B) ∧ (∀ B, b B ≤ 2 * ∑ i, |a i|) ∧
      (∀ i j, a i - a j = ∑ B, b B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0))) ∧
      (∀ i j, |a i - a j| = ∑ B, b B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) := by
  classical
  obtain ⟨b, hb0, hsgn, habs⟩ := cut_coord_core a
  refine ⟨fun B => if (∃ i, i ∈ B) ∧ (∃ j, j ∉ B) then b B else 0, ?_, ?_, ?_, ?_⟩
  · intro B
    dsimp only
    split_ifs
    · exact hb0 B
    · exact le_refl 0
  · intro B
    dsimp only
    have hS : 0 ≤ ∑ i, |a i| := Finset.sum_nonneg (fun i _ => abs_nonneg _)
    split_ifs with h
    · obtain ⟨⟨i, hi⟩, ⟨j, hj⟩⟩ := h
      have h1 := habs i j
      have hle : b B ≤ |a i - a j| := by
        rw [h1]
        have := Finset.single_le_sum
          (f := fun B' => b B' * |(if i ∈ B' then (1:ℝ) else 0) - (if j ∈ B' then (1:ℝ) else 0)|)
          (fun B' _ => mul_nonneg (hb0 B') (abs_nonneg _)) (Finset.mem_univ B)
        simpa [hi, hj] using this
      have h2 : |a i - a j| ≤ |a i| + |a j| :=
        abs_sub_le_iff.mpr ⟨by linarith [le_abs_self (a i), neg_abs_le (a j)],
          by linarith [le_abs_self (a j), neg_abs_le (a i)]⟩
      have hi' := Finset.single_le_sum (f := fun k => |a k|) (fun k _ => abs_nonneg _) (Finset.mem_univ i)
      have hj' := Finset.single_le_sum (f := fun k => |a k|) (fun k _ => abs_nonneg _) (Finset.mem_univ j)
      simp only at hi' hj'
      linarith
    · linarith
  · intro i j
    rw [hsgn i j]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    dsimp only
    by_cases h : (∃ i, i ∈ B) ∧ (∃ j, j ∉ B)
    · rw [if_pos h]
    · rw [if_neg h]
      have hd : (if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0) = 0 := by
        by_cases hi : i ∈ B
        · have hj : j ∈ B := by
            by_contra hj
            exact h ⟨⟨i, hi⟩, ⟨j, hj⟩⟩
```

```
          simp [hi, hj]
        · have hj : j ∉ B := by
            intro hj
            exact h ⟨⟨j, hj⟩, ⟨i, hi⟩⟩
          simp [hi, hj]
      rw [hd]
      ring
  · intro i j
    rw [habs i j]
    refine Finset.sum_congr rfl (fun B _ => ?_)
    dsimp only
    by_cases h : (∃ i, i ∈ B) ∧ (∃ j, j ∉ B)
    · rw [if_pos h]
    · rw [if_neg h]
      have hd : (if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0) = 0 := by
        by_cases hi : i ∈ B
        · have hj : j ∈ B := by
            by_contra hj
            exact h ⟨⟨i, hi⟩, ⟨j, hj⟩⟩
          simp [hi, hj]
        · have hj : j ∉ B := by
            intro hj
            exact h ⟨⟨j, hj⟩, ⟨i, hi⟩⟩
          simp [hi, hj]
      rw [hd, abs_zero]
      ring

theorem cut_l1_ext (f g : L1Space) (h : ∀ k, f k = g k) : f = g := by
  ext k
  exact h k

theorem cut_l1_apply {ι : Type*} [Fintype ι] (x0 : L1Space) (c : ι → ℝ) (β : ι → L1Space) (k : ℕ) :
    (x0 + ∑ B, c B • β B) k = x0 k + ∑ B, c B * β B k := by
  simp

theorem assembly_real_split {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) (p q : ℝ)
    (a : Fin n → ℝ) (b : Fin n → Fin n → ℝ) :
    ∑ i, π i * (p * a i + q * ∑ j, A i j * b i j)
      = p * ∑ i, π i * a i + q * ∑ i, ∑ j, π i * A i j * b i j := by
  rw [Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun i _ => ?_)
  have e : ∑ j, π i * A i j * b i j = π i * ∑ j, A i j * b i j := by
    rw [Finset.mul_sum]
    exact Finset.sum_congr rfl (fun j _ => by ring)
  rw [e]
  ring

theorem assembly_real_finish (q τ S1 S2 R : ℝ) (hq1 : q < 1) (hτ : τ * (1 - q) = q)
    (h : (1 - q) * S1 + q * S2 ≤ 108 * (1 - q) * R) : S1 + τ * S2 ≤ 108 * R := by
  have hpos : 0 < 1 - q := by linarith
  have e : (1 - q) * (S1 + τ * S2) = (1 - q) * S1 + q * S2 := by
    linear_combination S2 * hτ
  have h2 : (1 - q) * (S1 + τ * S2) ≤ (1 - q) * (108 * R) := by
    rw [e]; linarith
  exact le_of_mul_le_mul_left h2 hpos

theorem assembly_real {n : ℕ} {ι : Type*} [Fintype ι] (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B) (z : Fin n → ι → ℝ) (hz : ∀ i B, z i B = 0 ∨ z i B = 1)
```

```
  (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (τ : ℝ) (hτ : τ * (1 - q) = q)
  (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
  (hGπ : ∀ j, ∑ i, π i * G i j = π j)
  (hGl : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
  (hGr : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j) :
  ∃ h : Fin n → ι → ℝ,
    (∑ i, π i * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
      + τ * (∑ i, ∑ j, π i * A i j *
          (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ B, w B * |z i B - z j B|) ^ 2 := by
have hp0 : 0 ≤ 1 - q := by linarith
have hz01 : ∀ i B, 0 ≤ z i B ∧ z i B ≤ 1 := by
  intro i B
  rcases hz i B with e | e <;> rw [e] <;> norm_num
obtain ⟨h, hh⟩ : ∃ h : Fin n → ι → ℝ, ∀ i B, h i B = ∑ j, G i j * z j B :=
  ⟨fun i B => ∑ j, G i j * z j B, fun _ _ => rfl⟩
obtain ⟨hhb, hhbal⟩ := h_props G A (1 - q) q hG0 hGrow hGl z hz01 h hh
obtain ⟨φ, hφ⟩ : ∃ φ : ℝ → ℝ, ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3 := ⟨_, fun _ => rfl⟩
have hrow : ∀ i, (1 - q) + ∑ j, q * A i j = 1 := by
  intro i
  rw [← Finset.mul_sum, hA.2 i]
  ring
have hloc : ∀ k i, (1 - q) * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2
   + q * ∑ j, A i j * (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2
    ≤ 108 * ((1 - q) * (∑ B, w B * |z k B - z i B|) ^ 2
      + q * ∑ j, A i j * (∑ B, w B * (h j B - z k B) ^ 2) ^ 2
      - (∑ B, w B * (h i B - z k B) ^ 2) ^ 2) := by
  intro k i
  have key := local_ineq w hw φ hφ (z k) (h i) (z i) h (1 - q) (fun j => q * A i j)
    hp0 (fun j => mul_nonneg hq0 (hA.1 i j)) (hrow i) (hhbal i) (hz k)
    (hhb i) (hz01 i) hhb
  have e1 : ∀ B, |3 * z i B ^ 2 - 2 * z i B ^ 3 - (3 * h i B ^ 2 - 2 * h i B ^ 3)|
      = |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)| := by
    intro B
    rcases hz i B with e | e <;> rw [e] <;> norm_num
  have e2 : ∀ j B, |3 * h j B ^ 2 - 2 * h j B ^ 3 - (3 * h i B ^ 2 - 2 * h i B ^ 3)|
      = |3 * h i B ^ 2 - 2 * h i B ^ 3 - (3 * h j B ^ 2 - 2 * h j B ^ 3)| :=
    fun j B => abs_sub_comm _ _
  have e3 : ∀ B, (z i B - z k B) ^ 2 = |z k B - z i B| := by
    intro B
    rcases hz i B with e | e <;> rcases hz k B with e' | e' <;> rw [e, e'] <;> norm_num
  simp only [hφ, e1, e2, e3] at key
  have c1 : ∀ (f : Fin n → ℝ), ∑ j, q * A i j * f j = q * ∑ j, A i j * f j := by
    intro f
    rw [Finset.mul_sum]
    exact Finset.sum_congr rfl (fun j _ => by ring)
  rw [c1, c1] at key
  exact key
have gs := global_sum π hπ A G (1 - q) q hp0 hG0 hGπ hGr
  (fun i => (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
  (fun i j => (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
  (fun k i => (∑ B, w B * |z k B - z i B|) ^ 2)
  (fun k i => (∑ B, w B * (h i B - z k B) ^ 2) ^ 2)
  (fun k i => sq_nonneg _) hloc
have hL := assembly_real_split π A (1 - q) q
  (fun i => (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
  (fun i j => (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2)
refine ⟨h, assembly_real_finish q τ _ _ _ hq1 hτ ?_⟩
linarith [gs, hL]
```

```
theorem cut_l1_norm_combo {ι : Type*} [Fintype ι] (f : L1Space) (β : ι → L1Space) (d : ι → ℝ)
    (hβ : ∀ B k, 0 ≤ β B k) (hf : ∀ k, |f k| = ∑ B, β B k * d B) :
    ‖f‖ = ∑ B, ‖β B‖ * d B := by
  have hp : 0 < (1 : ENNReal).toReal := by norm_num
  have hnorm_f : ‖f‖ = ∑' k, |f k| := by
    rw [lp.norm_eq_tsum_rpow hp f]
    simp
  have hnorm_β : ∀ B, ‖β B‖ = ∑' k, |β B k| := by
    intro B
    rw [lp.norm_eq_tsum_rpow hp (β B)]
    simp
  have habs_β : ∀ B k, |β B k| = β B k := by
    intro B k
    rw [abs_of_nonneg (hβ B k)]
  rw [hnorm_f]


  have h_summable_β : ∀ B, Summable (fun k : ℕ => β B k) := by
    intro B
    have h_sum_abs : Summable (fun k : ℕ => |β B k|) := by
      have h := (lp.memℓp (β B)).summable hp
      simpa using h
    have h_eq : (fun k : ℕ => |β B k|) = (fun k : ℕ => β B k) := by
      ext k
      exact habs_β B k
    rwa [h_eq] at h_sum_abs

  have h_summable_mul : ∀ B, Summable (fun k : ℕ => β B k * d B) := by
    intro B
    exact Summable.mul_right (d B) (h_summable_β B)

  have h_exchange := Summable.tsum_finsetSum (s := Finset.univ)
    (fun B _ => h_summable_mul B)

  calc
    (∑' k, |f k|) = (∑' k, ∑ B, β B k * d B) := by
      refine tsum_congr (fun k => ?_)
      rw [hf k]
    _ = (∑' k, ∑ B ∈ (Finset.univ : Finset ι), β B k * d B) := rfl
    _ = (∑ B ∈ (Finset.univ : Finset ι), ∑' k, β B k * d B) := by
      rw [h_exchange]
    _ = (∑ B, ∑' k, β B k * d B) := rfl
    _ = (∑ B, (∑' k, β B k) * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      rw [tsum_mul_right]
    _ = (∑ B, (∑' k, |β B k|) * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      simp_rw [habs_β B]
    _ = (∑ B, ‖β B‖ * d B) := by
      refine Finset.sum_congr rfl (fun B _ => ?_)
      rw [hnorm_β B]

theorem cut_l1_vectors_summable_fsum {ι : Type*} (s : Finset ι) (f : ι → ℕ → ℝ)
    (hf : ∀ i, Summable (f i)) : Summable (fun k => ∑ i ∈ s, f i k) := by
  classical
  refine Finset.induction_on s ?_ ?_
  · simp only [Finset.sum_empty]
    exact summable_zero
```

```
  · intro a s ha ih
    simp only [Finset.sum_insert ha]
    exact (hf a).add ih

theorem cut_l1_vectors (n : ℕ) (x : Fin n → L1Space) :
    ∃ β : Finset (Fin n) → L1Space, (∀ B k, 0 ≤ β B k) ∧
      (∀ i j k, x i k = x j k + ∑ B, ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) * β B
↪  k) ∧
      (∀ i j k, |(x i - x j) k| = ∑ B, β B k * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else
↪  0)|) := by
  choose b hb0 hble hdiff habs using fun k : ℕ => cut_coord (fun i : Fin n => (x i : ℕ → ℝ) k)
  have hx : ∀ i, Summable (fun k => |(x i : ℕ → ℝ) k|) := by
    intro i
    have h := (memℓp_gen_iff (p := (1:ENNReal)) (by simp)).1 (lp.memℓp (x i))
    simpa [Real.norm_eq_abs] using h
  have hS : Summable (fun k => 2 * ∑ i, |(x i : ℕ → ℝ) k|) :=
    (cut_l1_vectors_summable_fsum Finset.univ (fun i k => |(x i : ℕ → ℝ) k|) hx).mul_left 2
  have hmem : ∀ B, Memℓp (fun k => b k B) 1 := by
    intro B
    apply (memℓp_gen_iff (p := (1:ENNReal)) (by simp)).2
    have hs : Summable (fun k => b k B) := by
      refine Summable.of_nonneg_of_le (fun k => hb0 k B) (fun k => ?_) hS
      exact hble k B
    have e : (fun k => ‖b k B‖ ^ (1:ENNReal).toReal) = fun k => b k B := by
      funext k
      simp [Real.norm_eq_abs, abs_of_nonneg (hb0 k B)]
    rw [e]
    exact hs
  obtain ⟨β, hβ⟩ : ∃ β : Finset (Fin n) → L1Space, ∀ B k, β B k = b k B :=
    ⟨fun B => ⟨fun k => b k B, hmem B⟩, fun _ _ => rfl⟩
  refine ⟨β, fun B k => by rw [hβ]; exact hb0 k B, ?_, ?_⟩
  · intro i j k
    have h : (x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k
        = ∑ B, b k B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) := hdiff k i j
    have e : ∑ B, ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) * β B k
        = ∑ B, b k B * ((if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)) :=
      Finset.sum_congr rfl (fun B _ => by rw [hβ, mul_comm])
    rw [e]
    linarith [h]
  · intro i j k
    have h : |(x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k|
        = ∑ B, b k B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| := habs k i j
    have hsub : (x i - x j) k = (x i : ℕ → ℝ) k - (x j : ℕ → ℝ) k := by
      first
      | rfl
      | simp [lp.coeFn_sub]
    rw [hsub, h]
    exact Finset.sum_congr rfl (fun B _ => by rw [hβ])

theorem cut_l1 (n : ℕ) (hn : 0 < n) (x : Fin n → L1Space) :
    ∃ (w : Finset (Fin n) → ℝ) (z : Fin n → Finset (Fin n) → ℝ)
      (T : (Finset (Fin n) → ℝ) → L1Space),
      (∀ B, 0 ≤ w B) ∧ (∀ i B, z i B = 0 ∨ z i B = 1) ∧ (∀ i, T (z i) = x i) ∧
      (∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|) ∧
      (∀ i j, ‖x i - x j‖ = ∑ B, w B * |z i B - z j B|) := by
  obtain ⟨β, hβ0, hβd, hβa⟩ := cut_l1_vectors n x
  obtain ⟨ind, hind⟩ : ∃ ind : Fin n → Finset (Fin n) → ℝ,
      ∀ i B, ind i B = if i ∈ B then (1:ℝ) else 0 := ⟨_, fun _ _ => rfl⟩
  let i0 : Fin n := ⟨0, hn⟩
```

```
  obtain ⟨T, hT⟩ : ∃ T : (Finset (Fin n) → ℝ) → L1Space,
      ∀ u, T u = x i0 + ∑ B, (u B - ind i0 B) • β B := ⟨_, fun _ => rfl⟩
  refine ⟨fun B => ‖β B‖, ind, T, fun B => norm_nonneg _, ?_, ?_, ?_, ?_⟩
  · intro i B
    rw [hind]
    split_ifs <;> simp
  · intro i
    rw [hT]
    apply cut_l1_ext
    intro k
    rw [cut_l1_apply]
    simp only [hind]
    exact (hβd i i0 k).symm
  · intro u u'
    rw [hT, hT, add_sub_add_left_eq_sub, ← Finset.sum_sub_distrib]
    refine (norm_sum_le _ _).trans (le_of_eq ?_)
    refine Finset.sum_congr rfl (fun B _ => ?_)
    rw [← sub_smul, sub_sub_sub_cancel_right, norm_smul, Real.norm_eq_abs, mul_comm]
  · intro i j
    exact cut_l1_norm_combo (x i - x j) β (fun B => |ind i B - ind j B|) hβ0
      (fun k => by simp only [hind]; exact hβa i j k)

theorem assembly {n : ℕ} {ι : Type*} [Fintype ι] (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A) (x : Fin n → L1Space)
    (w : ι → ℝ) (hw : ∀ B, 0 ≤ w B) (z : Fin n → ι → ℝ) (hz : ∀ i B, z i B = 0 ∨ z i B = 1)
    (T : (ι → ℝ) → L1Space) (hTz : ∀ i, T (z i) = x i)
    (hT : ∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|)
    (hdist : ∀ i j, ‖x i - x j‖ = ∑ B, w B * |z i B - z j B|)
    (q : ℝ) (hq0 : 0 ≤ q) (hq1 : q < 1) (τ : ℝ) (hτ : τ * (1 - q) = q)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (hGl : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, A i l * G l j)
    (hGr : ∀ i j, G i j = (1 - q) * (1 : Matrix (Fin n) (Fin n) ℝ) i j + q * ∑ l, G i l * A l j) :
    ∃ y : Fin n → L1Space,
      (∑ i, π i * ‖x i - y i‖ ^ 2) + τ * (∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2)
        ≤ 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by
  obtain ⟨h, hh⟩ := assembly_real π hπ A hA w hw z hz q hq0 hq1 τ hτ G hG0 hGrow hGπ hGl hGr
  refine ⟨fun i => T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3), ?_⟩
  have hq1' : 0 < 1 - q := by linarith
  have hτ0 : 0 ≤ τ := by
    by_contra hneg
    push_neg at hneg
    nlinarith [mul_neg_of_neg_of_pos hneg hq1']
  have h1 : ∀ i, ‖x i - T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)‖ ^ 2
      ≤ (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2 := by
    intro i
    have h' : ‖x i - T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)‖
        ≤ ∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)| := by
      have := hT (z i) (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3)
      rw [hTz i] at this
      exact this
    exact pow_le_pow_left₀ (norm_nonneg _) h' 2
  have h2 : ∀ i j, ‖T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3) - T (fun B => 3 * h j B ^ 2 - 2 * h j B ^ 3)‖ ^ 2
      ≤ (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2 := by
    intro i j
    have h' : ‖T (fun B => 3 * h i B ^ 2 - 2 * h i B ^ 3) - T (fun B => 3 * h j B ^ 2 - 2 * h j B ^ 3)‖
        ≤ ∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)| :=
      hT _ _
    exact pow_le_pow_left₀ (norm_nonneg _) h' 2
```

```
  calc _ ≤ (∑ i, π i * (∑ B, w B * |z i B - (3 * h i B ^ 2 - 2 * h i B ^ 3)|) ^ 2)
         + τ * (∑ i, ∑ j, π i * A i j *
             (∑ B, w B * |(3 * h i B ^ 2 - 2 * h i B ^ 3) - (3 * h j B ^ 2 - 2 * h j B ^ 3)|) ^ 2) :=
       add_le_add (Finset.sum_le_sum fun i _ => mul_le_mul_of_nonneg_left (h1 i) (hπ i))
         (mul_le_mul_of_nonneg_left
           (Finset.sum_le_sum fun i _ => Finset.sum_le_sum fun j _ =>
             mul_le_mul_of_nonneg_left (h2 i j) (mul_nonneg (hπ i) (hA.1 i j))) hτ0)
    _ ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ B, w B * |z i B - z j B|) ^ 2 := hh
    _ = 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by simp only [hdist]

theorem solution
    (n t : ℕ) (hn : 0 < n) (ht : 0 < t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochasticMatrix A)
    (hrev : IsReversibleWrt π A)
    (x : Fin n → L1Space) :
    ∃ y : Fin n → L1Space,
      (∑ i, π i * ‖x i - y i‖ ^ 2)
        + (t : ℝ) * (∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2)
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  obtain ⟨w, z, T, hw, hz, hTz, hT, hdist⟩ := cut_l1 n hn x
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t : ℝ) / ((t : ℝ) + 1) := ⟨_, rfl⟩
  have htpos : (0 : ℝ) < t := by exact_mod_cast ht
  have ht1 : (t : ℝ) + 1 ≠ 0 := by positivity
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hqt : q * ((t : ℝ) + 1) = t := by rw [hq]; field_simp
  have hq1 : q < 1 := by nlinarith [hqt, htpos]
  have hτ : (t : ℝ) * (1 - q) = q := by linear_combination (-1 : ℝ) * hqt
  obtain ⟨G, hG⟩ : ∃ G : Matrix (Fin n) (Fin n) ℝ,
      ∀ i j, G i j = ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j :=
    ⟨fun i j => ∑' s : ℕ, (1 - q) * q ^ s * (A ^ s) i j, fun i j => rfl⟩
  obtain ⟨y, hy⟩ := assembly π hπ.1 A hA x w hw z hz T hTz hT hdist q hq0 hq1 (t : ℝ) hτ G
    (res_nonneg A hA q hq0 hq1 G hG) (res_rowsum A hA q hq0 hq1 G hG)
    (res_stationary π A hA hrev q hq0 hq1 G hG) (res_left A hA q hq0 hq1 G hG)
    (res_right A hA q hq0 hq1 G hG)
  have hcomp := res_comparison t ht π hπ A hA hrev x q hq G hG
  refine ⟨y, ?_⟩
  linarith
```